

Memory Hierarchy Design Advances

ten advanced optimizations

Ten Advanced Cache Opts

- Goal: average memory access time 
- Metrics to reduce/optimize
 - hit time
 - miss rate
 - miss penalty
 - cache bandwidth
 - power consumption

Ten Advanced Cache Opts

- **Reduce hit time**
small and simple first-level caches;
way prediction;
decrease power;
- **Increase cache bandwidth**
pipelined/multibanked/nonblocking cache;
- **Reduce miss penalty**
critical word first;
merging write buffers;
- **Reduce miss rate**
compiler optimizations; *decrease power;*
- **Reduce miss penalty/rate via parallelism**
hardware/compiler prefetching; *increase power;*

Opt #1: Small and Simple First-Level Caches

- **Small size**

support a fast clock cycle

reduce power

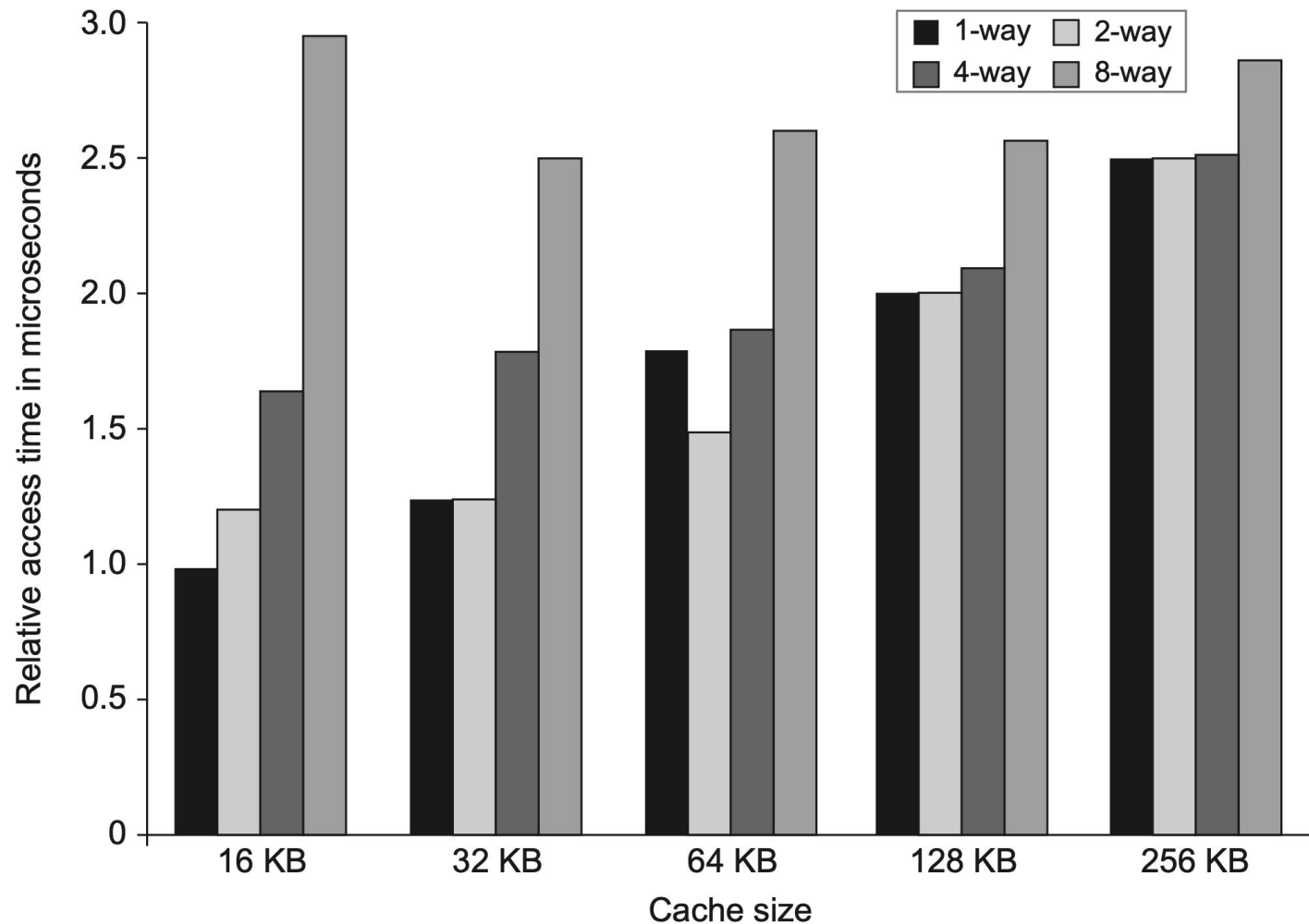
- **Lower associativity**

reduce both hit time and power

(direct-mapped caches can overlap the tag check with the transmission of the data, there's only one piece of data for each index)

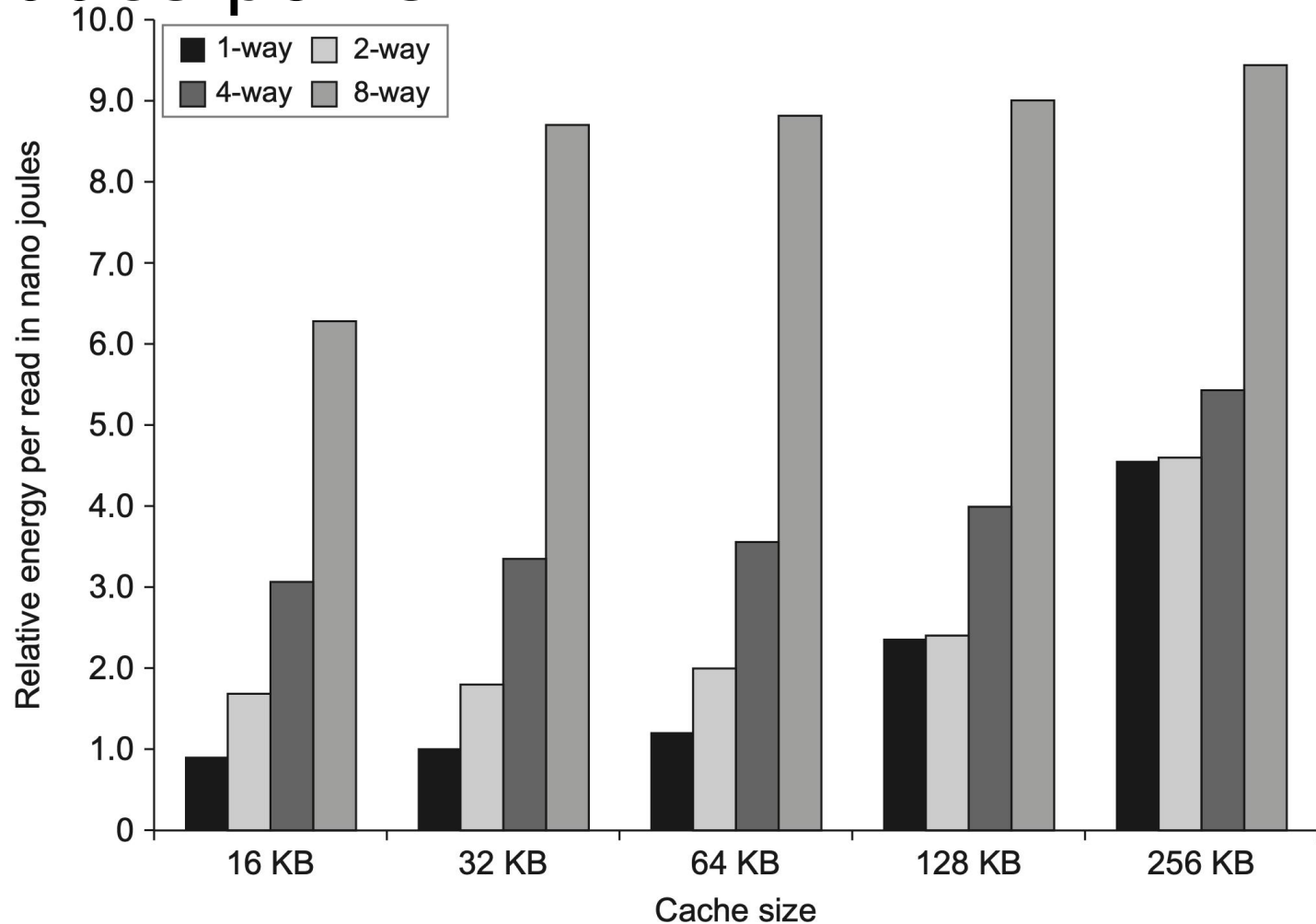
Opt #1: Small and Simple First-Level Caches

- Reduce hit time



Opt #1: Small and Simple First-Level Caches

- Reduce power



Opt #2: Way Prediction

- Reduce conflict misses and hit time
- **Way prediction**
block predictor bits are added to each block to predict the way/block within the set of the *next* cache access

the multiplexor is set **early to select the desired block**;

only a single tag comparison is performed **in parallel with cache reading**;

a miss results in checking the other blocks for matches in the next clock cycle;

way prediction

- Used in a cache with more than one block in a set
- i.e., in any set-associative cache, fully associative cache

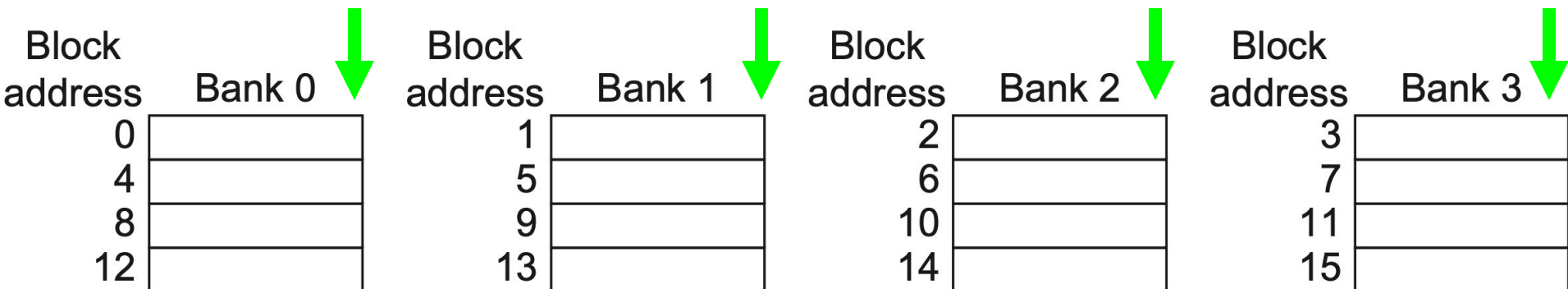
Opt #3: Pipelined Access

- Increase cache bandwidth
- Higher latency
- Greater penalty on mispredicted branches and more clock cycles between issuing load and using data

Opt #3: Multibanked Caches

- Increase cache bandwidth
- Divide cache into independent banks that support **simultaneous accesses**
- **Sequential interleaving**

spread the addresses of blocks sequentially across the banks



Opt #4: Nonblocking Caches

- Increase cache bandwidth
- **Nonblocking/lockup-free cache**
leverage out-of-order execution

allows data cache to continue to supply
cache hits during a miss;

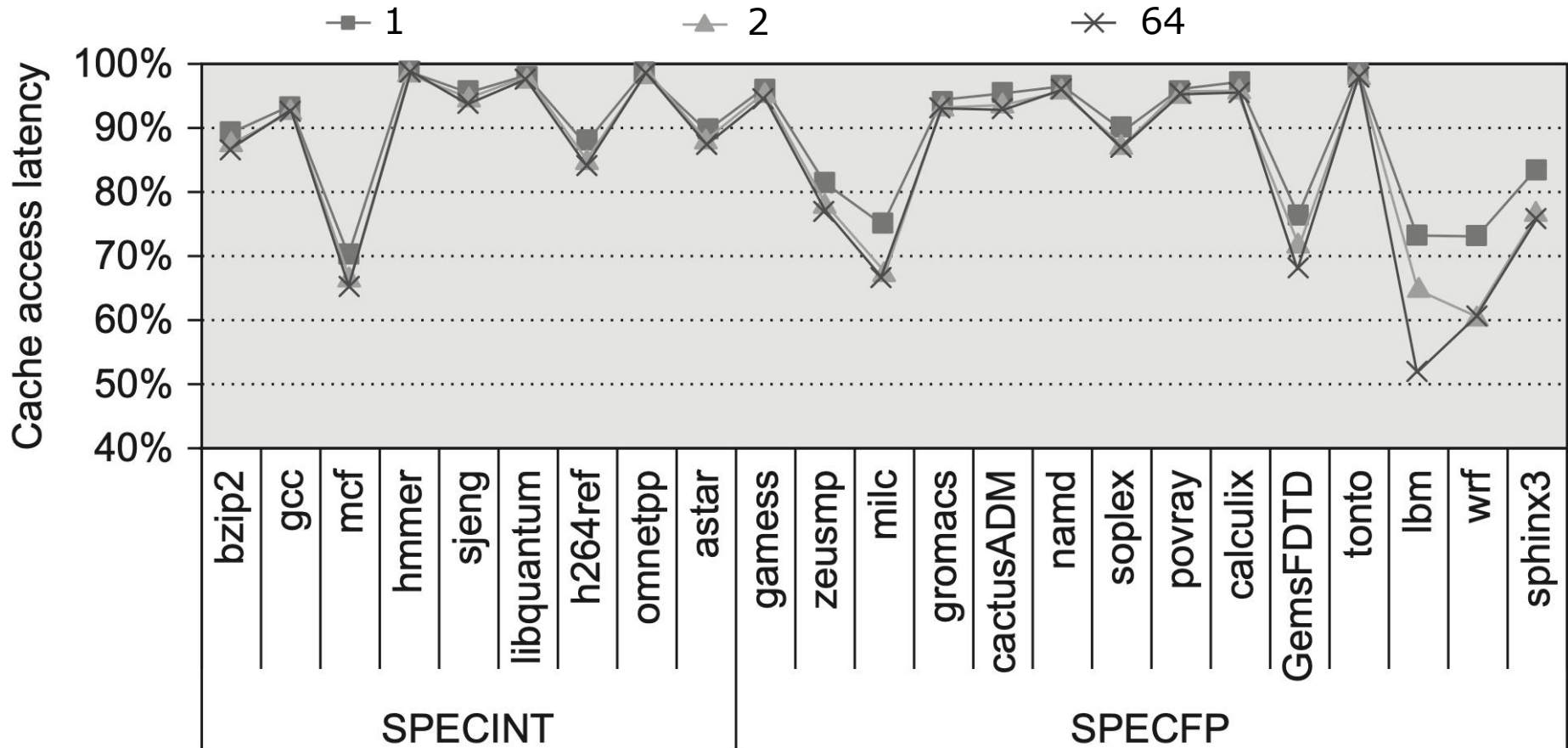
hit under miss;

miss under miss;

hit under multiple misses;

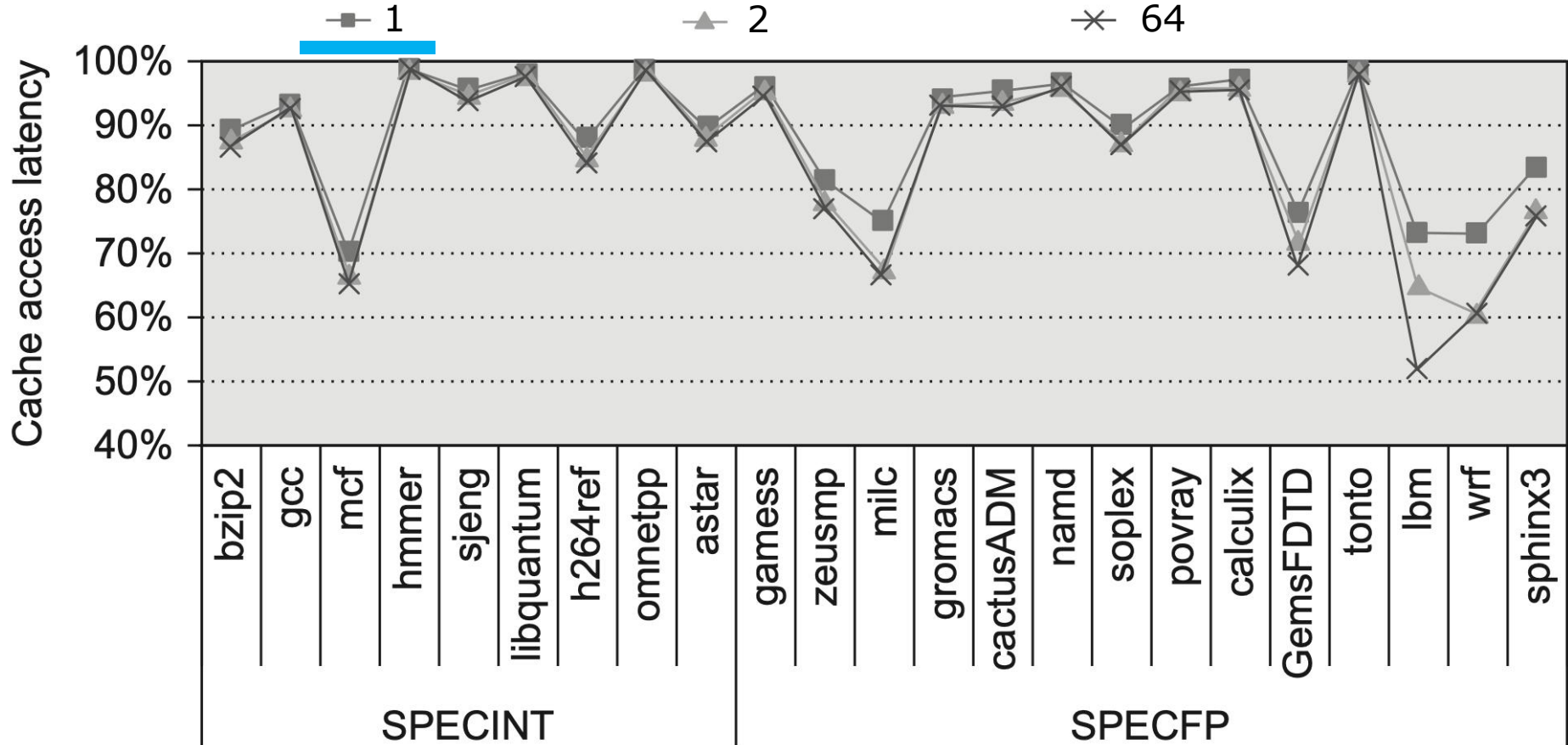
Opt #4: Nonblocking Caches

- Performance improvement by allowing 1, 2, and 64 hits under a cache miss



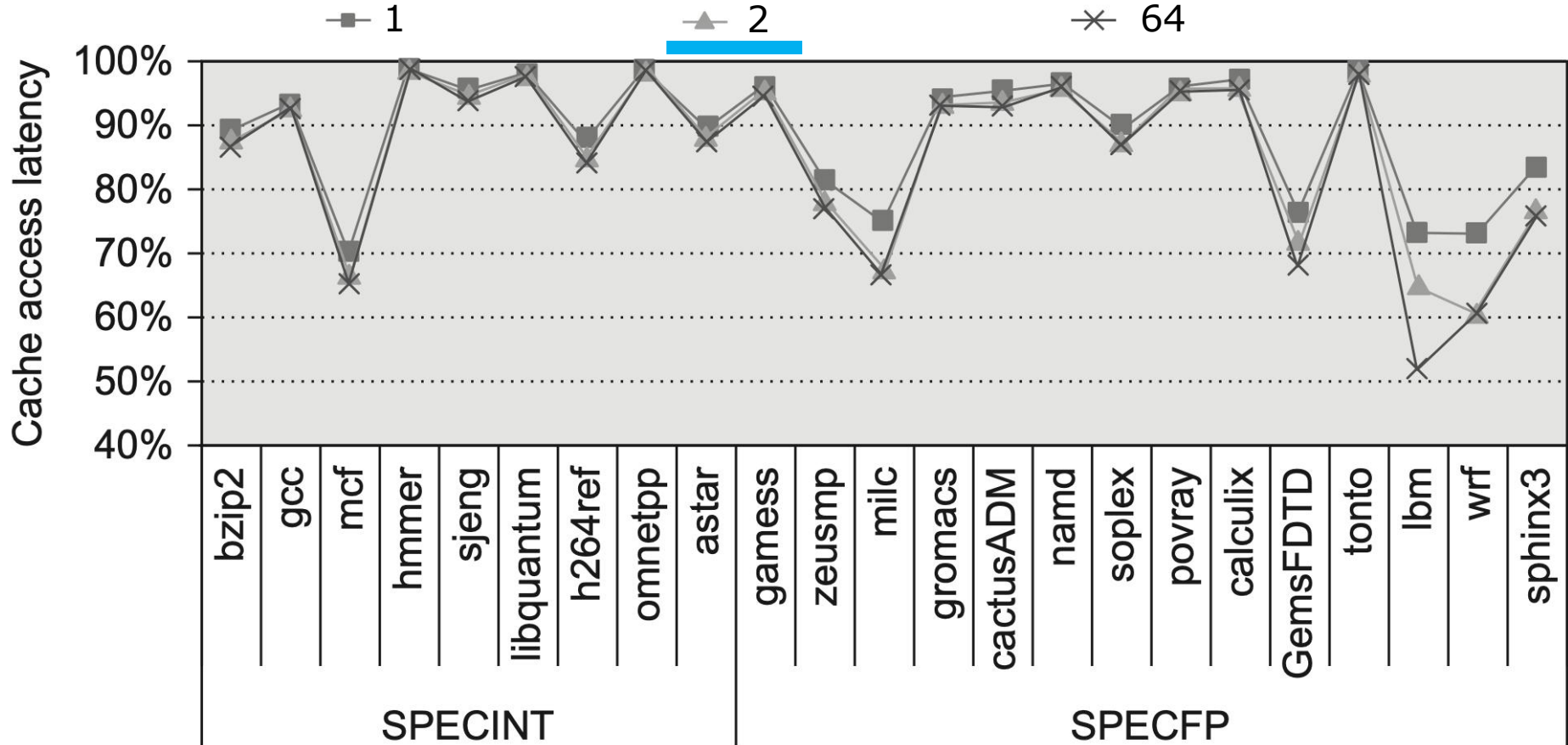
Opt #4: Nonblocking Caches

- Integer benchmarks: 9%
- Floating-point benchmarks: 12.5%



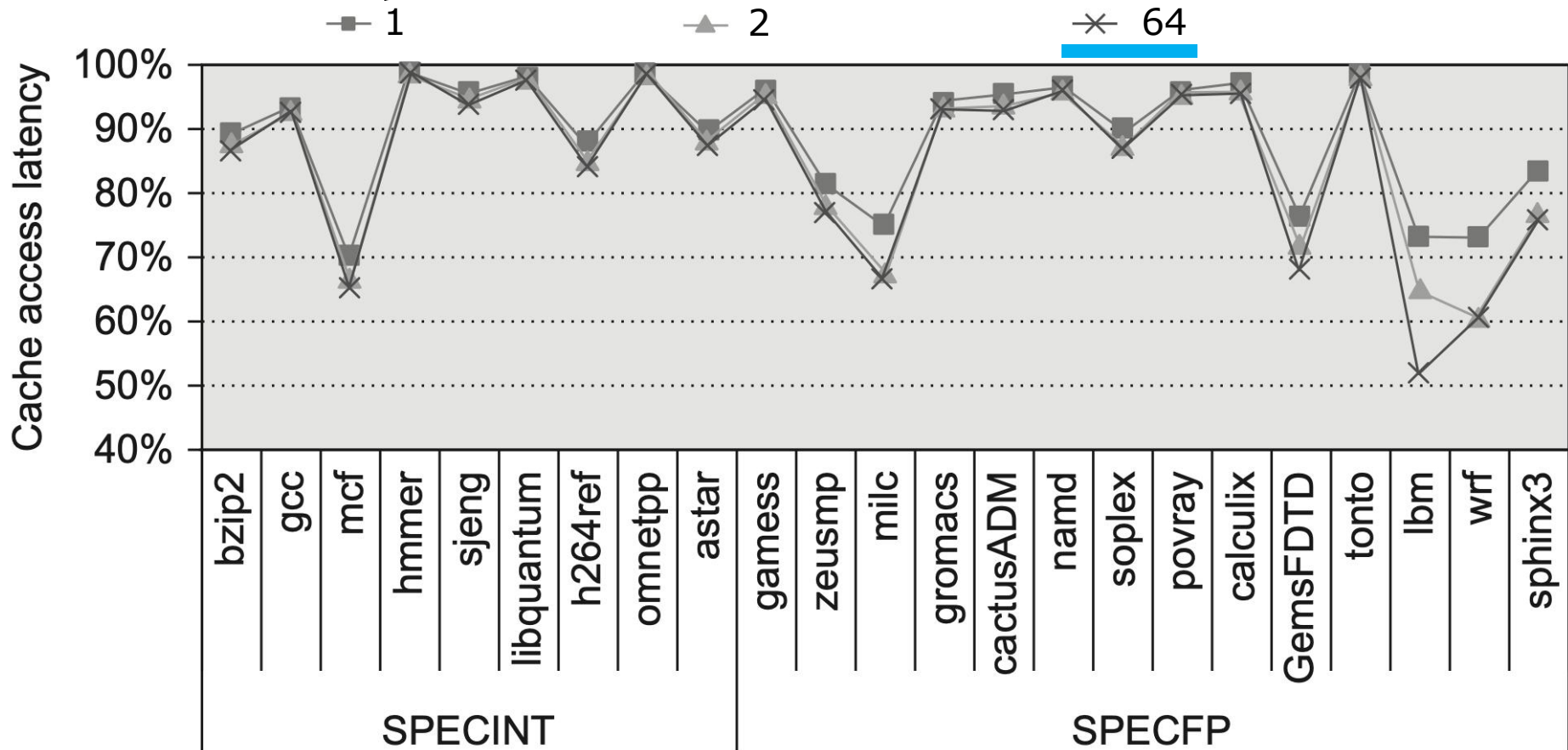
Opt #4: Nonblocking Caches

- Integer benchmarks: 10%
Floating-point benchmarks: 16%



Opt #4: Nonblocking Caches

- Integer benchmarks: ~10%
Floating-point benchmarks: ~16%



Opt #4: Nonblocking Caches

- **Example:** 32 KiB L1 data cache
miss penalty to L2 is 10 cycles
miss rates:

	direct-mapped	2-way set assoc.
integer	3.5%	3.2%
floating-point	5.2%	4.9%

- Two-way set associativity or hit under one miss improves more?

Opt #4: Nonblocking Caches

- **Example:** 32 KiB L1 data cache
miss penalty to L2 is 10 cycles
miss rates:

	direct-mapped	2-way set assoc.
integer	3.5%	3.2%
floating-point	5.2%	4.9%

- Two-way set associativity or hit under one miss improves more?

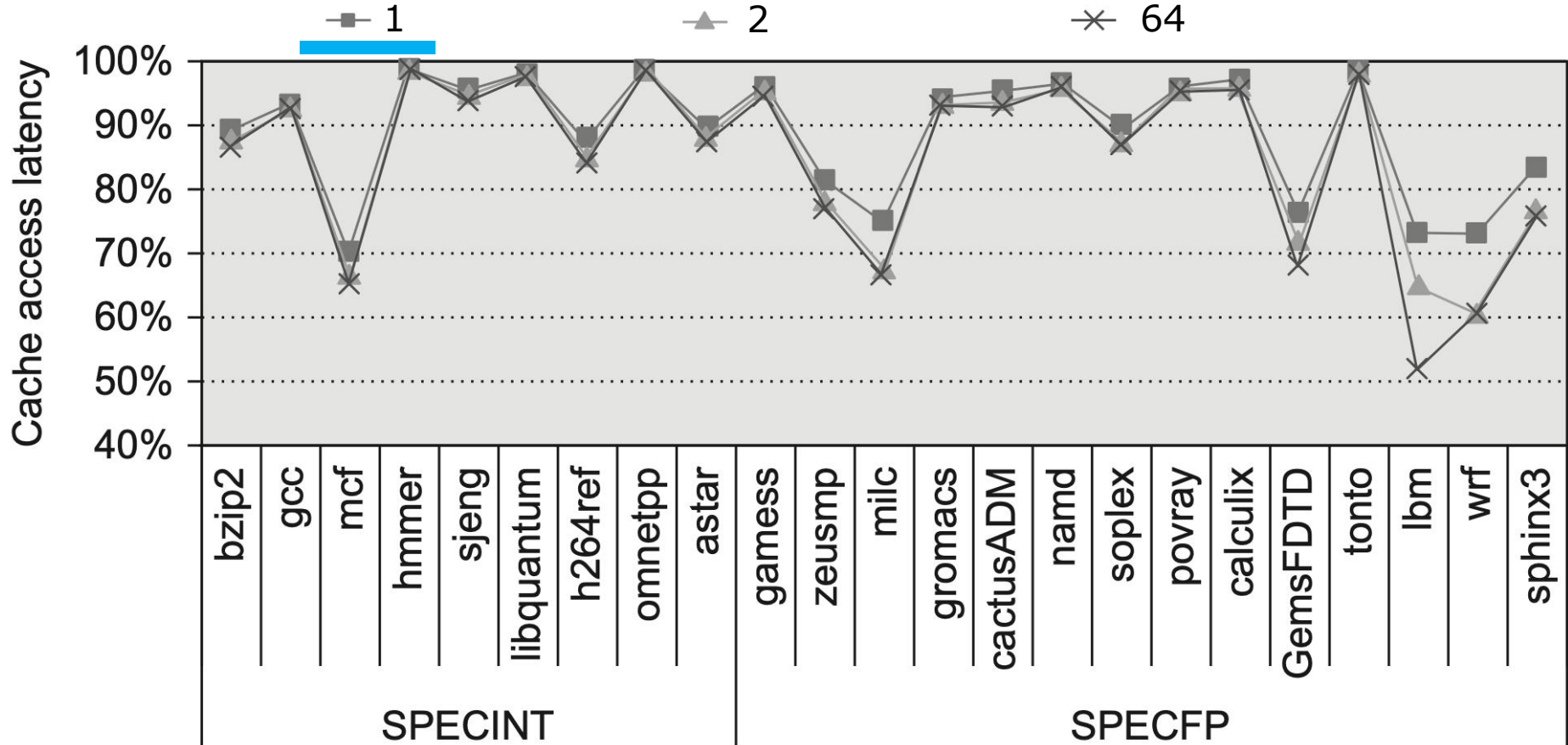
$0.32/0.35 = 91\% \rightarrow 9\% \text{ improvement}$

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 3.5\% \times 10 = 0.35$$

$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 3.2\% \times 10 = 0.32$$

Opt #4: Nonblocking Caches

- Integer benchmarks: 9%
- Floating-point benchmarks: 12.5%



Opt #4: Nonblocking Caches

- **Example:** 32 KiB L1 data cache
miss penalty to L2 is 10 cycles
miss rates:

	direct-mapped	2-way set assoc.
integer	3.5%	3.2%
floating-point	5.2%	4.9%

- Two-way set associativity or hit under one miss improves more? **about equal**
 $0.32/0.35 = 91\% \rightarrow 9\%$ improvement

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 3.5\% \times 10 = 0.35$$

$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 3.2\% \times 10 = 0.32$$

Opt #4: Nonblocking Caches

- **Example:** 32 KiB L1 data cache
miss penalty to L2 is 10 cycles
miss rates:

	direct-mapped	2-way set assoc.
integer	3.5%	3.2%
floating-point	5.2%	4.9%

- Two-way set associativity or hit under one miss improves more?

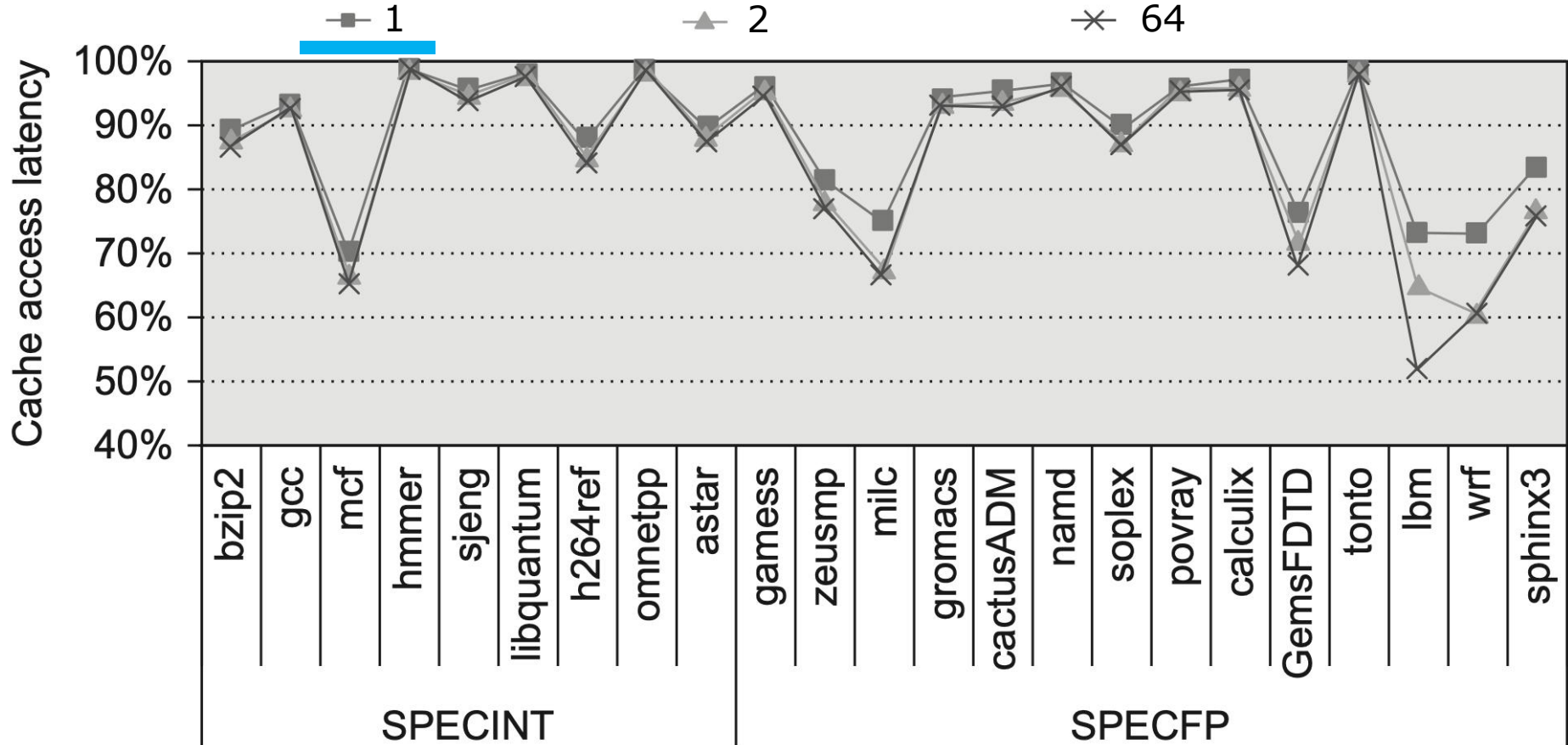
$0.49/0.52 = 94\% \rightarrow 6\%$ improvement

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 5.2\% \times 10 = 0.52$$

$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 4.9\% \times 10 = 0.49$$

Opt #4: Nonblocking Caches

- Integer benchmarks: 9%
Floating-point benchmarks: 12.5%



Opt #4: Nonblocking Caches

- **Example:** 32 KiB L1 data cache
miss penalty to L2 is 10 cycles
miss rates:

	direct-mapped	2-way set assoc.
integer	3.5%	3.2%
floating-point	5.2%	4.9%

- Two-way set associativity or **hit under one miss improves more?**

$0.49/0.52 = 94\% \rightarrow 6\%$ improvement

$$\text{Miss rate}_{\text{DM}} \times \text{Miss penalty} = 5.2\% \times 10 = 0.52$$

$$\text{Miss rate}_{\text{2-way}} \times \text{Miss penalty} = 4.9\% \times 10 = 0.49$$

Opt #5: Critical Word First

- Reduce miss penalty
- Motivation: processor normally needs just one word of the block at a time
- **Critical word first**
request the missed word first from the memory and send it to the processor as soon as it arrives;
processor continues execution while filling the rest of the words in the block

Opt #5: Early Restart

- Reduce miss penalty
- Motivation: processor normally needs just one word of the block at a time
- **Early restart**
fetch the words in normal order,
as soon as the requested word arrives
send it to the processor
let the processor continue execution

Opt #6: Merging Write Buffer

- Reduce miss penalty

Write address	V	V	V	V		
100	1	Mem[100]	0	0	0	
108	1	Mem[108]	0	0	0	
116	1	Mem[116]	0	0	0	
124	1	Mem[124]	0	0	0	

performing merges four entries

- Write merging merges four entries (with sequential addresses) into a single buffer entry

Write address

100

1

Mem[100]

1

Mem[108]

1

Mem[116]

1

Mem[124]

0

0

0

0

0

0

0

0

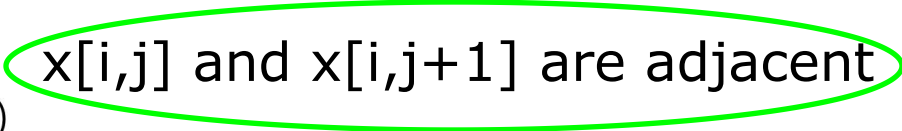
Opt #7: Compiler Optimization

- Reduce miss rates, w/o hw changes
- **Tech 1: Loop interchange**

exchange the nesting of the loops to make the code access the data in the order in which they are stored

/* Before */

```
for (j = 0; j < 100; j = j + 1)
    for (i = 0; i < 5000; i = i + 1)
        x[i][j] = 2 * x[i][j];
```

x[i,j] and x[i,j+1] are adjacent

/* After */

```
for (i = 0; i < 5000; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        x[i][j] = 2 * x[i][j];
```

Opt #7: Compiler Optimization

- Reduce miss rates, w/o hw changes
- **Example: matrix multiplication**

*$x = y * z$; both row&column accesses*

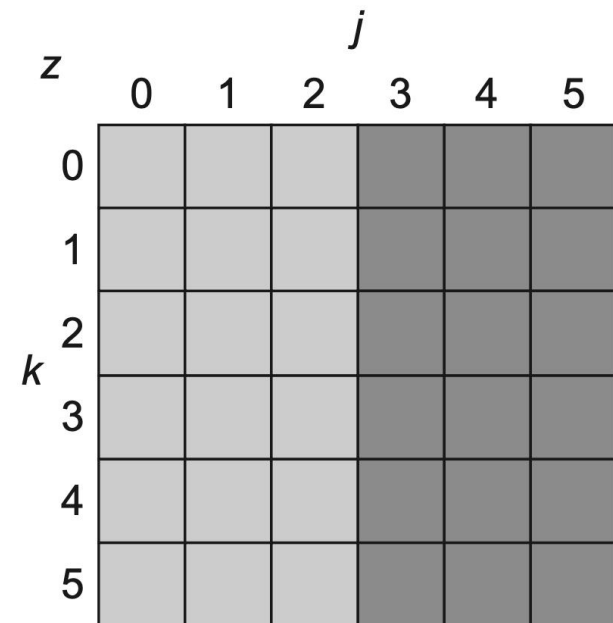
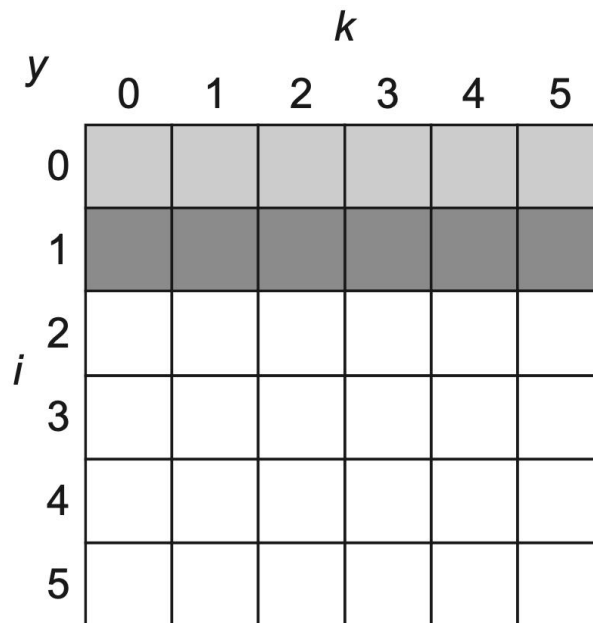
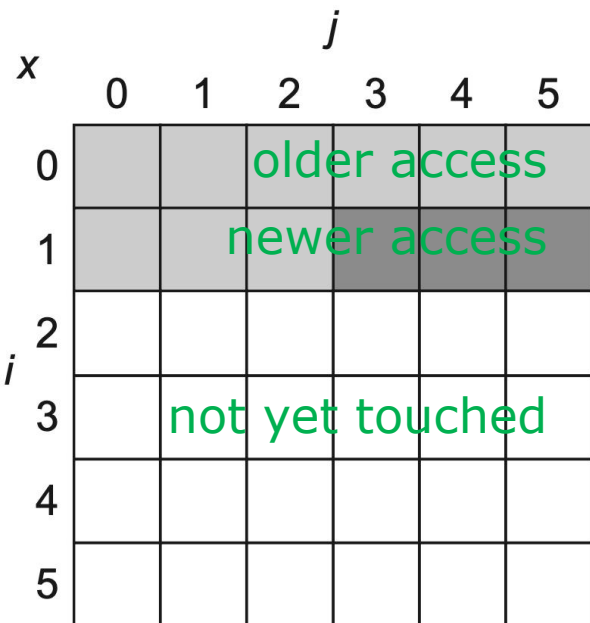
/ Before */*

```
for (i = 0; i < N; i = i + 1)
    for (j = 0; j < N; j = j + 1)
        { r = 0;
          for (k = 0; k < N; k = k + 1)
              r = r + y[i][k]*z[k][j];
          x[i][j] = r;
        };
```

Opt #7: Compiler Optimization

- Reduce miss rates, w/o hw changes
- **Tech 2: Blocking**

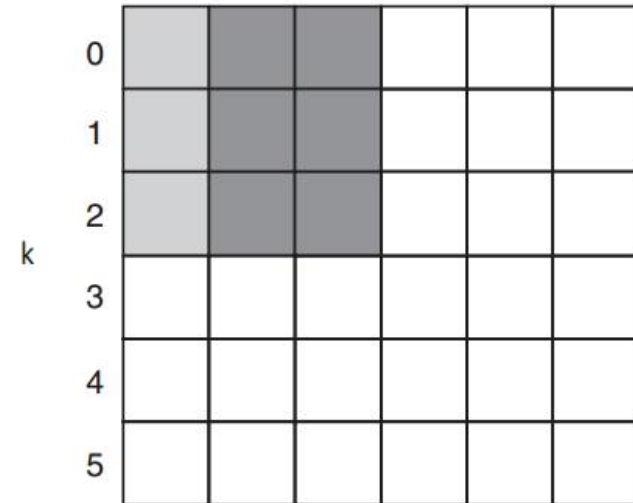
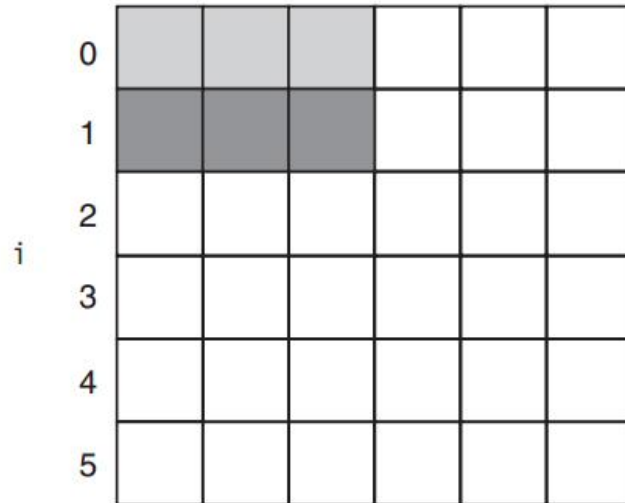
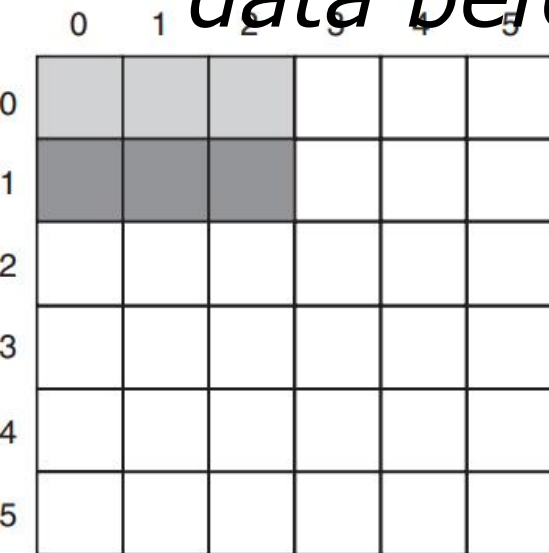
$x = y * z$; *both row&column accesses*
before



Opt #7: Compiler Optimization

- Reduce miss rates, w/o hw changes
- **Tech 2: Blocking**

$x = y * z$; compute using *submatrices*
after; maximize accesses to loaded
data before they are replaced

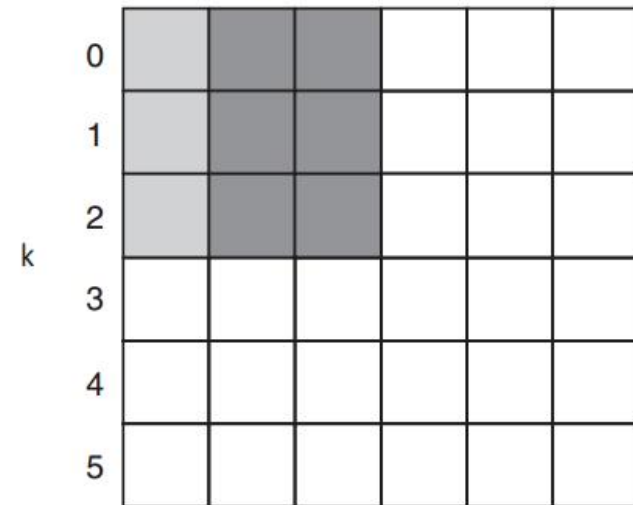
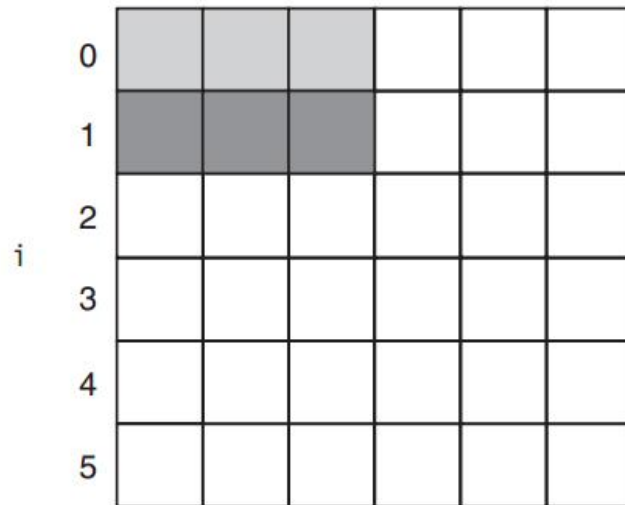
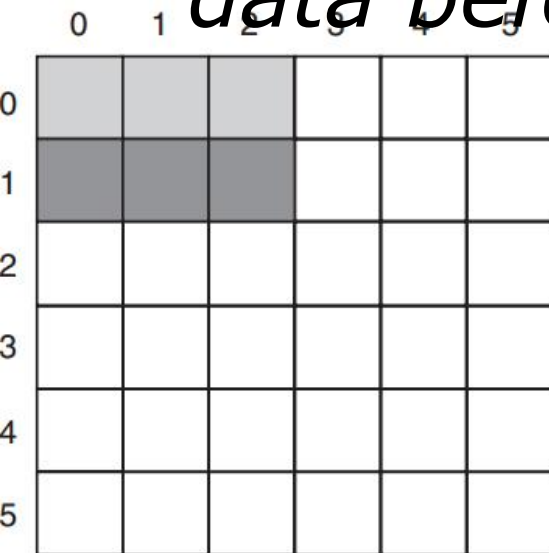


Opt #7: Compiler Optimization

- Reduce miss rates, w/o hw changes
- **Tech 2: Blocking**

$x = y * z$; compute using *blocks*

after; maximize accesses to loaded data before they are replaced



Opt #8: Hardware Prefetching

- Reduce miss penalty/rate
- Prefetch items before the processor requests them, *into the cache or external buffer*

Opt #8: Hardware Prefetching

- **Instruction prefetch**

fetch two blocks on a miss:

requested one *into cache*

next consecutive one *into instruction stream buffer*

- If requested block is in stream buffer:
 - cancel the original cache request
 - read the block from stream buffer
 - issue next prefetch request

Opt #8: Hardware Prefetching

- **Instruction prefetch**

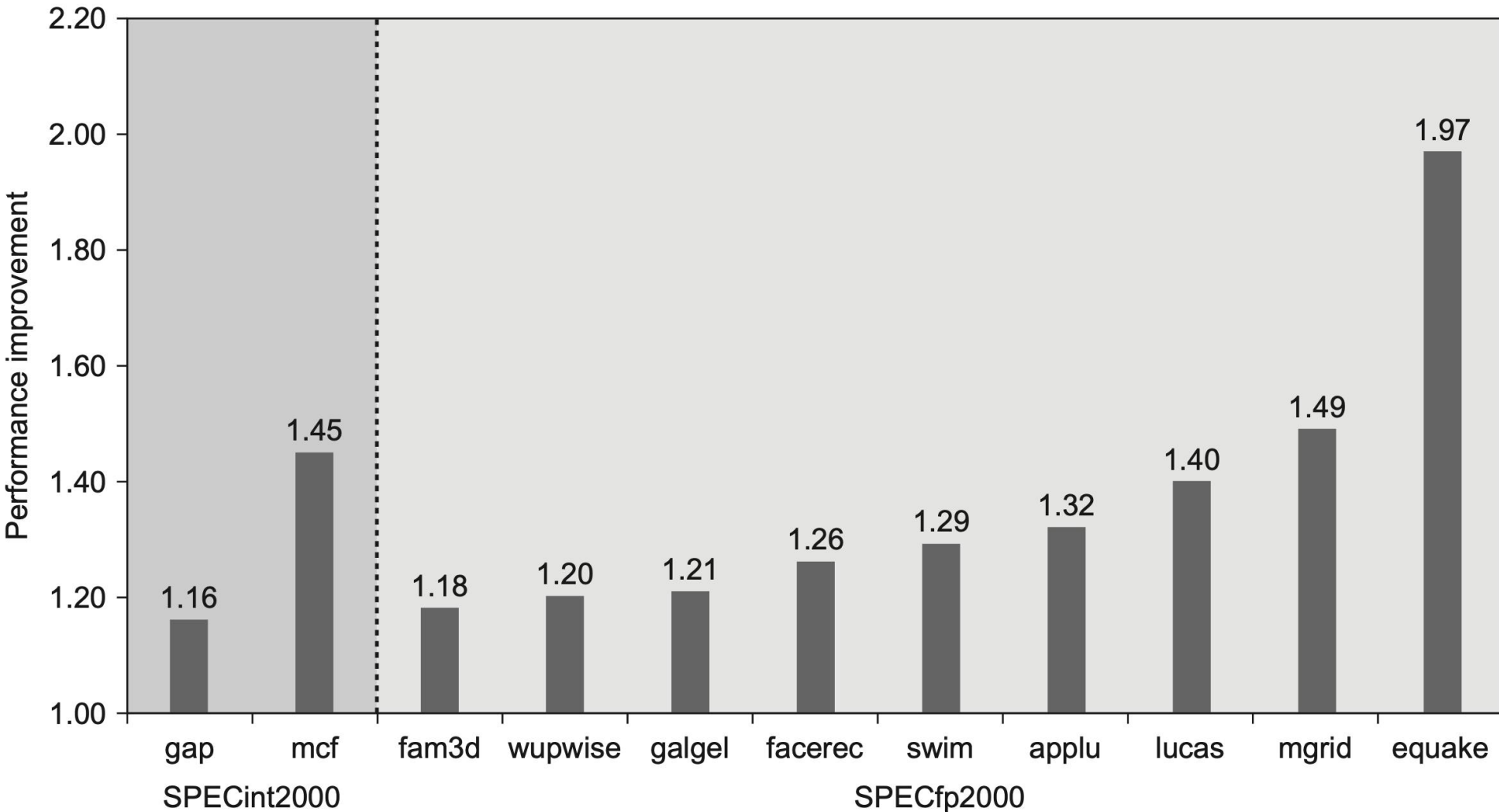
fetch two blocks on a miss:

requested one *into cache*

next consecutive one *into instruction stream buffer*

- Similar **Data prefetch** approaches

Opt #8: Hardware Prefetching



Opt #9: Compiler Prefetching

- Reduce miss penalty/rate
- Compiler to insert prefetch instructions to request data before the processor needs it
- **Register prefetch**
load the value into a register
- **Cache prefetch**
load data into the cache

Opt #9: Compiler Prefetching

- **Example**

```
for (i = 0; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        a[i][j] = b[j][0] * b[j + 1][0];
```

16-byte blocks;

8-byte elements for a and b;

write-back strategy;

a[0][0] miss, copy both a[0][0], a[0][1]
as one block contains $16/8 = 2$;

Opt #9: Compiler Prefetching

- **Example**

```
for (i = 0; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        a[i][j] = b[j][0] * b[j + 1][0];
```

16-byte blocks;

8-byte elements for a and b;

write-back strategy;

a[0][0] miss, copy both a[0][0], a[0][1]
as one block contains $16/8 = 2$;

how many misses?

Opt #9: Compiler Prefetching

- **Example: 251 misses**

```
for (i = 0; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        a[i][j] = b[j][0] * b[j + 1][0];
```

16-byte blocks;

8-byte elements for a and b;

write-back strategy;

a[0][0] miss, copy both a[0][0], a[0][1]
as one block contains $16/8 = 2$;

so for **a**: $3 \times (100/2) = \mathbf{150 \text{ misses}}$

b[0][0] – b[100][0]: 101 misses

Opt #9: Compiler Prefetching

- **Example: ? misses by prefetching**

```
for (j = 0; j < 100; j = j + 1) {  
    prefetch(b[j + 7][0]); 7 misses: b[0][0] – b[6][0]  
    /* b(j,0) for 7 iterations later */  
    prefetch(a[0][j + 7]); 4 misses: 1/2 of a[0][0] – a[0][6]  
    /* a(0,j) for 7 iterations later */  
    a[0][j] = b[j][0] * b[j + 1][0];  
}  
  
for (i = 1; i < 3; i = i + 1)  
    for (j = 0; j < 100; j = j + 1) {  
        prefetch(a[i][j + 7]); 4 misses: a[1][0] – a[1][6]  
        prefetch(a[i][j + 7]); 4 misses: a[2][0] – a[2][6]  
        /* a(i,j) for +7 iterations */  
        a[i][j] = b[j][0] * b[j + 1][0];  
    }
```


Opt #9: Compiler Prefetching

- **Example: 19 misses by prefetching**

```
for (j = 0; j < 100; j = j + 1) {  
    prefetch(b[j + 7][0]); 7 misses: b[0][0] – b[6][0]  
    /* b(j,0) for 7 iterations later */  
    prefetch(a[0][j + 7]); 4 misses: 1/2 of a[0][0] – a[0][6]  
    /* a(0,j) for 7 iterations later */  
    a[0][j] = b[j][0] * b[j + 1][0];  
}  
for (i = 1; i < 3; i = i + 1)  
    for (j = 0; j < 100; j = j + 1) { 4 misses: a[1][0] – a[1][6]  
        prefetch(a[i][j + 7]); 4 misses: a[2][0] – a[2][6]  
        /* a(i,j) for +7 iterations */  
        a[i][j] = b[j][0] * b[j + 1][0];  
    }
```

Opt #9: Compiler Prefetching

- **Example**

original loop: 7 CC per iteration

first prefetch loop: 9 CC per iteration

second prefetch loop: 8 CC per iteration

cache miss: 100 CC

- **Question**

calculate the time saved by prefetching

assuming prefetches are completely

overlapped with the rest of execution

Opt #9: Compiler Prefetching

- **Example**

```
for (i = 0; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        a[i][j] = b[j][0] * b[j + 1][0];
```

3 x 100 = 300 iterations

251 misses

Opt #9: Compiler Prefetching

- **Example**

```
for (i = 0; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        a[i][j] = b[j][0] * b[j + 1][0];
```

3 x 100 = 300 iterations

251 misses

$300 \times 7 + 251 \times 100 = 27200$ CC

Opt #9: Compiler Prefetching

- **Example**

```
for (j = 0; j < 100; j = j + 1) { 100 iterations
    prefetch(b[j + 7][0]); 11 misses
    /* b(j,0) for 7 iterations later */
    prefetch(a[0][j + 7]);
    /* a(0,j) for 7 iterations later */
    a[0][j] = b[j][0] * b[j + 1][0];};
for (i = 1; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1) { 200 iterations
        prefetch(a[i][j + 7]); 8 misses
        /* a(i,j) for +7 iterations */
        a[i][j] = b[j][0] * b[j + 1][0];}
```

Opt #9: Compiler Prefetching

- **Example**

```
for (j = 0; j < 100; j = j + 1) { 900 CC
    prefetch(b[j + 7][0]); 1100 CC
    /* b(j,0) for 7 iterations later */
    prefetch(a[0][j + 7]);
    /* a(0,j) for 7 iterations later */
    a[0][j] = b[j][0] * b[j + 1][0];};
for (i = 1; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1) { 1600 CC
        prefetch(a[i][j + 7]); 800 CC
        /* a(i,j) for +7 iterations */
        a[i][j] = b[j][0] * b[j + 1][0];}
```

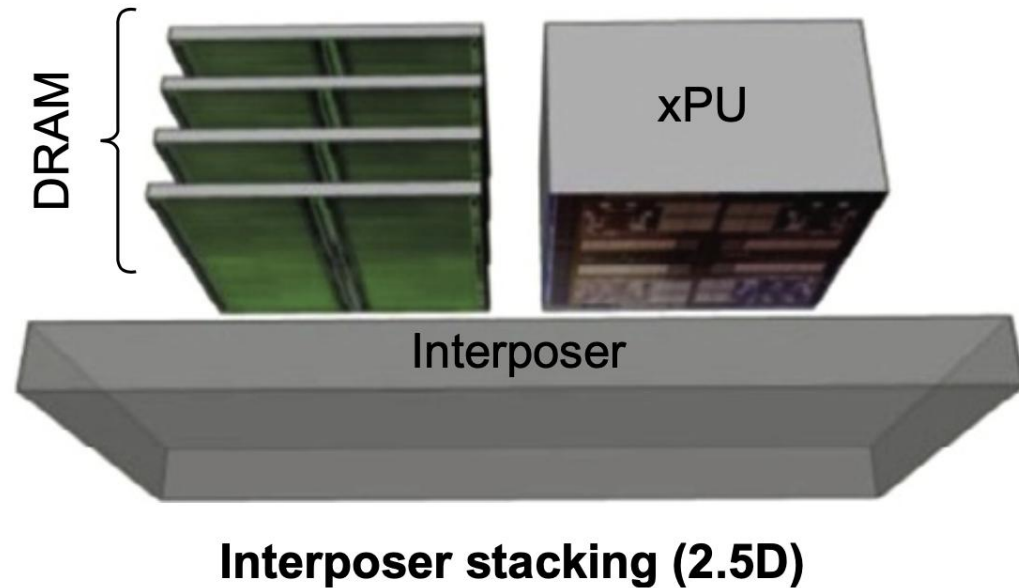
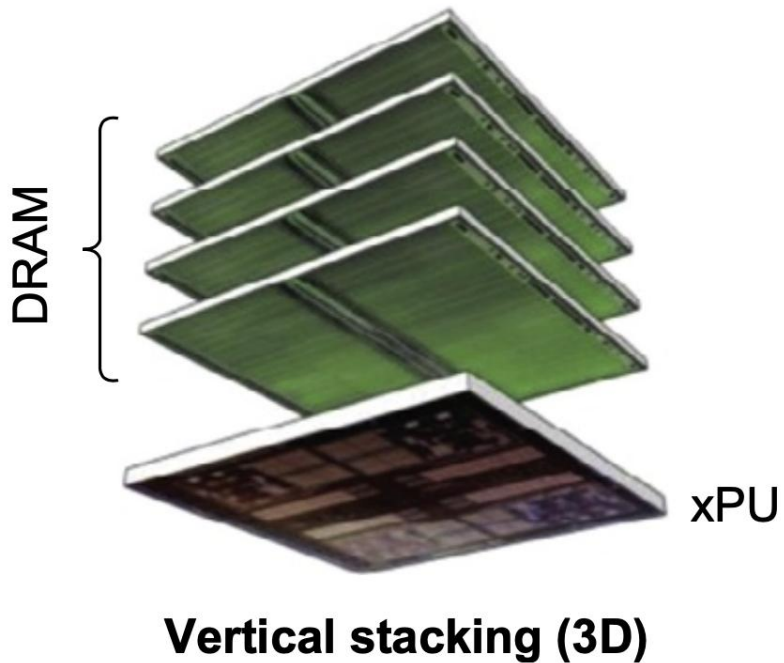
Opt #9: Compiler Prefetching

- **Example: 4400 clock cycles**

```
for (j = 0; j < 100; j = j + 1) { 900 CC
    prefetch(b[j + 7][0]); 1100 CC
    /* b(j,0) for 7 iterations later */
    prefetch(a[0][j + 7]);
    /* a(0,j) for 7 iterations later */
    a[0][j] = b[j][0] * b[j + 1][0];};
for (i = 1; i < 3; i = i + 1)
    for (j = 0; j < 100; j = j + 1) { 1600 CC
        prefetch(a[i][j + 7]); 800 CC
        /* a(i,j) for +7 iterations */
        a[i][j] = b[j][0] * b[j + 1][0];}
```

Opt #10: HBM

- Use HBM as L4 cache



Opt #10: HBM

- Use HBM as L4 cache
- Potentially large volume of tags
given 1 GiB L4 cache
64B block size → 96 MiB of tags
4 KiB block size → 1 MiB of tags

Opt #10: HBM

- Use HBM as L4 cache
- Potentially large volume of tags
given 1 GiB L4 cache
64B block size → 96 MiB of tags
4 KiB block size → 1 MiB of tags
- Require two accesses to DRAM for each L4 access
one for tags (address translation)
one for data (tag comparison)

Opt #10: HBM

- Optimization 1: place tags and data in the same row

albeit a long time to open a row

1/3 time to access an opened row;

access tag first, then data if tag hit;

Opt #10: HBM

- Optimization 2: alloy cache
mold tag and data together
direct mapped cache structure
single HBM cycle for access: directly
index HBM cache and do a burst
transfer of both tag and data

Opt #10: HBM

- HBM miss requires two DRAM accesses

one to get the initial tag

one to get data in the main memory

- Speed up miss detection to reduce miss time

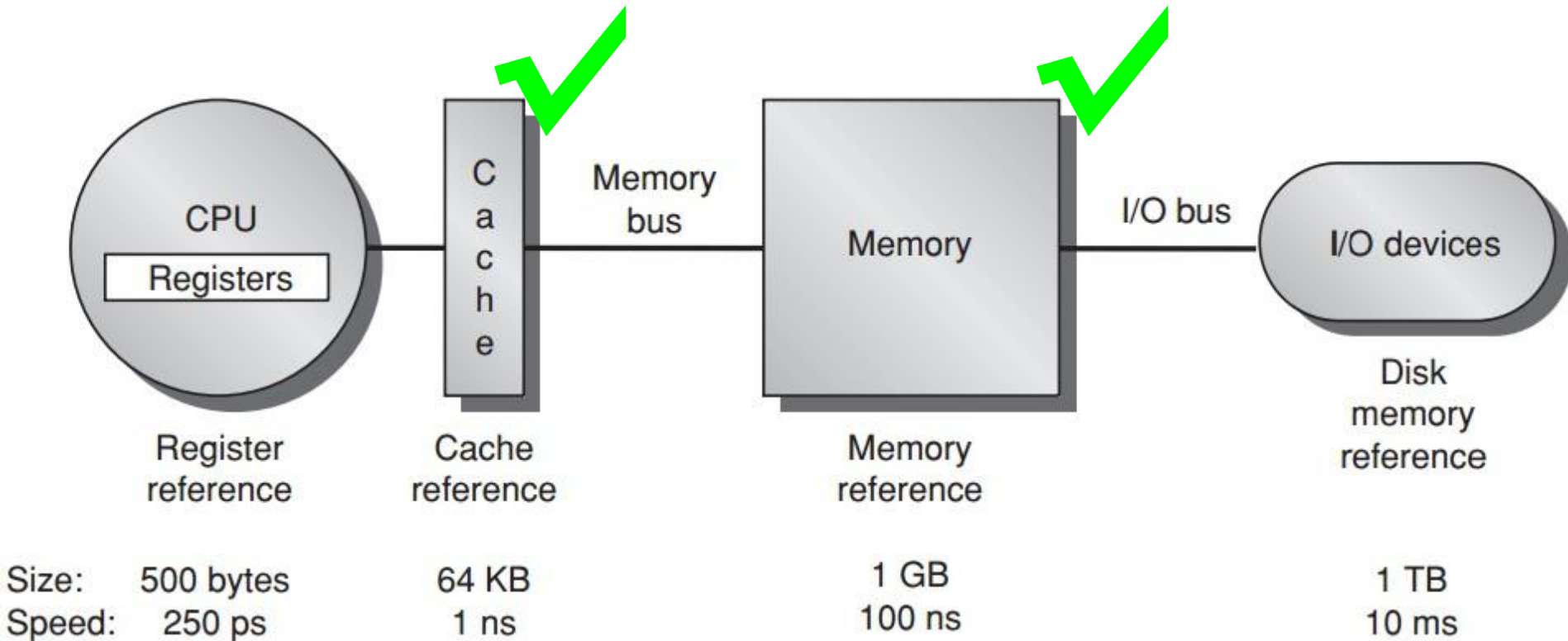
use a map to track where a block is present in cache

use memory access predictor to predict likely misses using access history

Summary of Cache Opt

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined cache access	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Banked caches		+			+	1	Used in L2 of both i7 and Cortex-A8
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware.
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs

Optimization

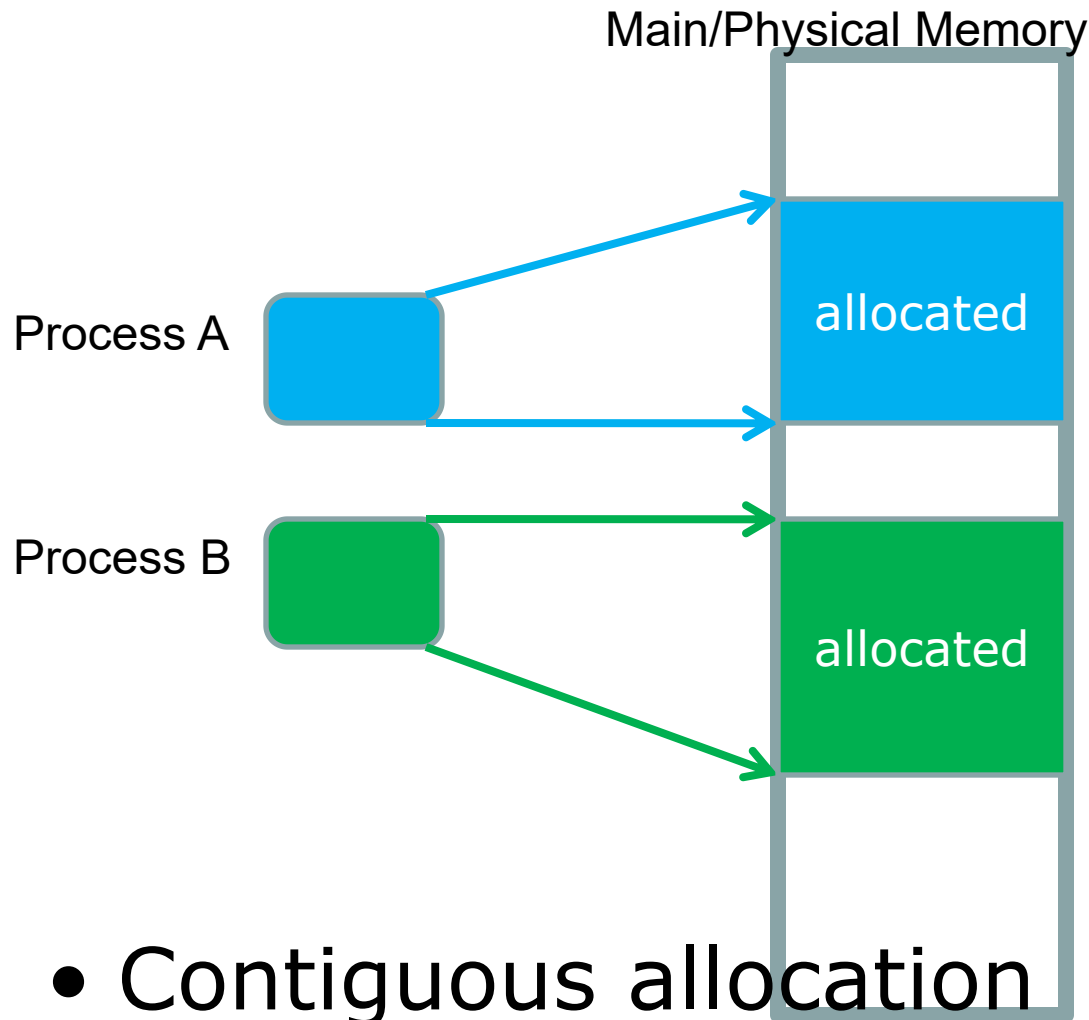


performance so far

on to protection

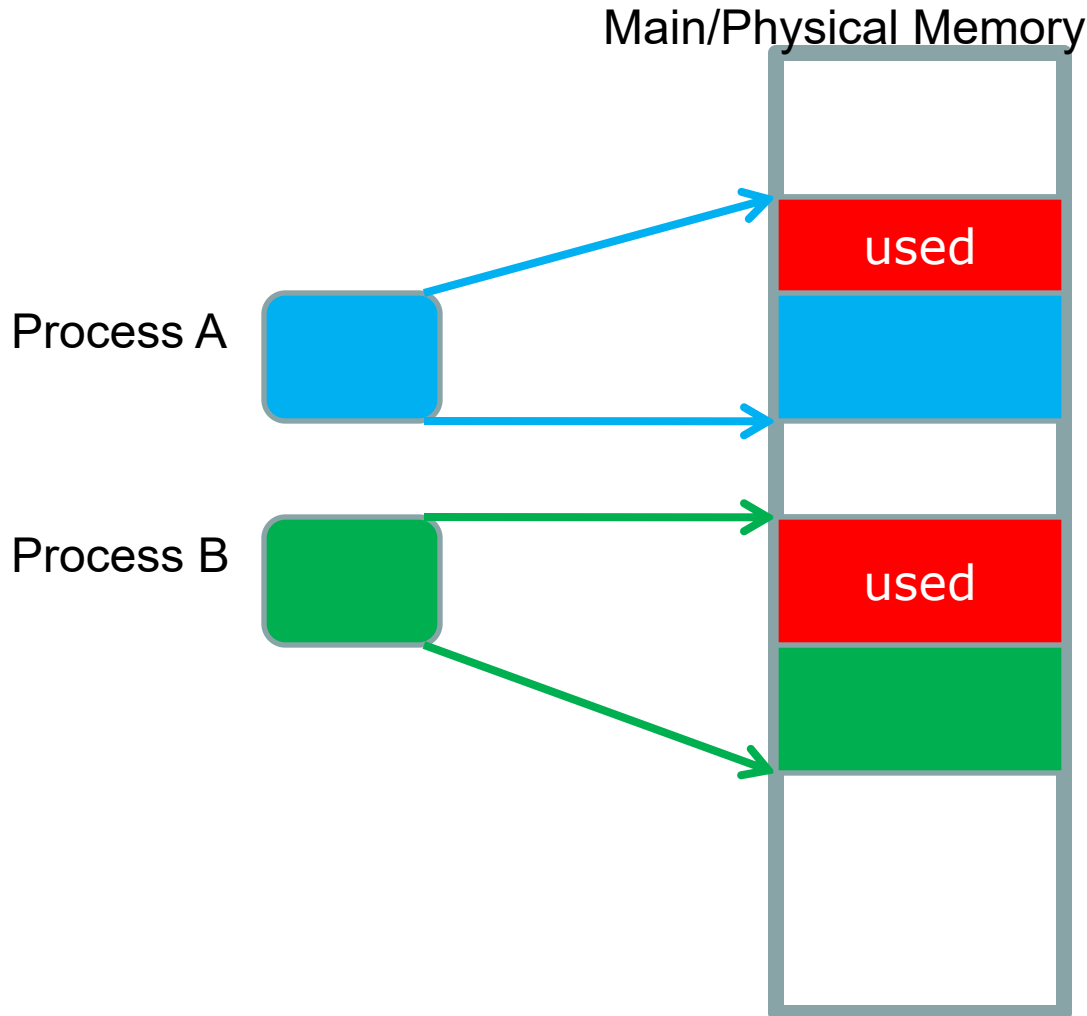
on to protection
via virtual memory

Prior to Virtual Memory



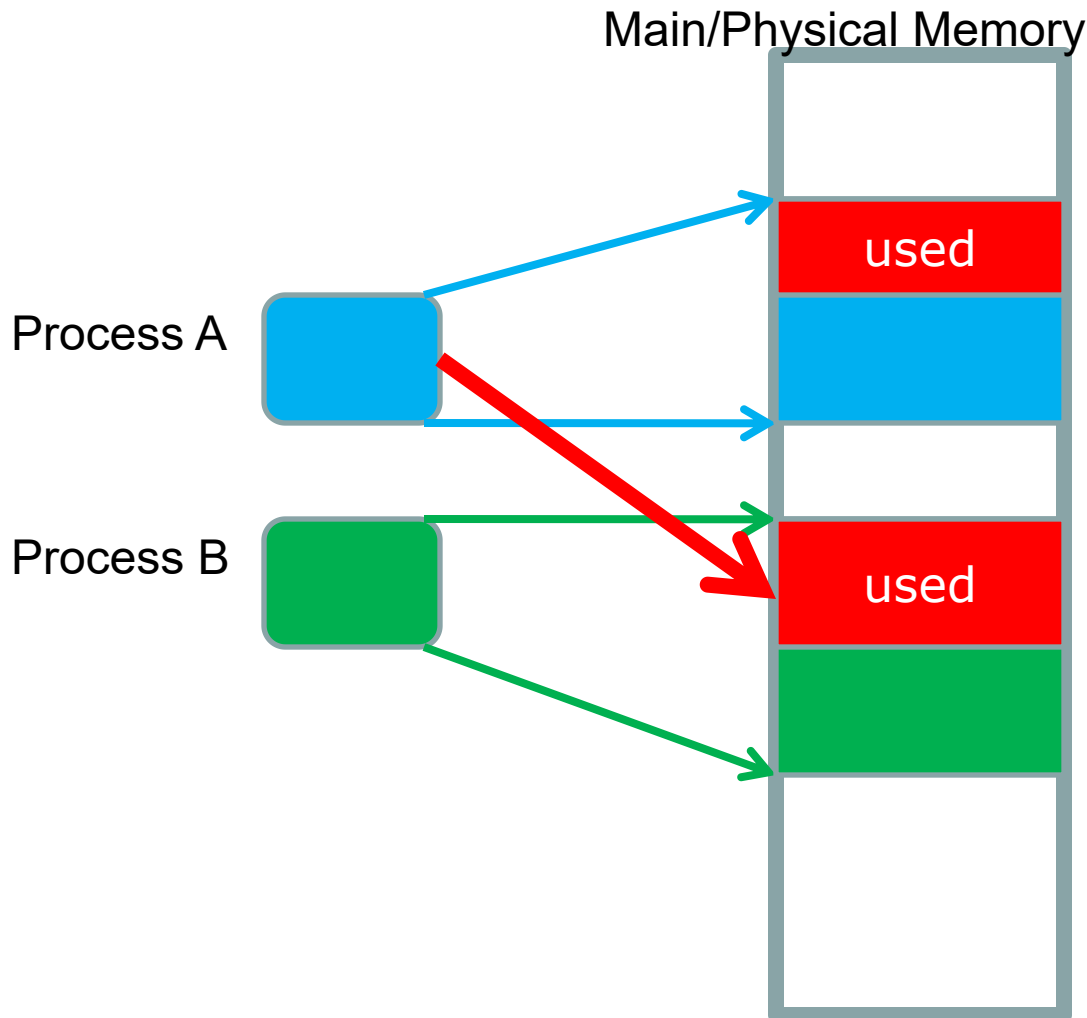
- Contiguous allocation
- Direct memory access

Prior to Virtual Memory



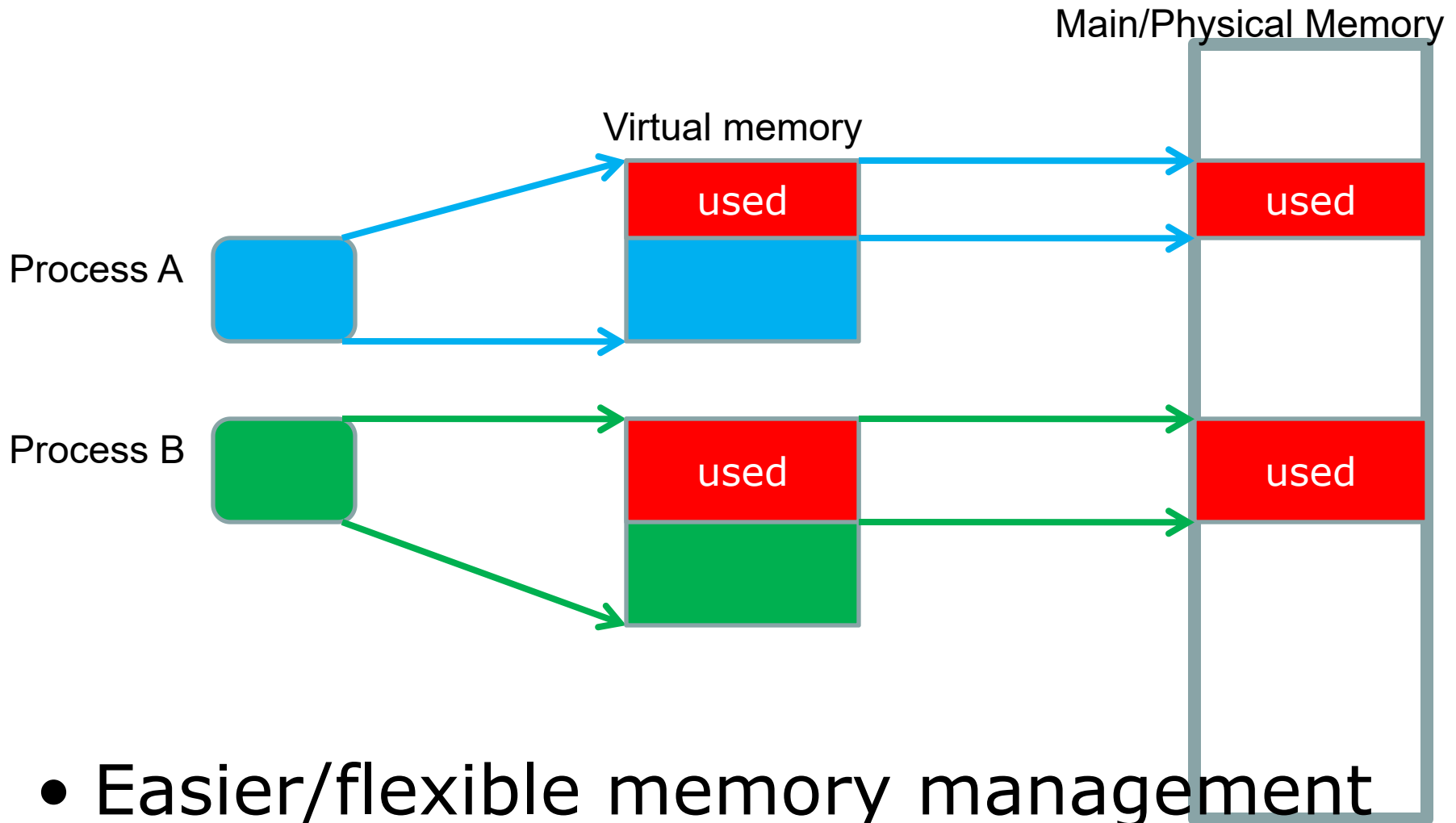
- *efficiency?*

Prior to Virtual Memory

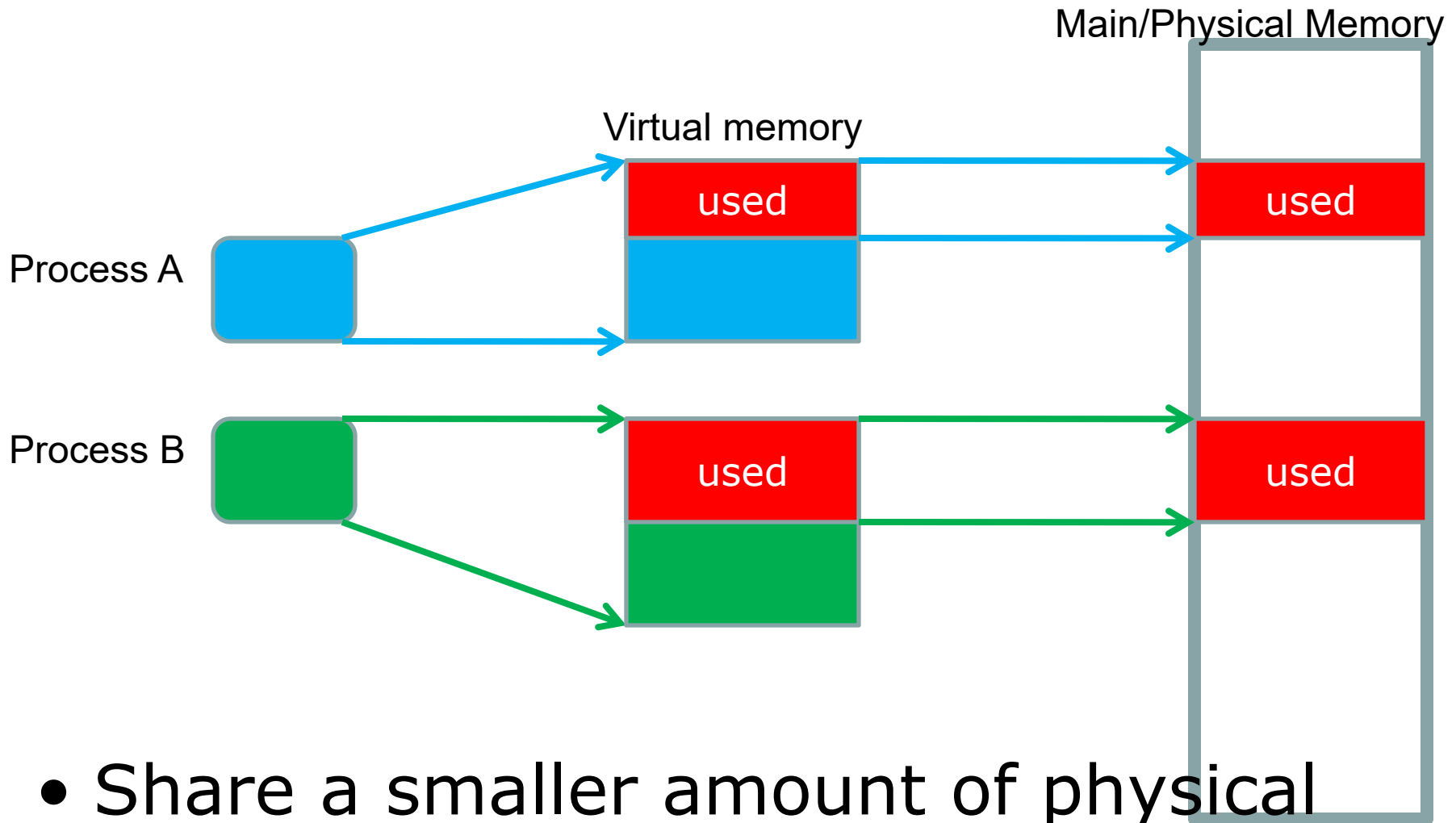


- *protection?*

Virtual Memory

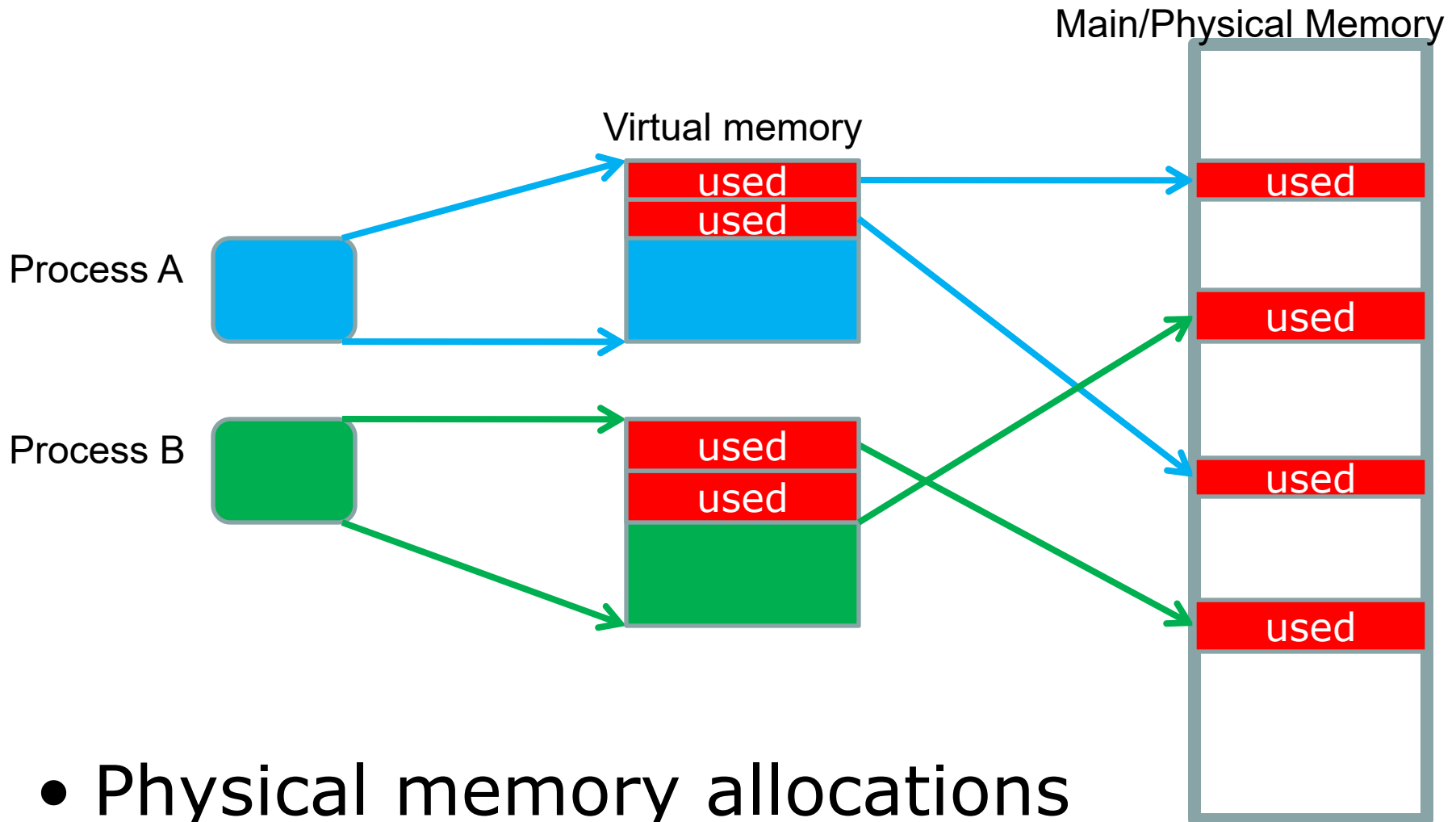


Virtual Memory



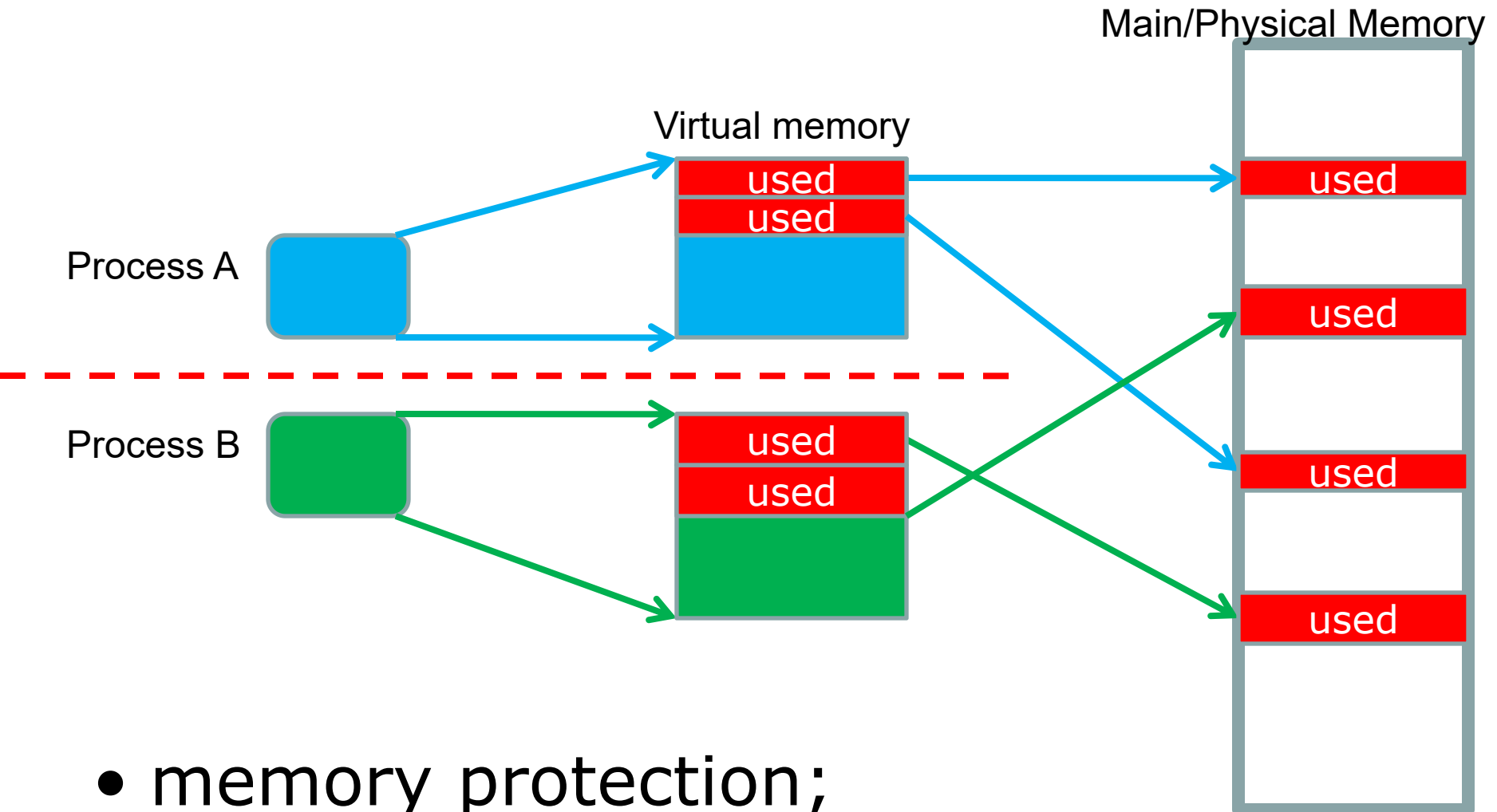
- Share a smaller amount of physical memory among many processes

Virtual Memory



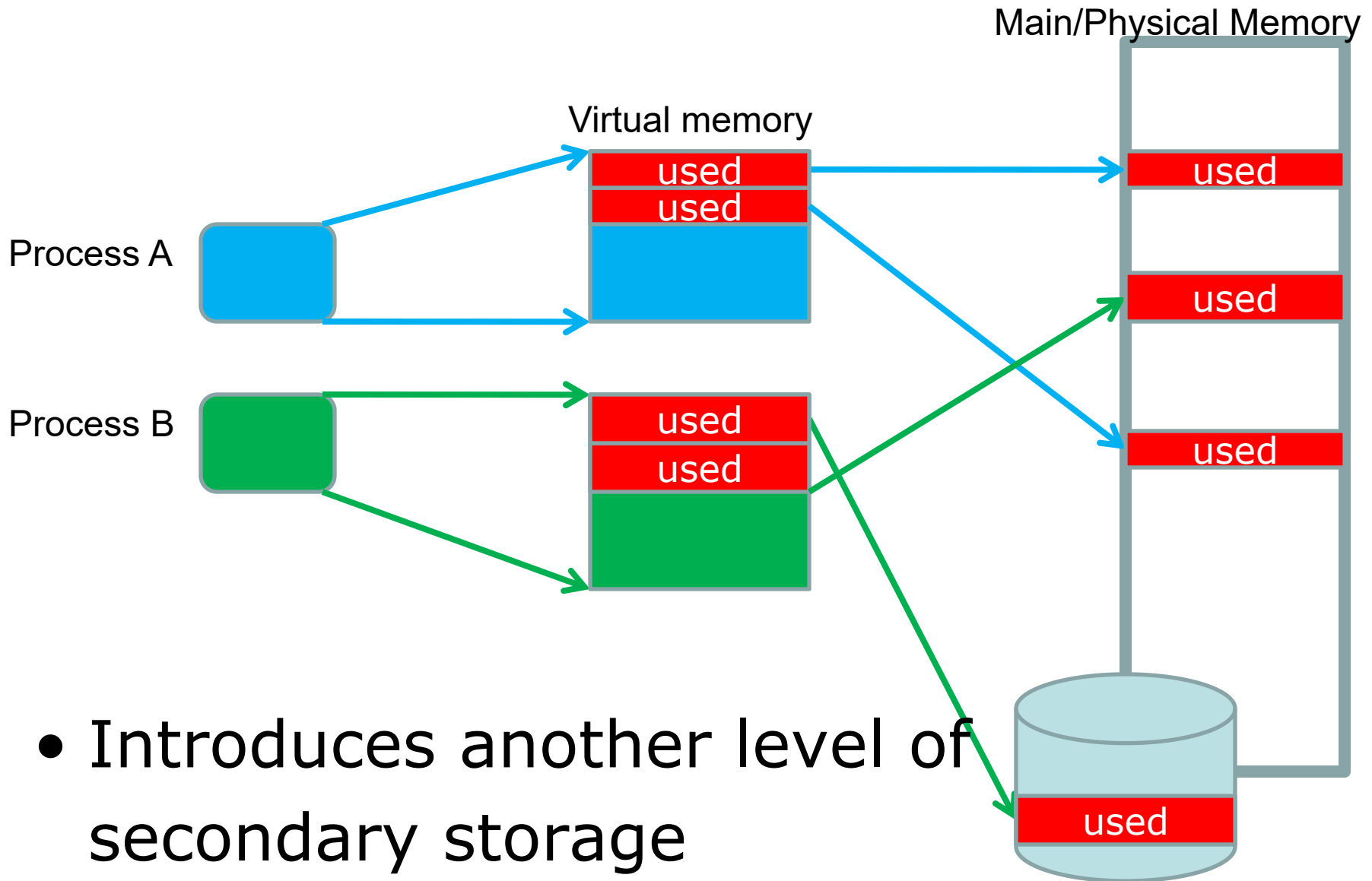
- Physical memory allocations need not be contiguous

Virtual Memory



- memory protection;
process isolation

Virtual Memory



Virtual Memory

- The architecture must limit what a process can access when running a user process yet allow an OS process to access more
- **Four tasks** for the architecture

Virtual Memory

- **Task 1**

The architecture provides at least two modes, indicating whether the running process is a user process or an OS process (i.e., kernel/supervisor process)

Virtual Memory

- **Task 2**

The architecture provides a portion of the processor state that a user process can use but not write

Virtual Memory

- **Task 3**

The architecture provides mechanisms whereby the processor can go from user mode to supervisor mode (i.e., **system call**) and vice versa

Virtual Memory

- **Task 4**

The architecture provides mechanisms to limit memory accesses to protect the memory state of a process without having to swap the process to disk on a context switch

1000s bugs in production os

Virtual Machines

- **Virtual Machine**

a protection mode with a much smaller code base than the full OS

- **VMM: virtual machine monitor
hypervisor**

software that supports VMs

- **Host**

underlying hardware platform

Motivation

- Increasing importance of isolation and security in modern systems

Motivation

- Increasing importance of isolation and security in modern systems
- Failures in security and reliability of standard operating systems

Motivation

- Increasing importance of isolation and security in modern systems
- Failures in security and reliability of standard operating systems
- Sharing of a single computer among many unrelated users, e.g., in a data center or cloud

Motivation

- Increasing importance of isolation and security in modern systems
- Failures in security and reliability of standard operating systems
- Sharing of a single computer among many unrelated users, e.g., in a data center or cloud
- Dramatic increases in processor speed, making VM overhead more acceptable

Virtual Machines

- Properties/Benefits
 1. a single computer runs multiple VMs and can support a number of different OSes
 2. multiple OSes all share the hardware resources

VMM: Virtual Machine Monitor

- three essential **characteristics**:
 1. VMM provides an environment for programs which is essentially identical with the original machine;
 2. programs run in this environment show at worst only minor decreases in speed;
 3. VMM is in complete control of system resources;
- **Mainly for security and privacy**
sharing and protection among multiple processes

VMM

- Qualitative Requirements
 1. guest software should behave on a VM exactly as if it were running on the native hardware
 2. guest software should not be able to change allocation of real system resources directly

VMM

- Privilege Requirements
 1. at least two processor modes, system and user
 2. a privileged subset of instructions that is available only in system mode, resulting in a trap if executed in user mode;

all system resources must be controllable only via these instructions

examples

ARM Cortex-A53

Intel Core i7-6700

ARM Cortex-A53

- ARMv8A ISA
- 32-bit / 64-bit mode
- 2-instr/clock, clock rate up to 1.3 GHz
- IP (intellectual property) core.
designed to be incorporated with other
logic (e.g., app-specific processors, I/O
interfaces, mem interfaces);
fabricated to yield a processor
optimized for a particular application

ARM Cortex-A53

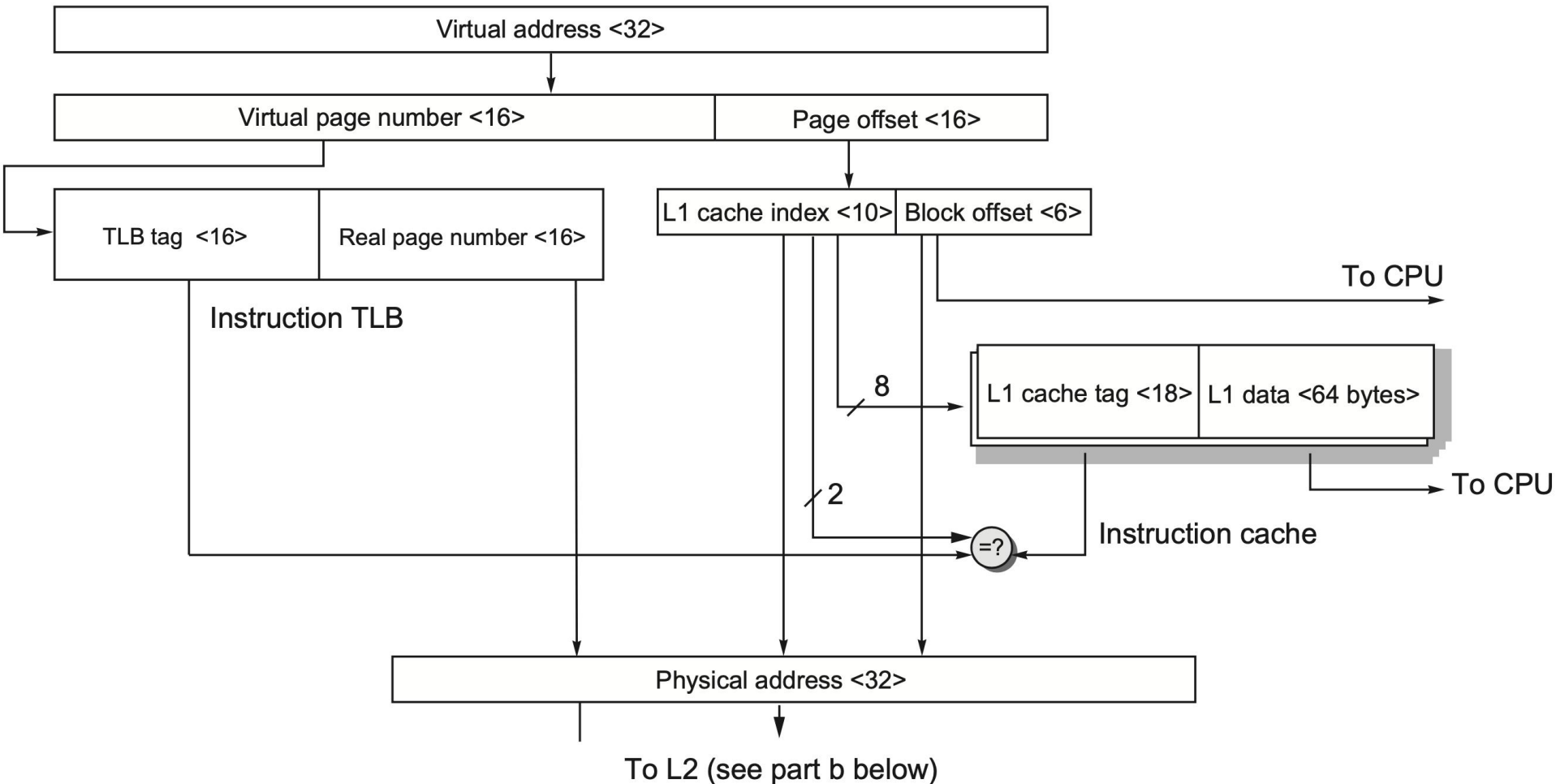
- ARMv8A ISA
- 32-bit / 64-bit mode
- 2-instr/clock, clock rate up to 1.3 GHz
- IP (intellectual property) core.
 - hard cores: optimized for a particular semiconductor vendor and cannot be modified
 - soft cores: use a standard lib of logic elements and can be compiled/modified for different semiconductor vendors

Memory Hierarchy

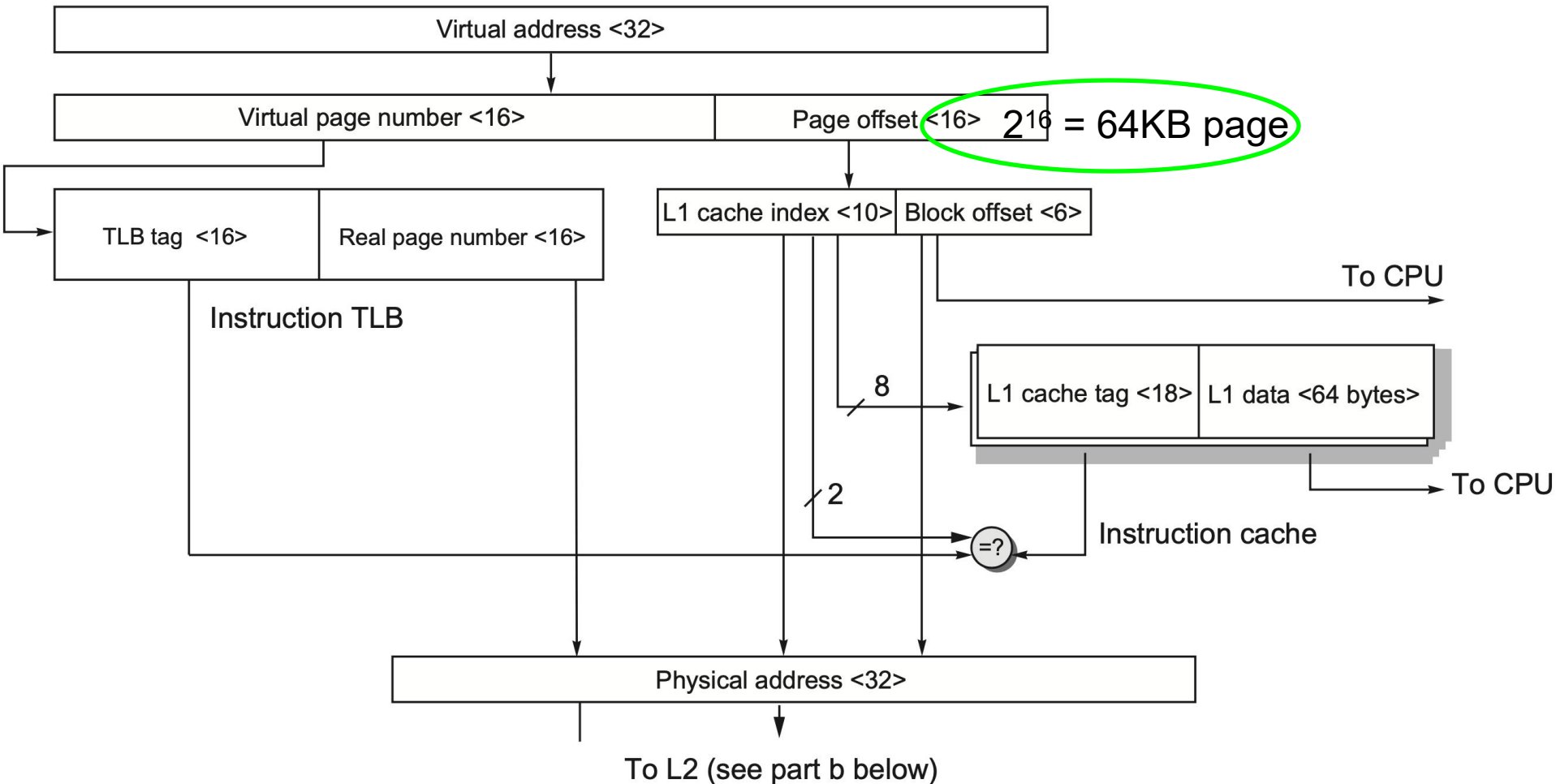
- Two-level TLB
- Two-level cache
- LRU-approximation replacement policy

Structure	Size	Organization	Typical miss penalty (clock cycles)
Instruction MicroTLB	10 entries	Fully associative	2
Data MicroTLB	10 entries	Fully associative	2
L2 Unified TLB	512 entries	4-way set associative	20
L1 Instruction cache	8–64 KiB	2-way set associative; 64-byte block	13
L1 Data cache	8–64 KiB	2-way set associative; 64-byte block	13
L2 Unified cache	128 KiB to 2 MiB	16-way set associative; LRU	124

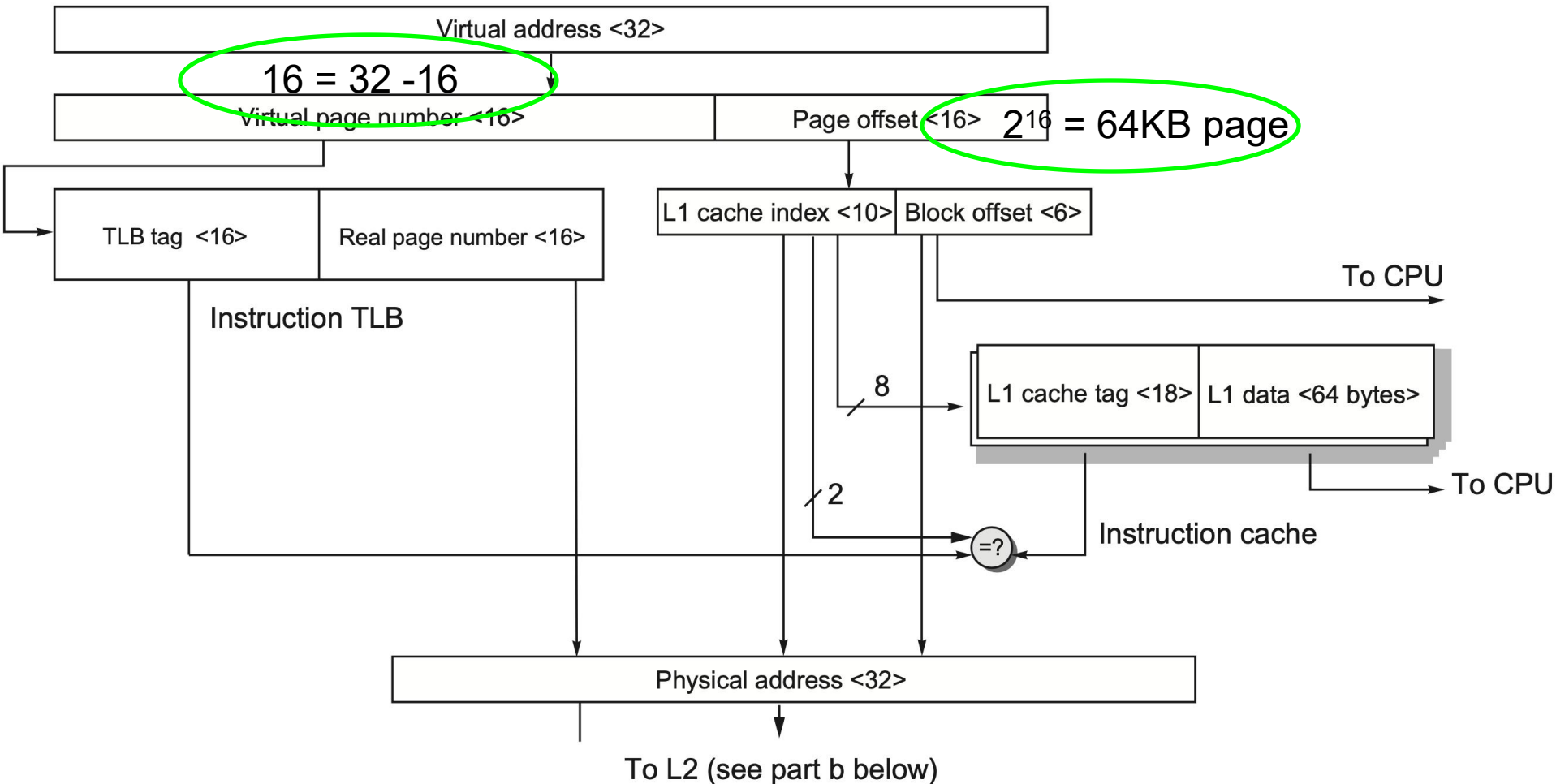
Instruction Access Path



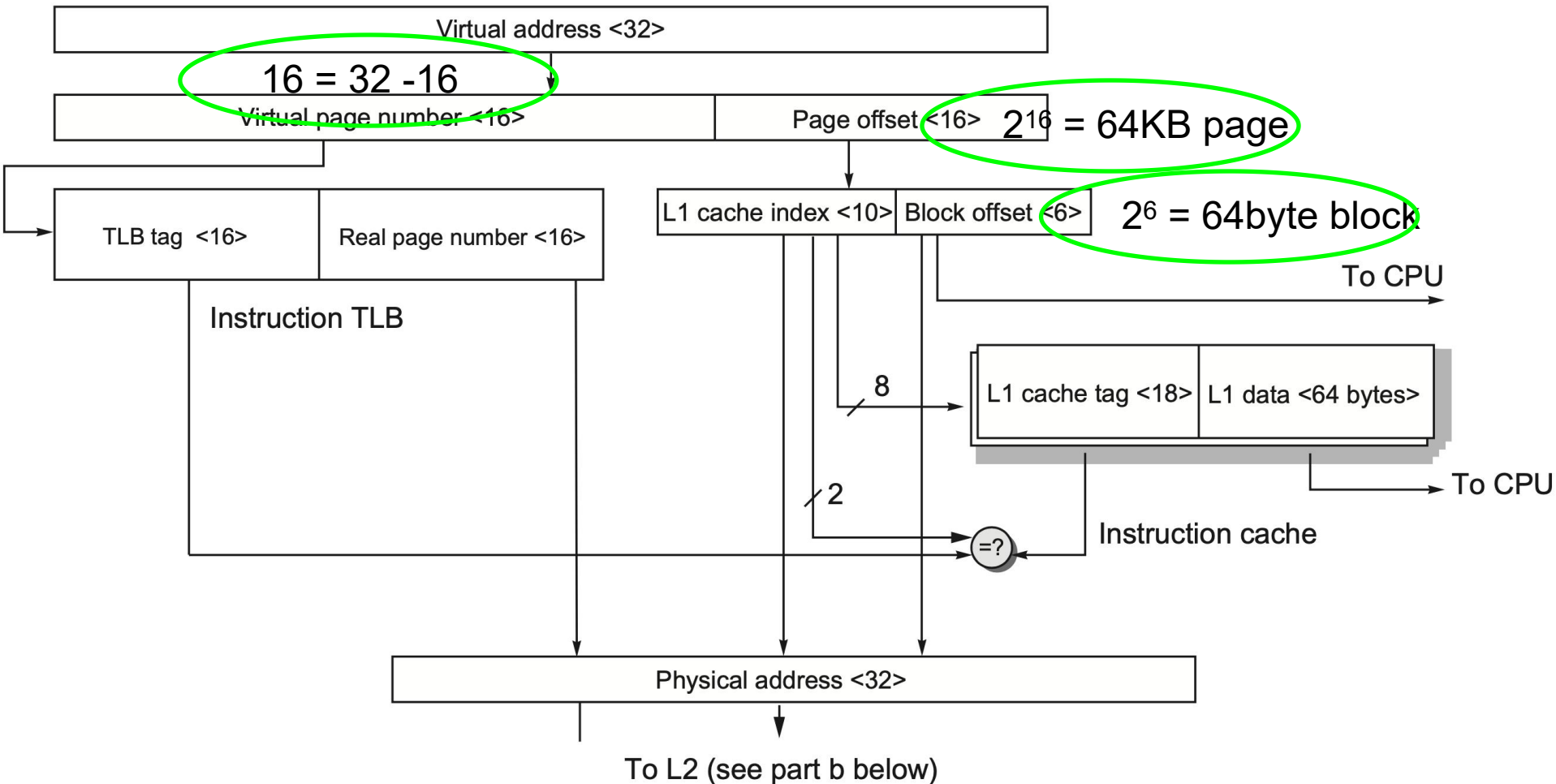
Instruction Access Path



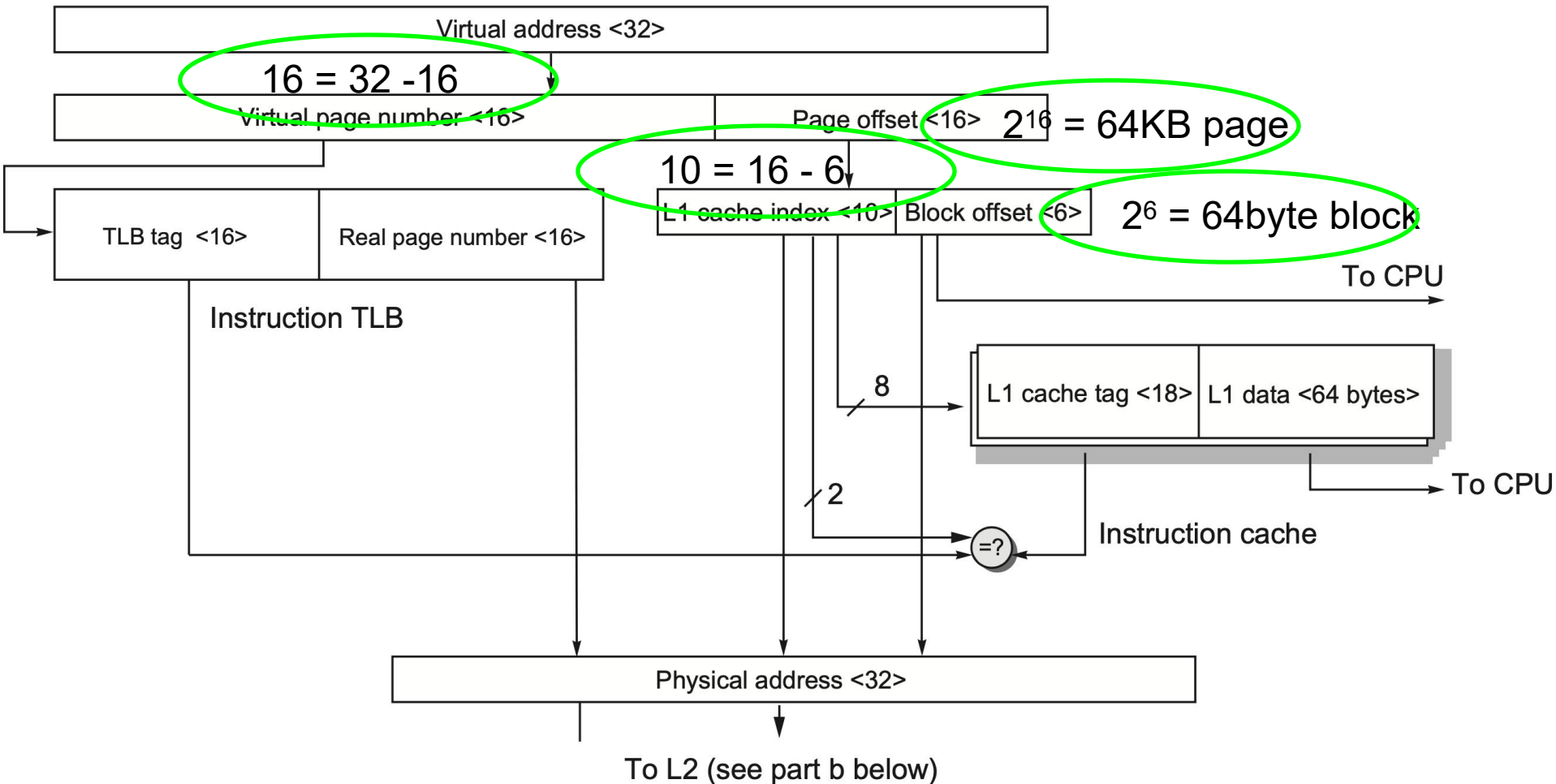
Instruction Access Path



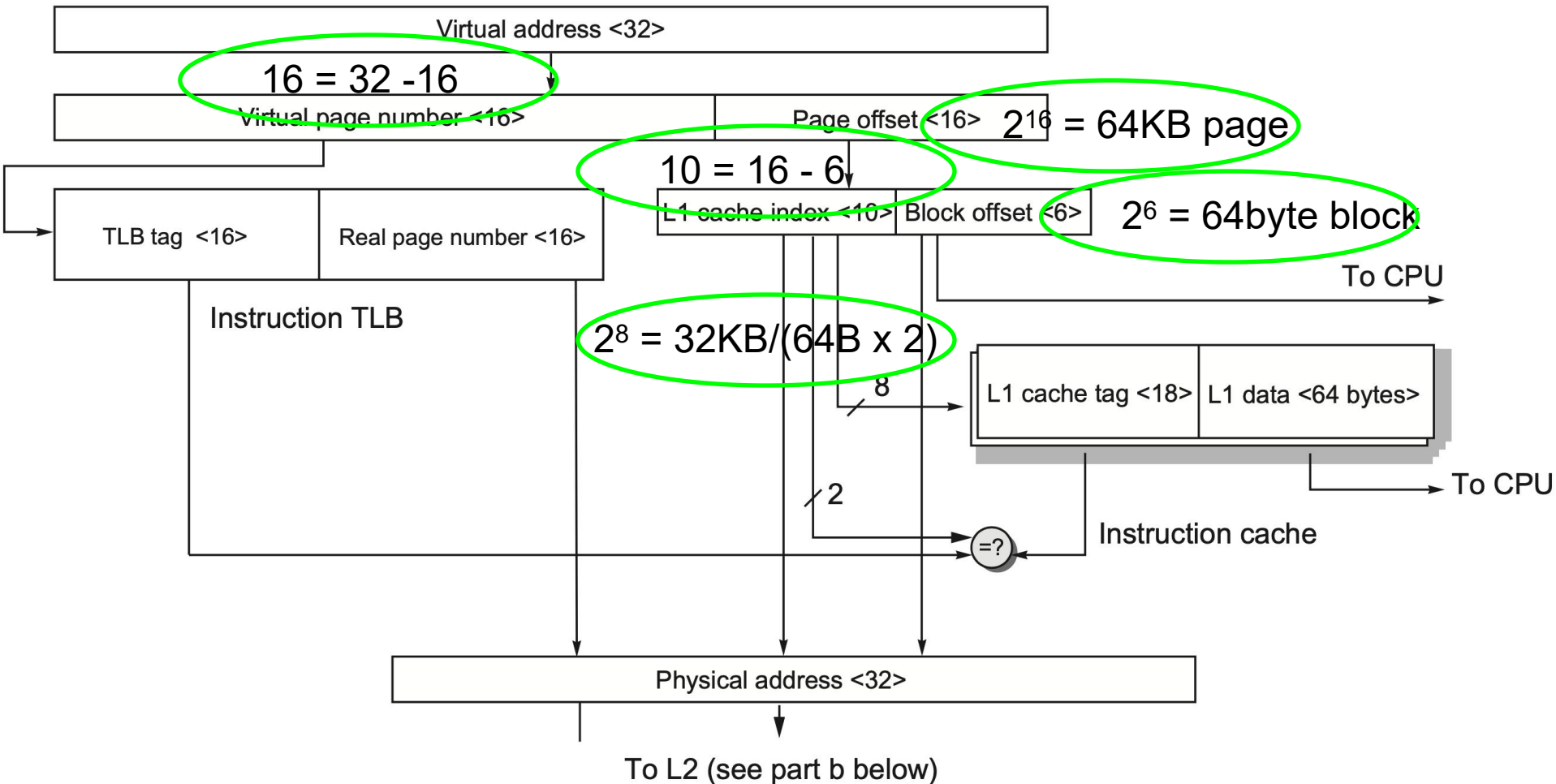
Instruction Access Path



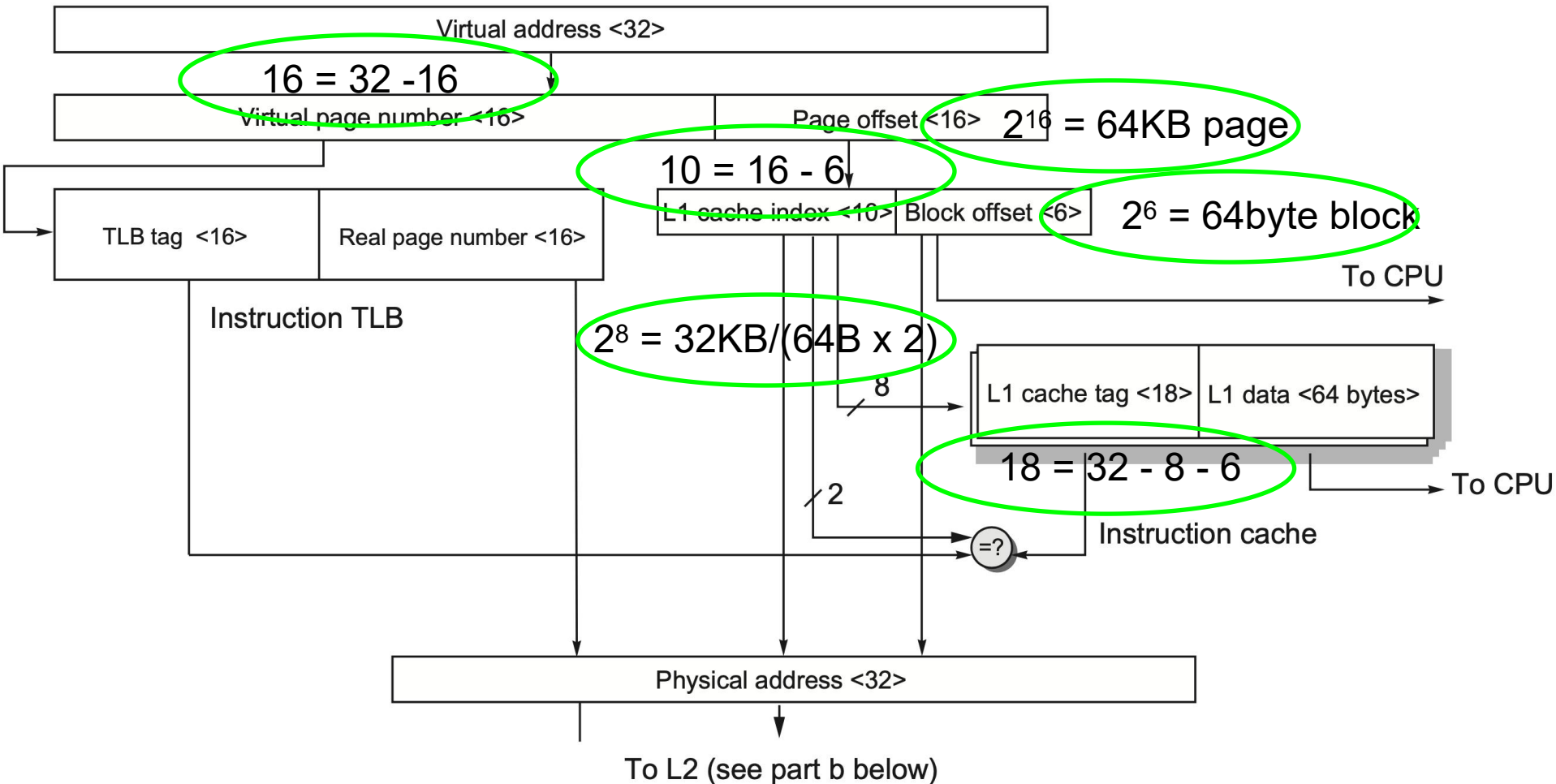
Instruction Access Path



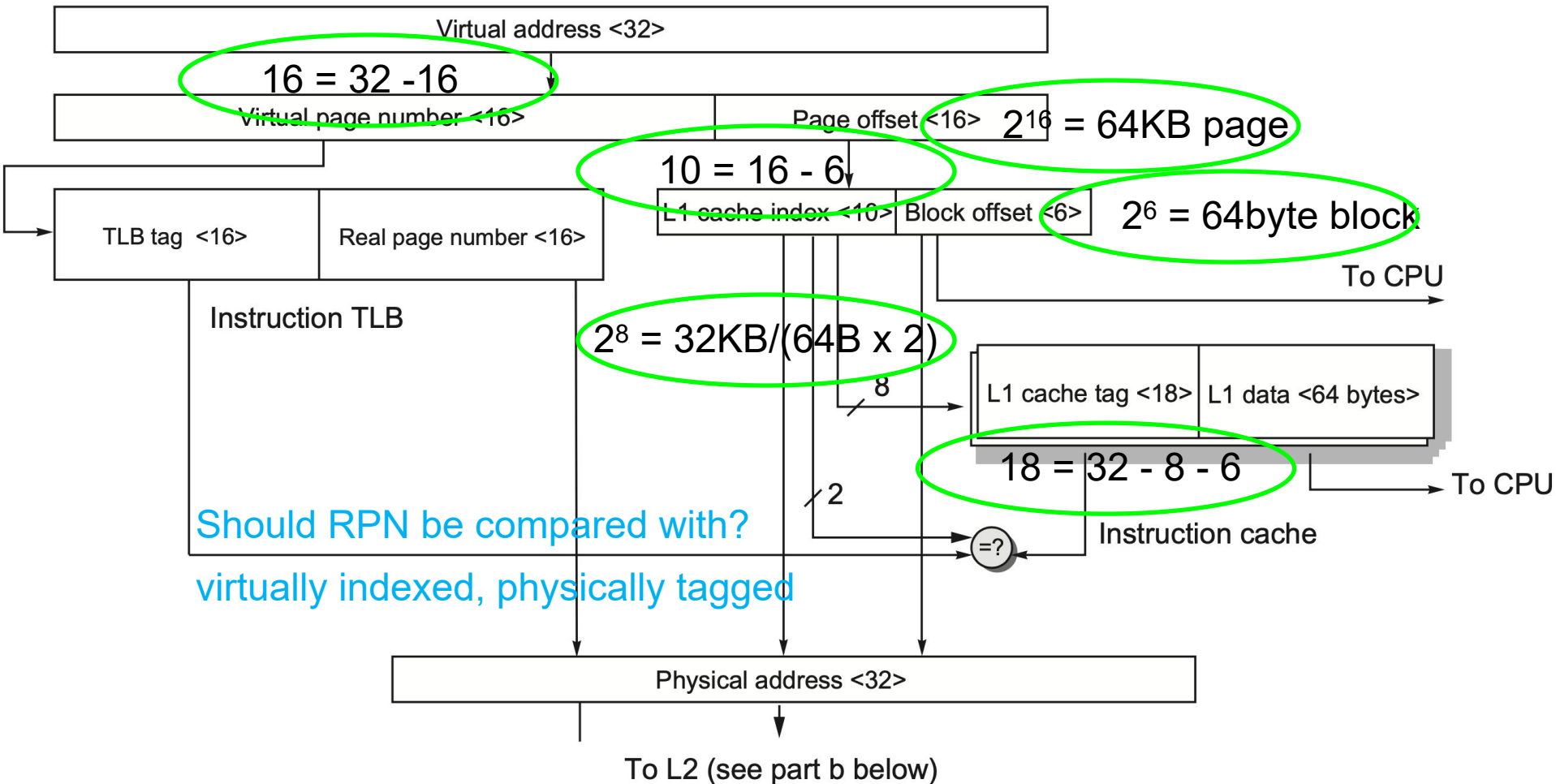
Instruction Access Path



Instruction Access Path



Instruction Access Path



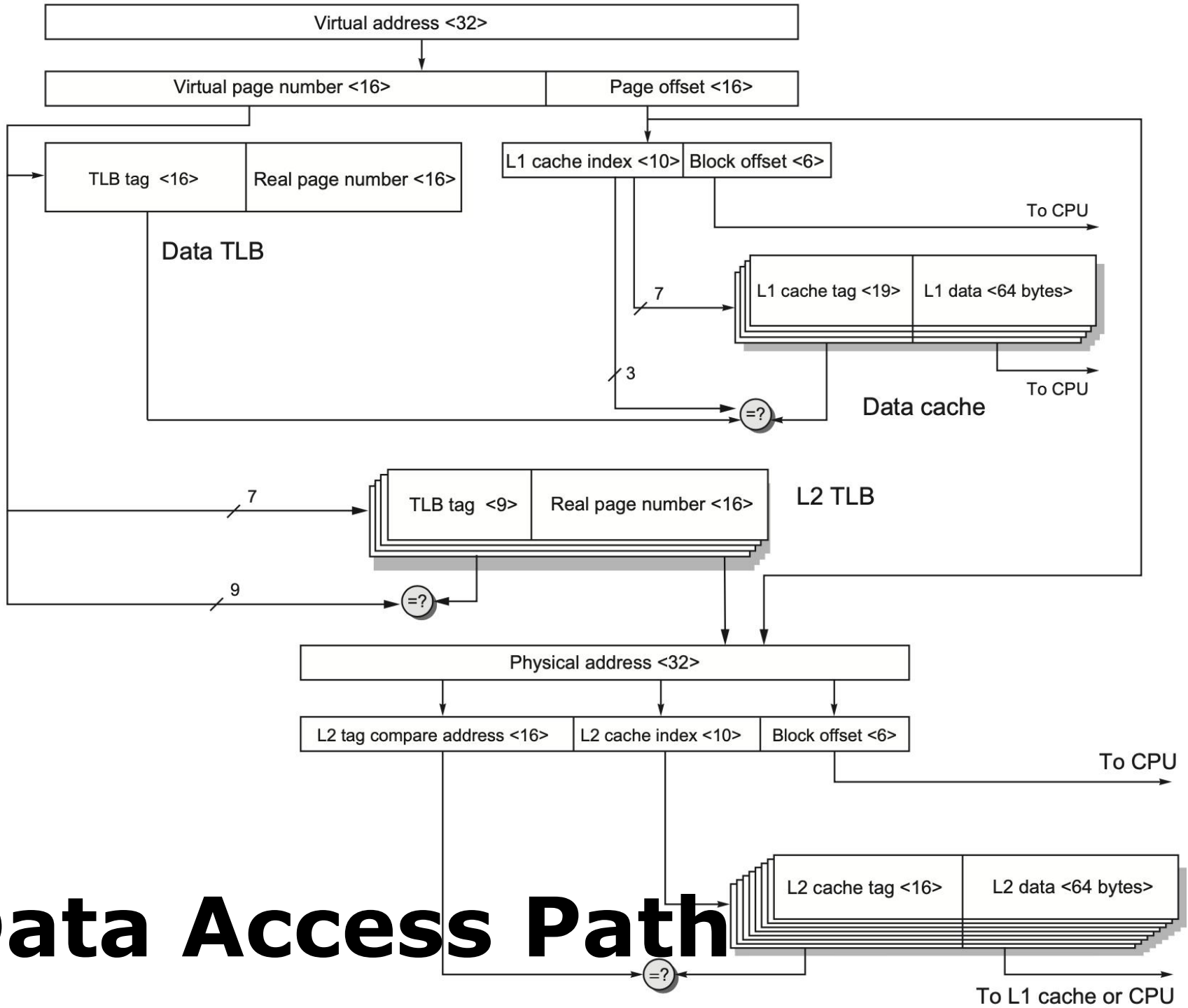
W

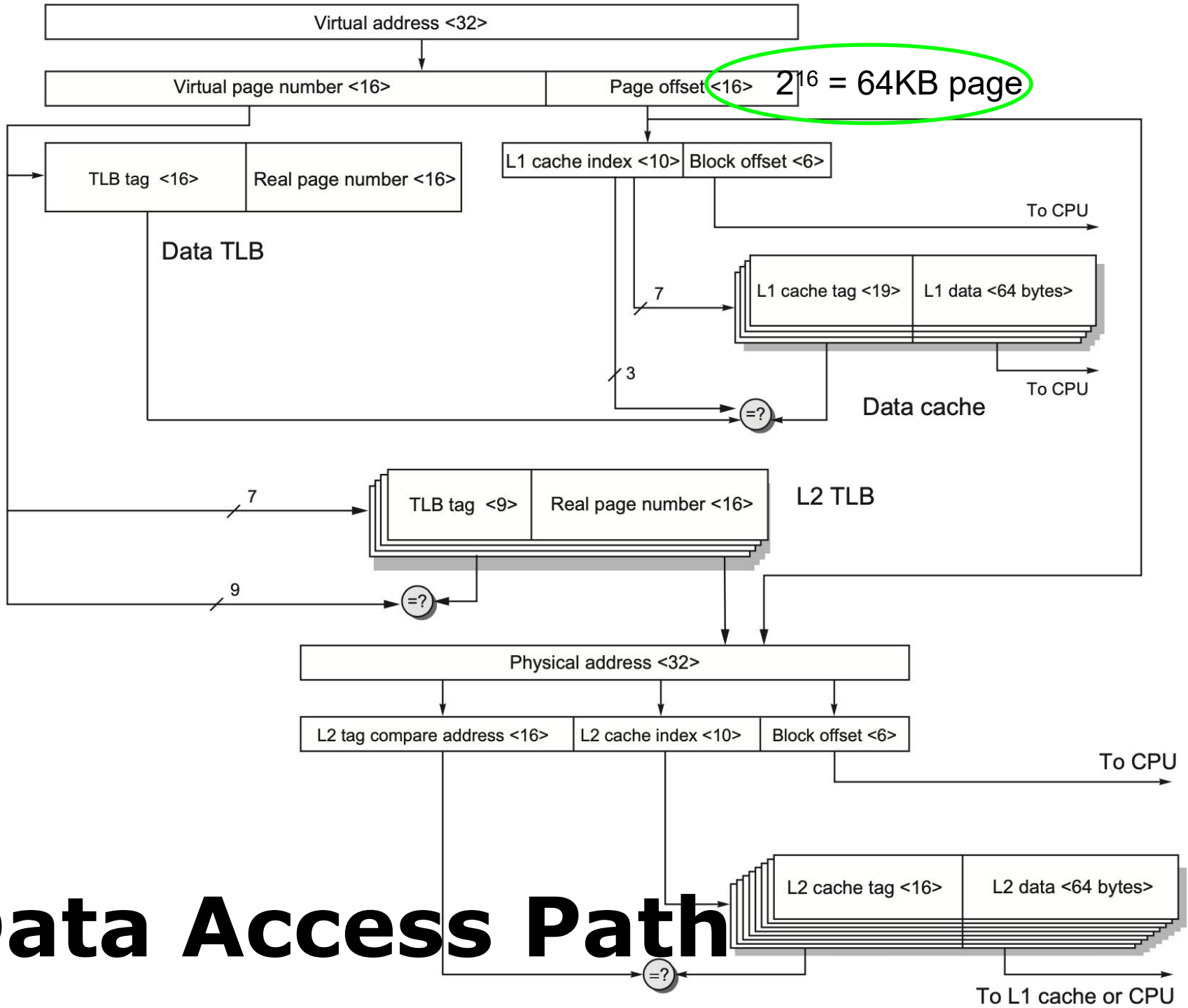
1. 指令路径（上图）

- 来源：取指单元 (IFetch)
- 只读，不写
- 只有 L1 指令缓存 (I\$)
- 只有一级 TLB (ITLB)
- 路径简单，不阻塞流水线

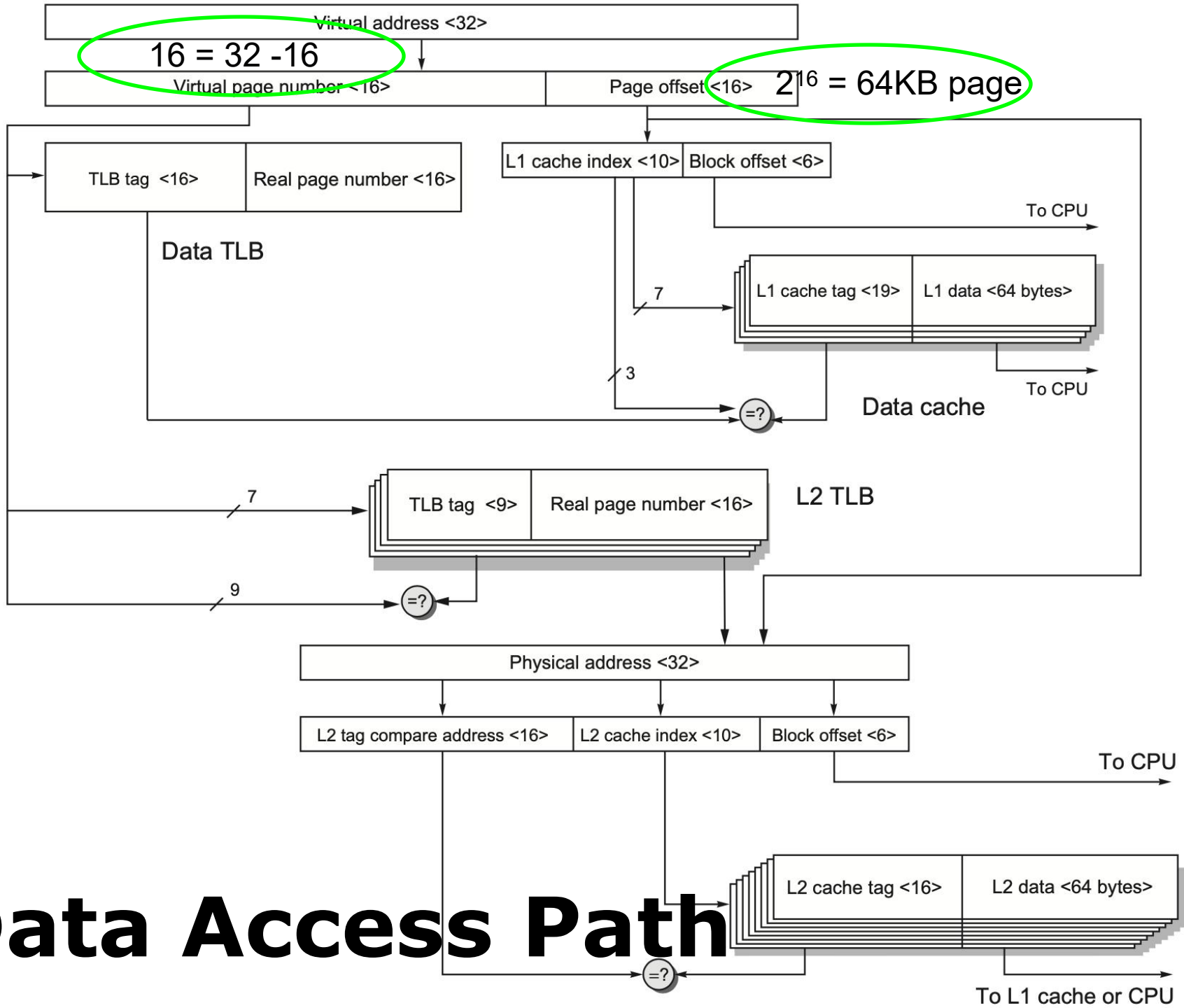
2. 数据路径（下图）

- 来源：加载 / 存储单元 (Load/Store)
- 既读又写
- 包含 L1 数据缓存 (D\$) + L2 统一缓存
- 包含多级 TLB：L1 DTLB + L2 TLB (Unified L2 TLB)
- 路径复杂，涉及缓存命中、缺失、写回、写分配等多种逻辑

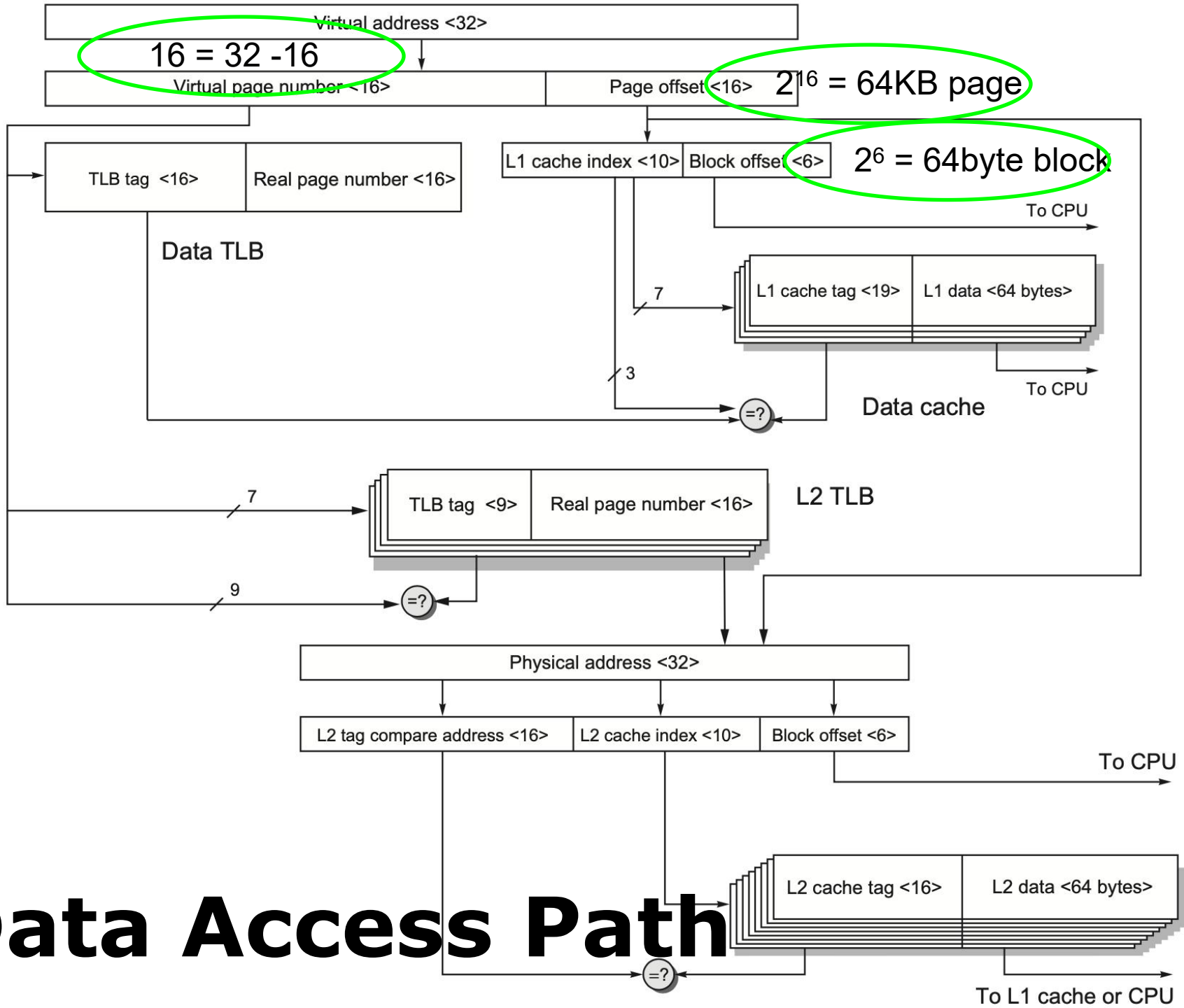


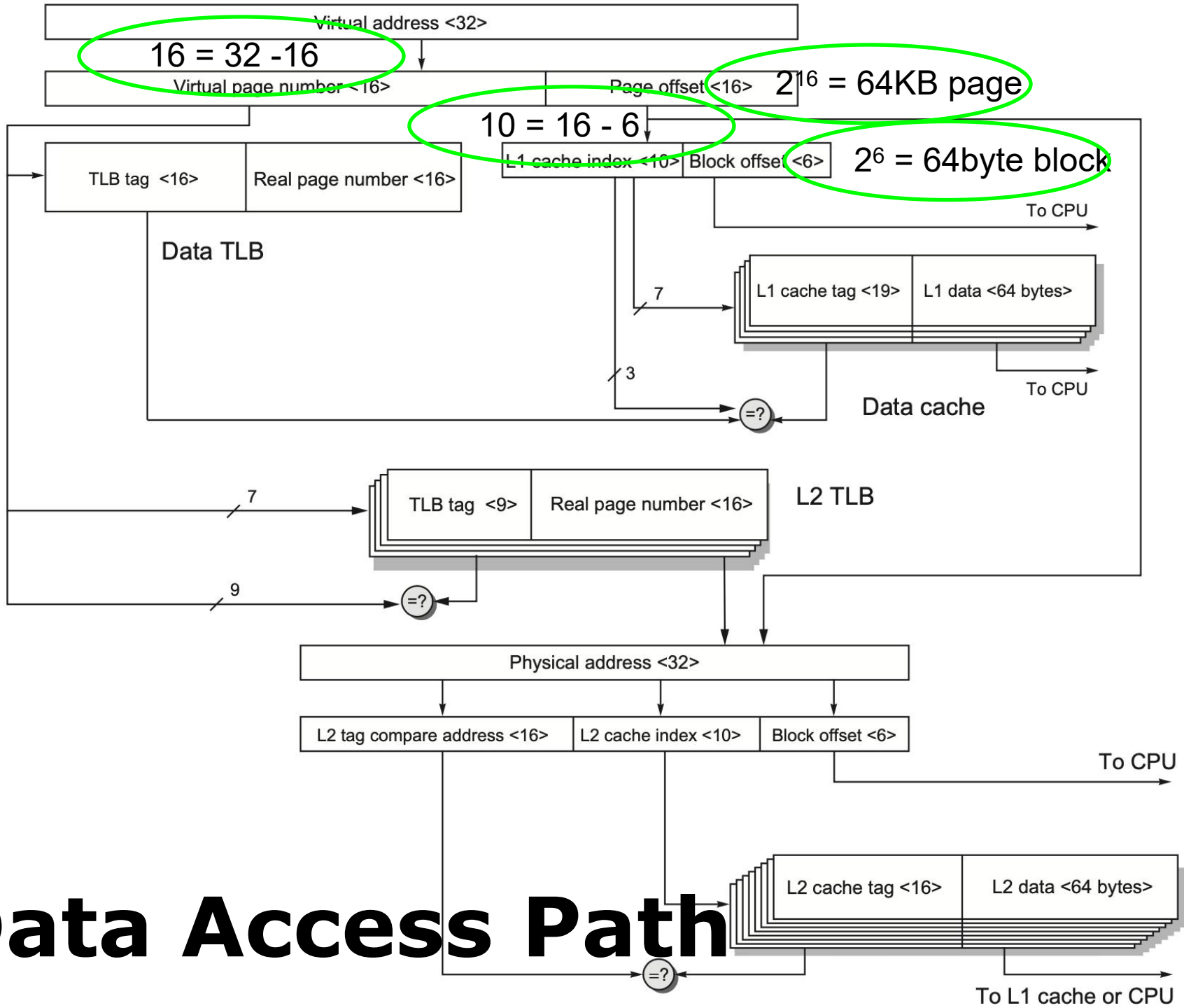


Data Access Path



Data Access Path





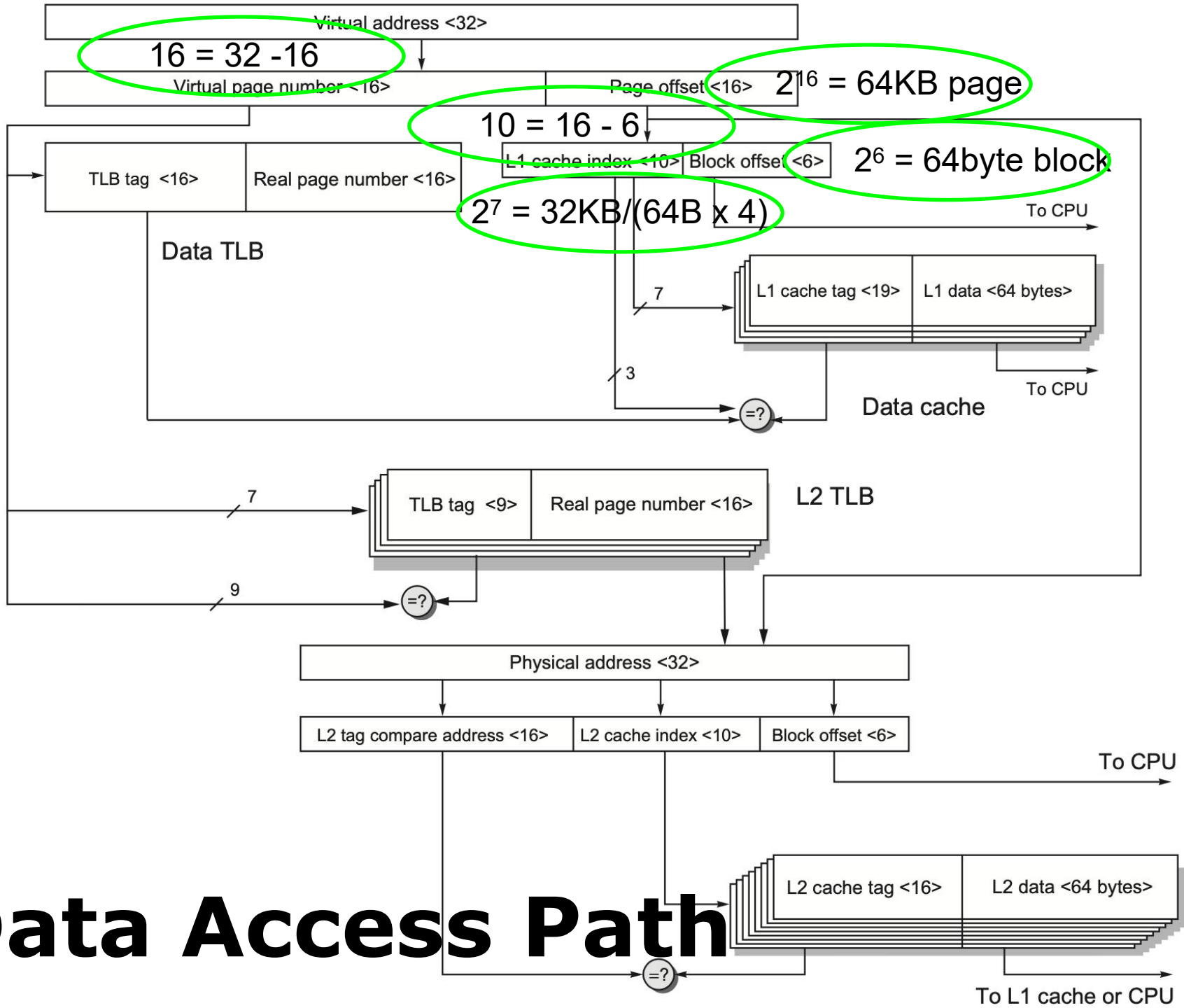


Figure 2.20 The virtual address, physical address, and data blocks for the ARM Cortex-A53 caches and TLBs, assuming 32-bit addresses. The top half (A) shows the instruction access; the bottom half (B) shows the data access, including L2. The TLB (instruction or data) is fully associative each with 10 entries, using a 64 KiB page in this example. The L1 I-cache is two-way set associative, with 64-byte blocks and 32 KiB capacity; the L1 D-cache is 32 KiB four-way set associative, and 64-byte blocks. The L2 TLB is 512 entries and four-way set associative. The L2 cache is 16-way set associative with 64-byte blocks and 128 cKiB to 2 MiB capacity; a 1 MiB L2 is shown. This figure doesn't show the valid bits and protection bits for the caches and TLB.

这里的 **32KB** 是 **L1 数据 Cache (L1 D-Cache)** 的实际硬件设计容量，不是算出来的，是处理器的固定配置，和指令 Cache (L1 I-Cache) 的 32KB 是分开的，总 L1 Cache 是 64KB (32KB I\$ + 32KB D\$)。

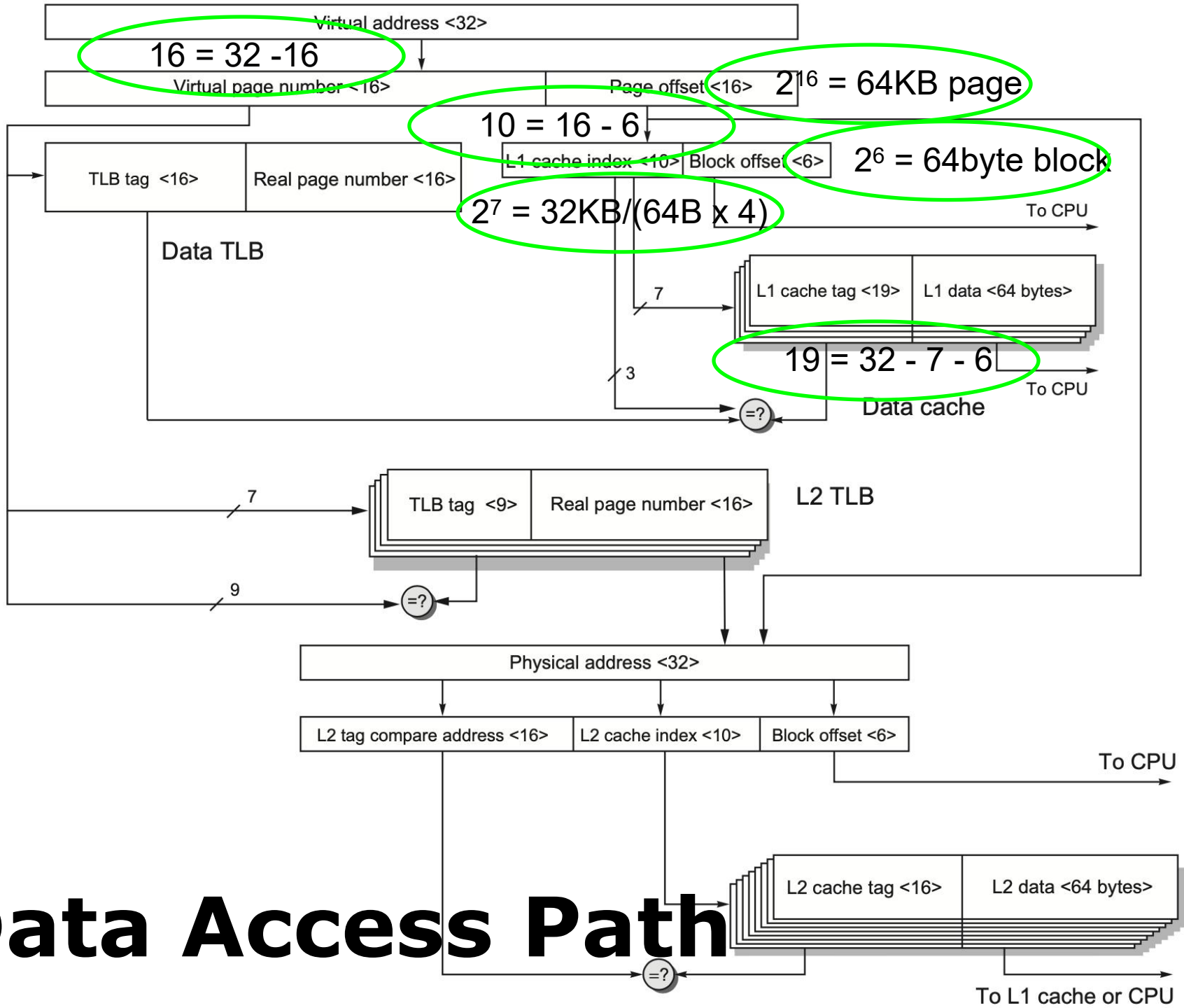
二、为什么不是 64KB？（核心原因）

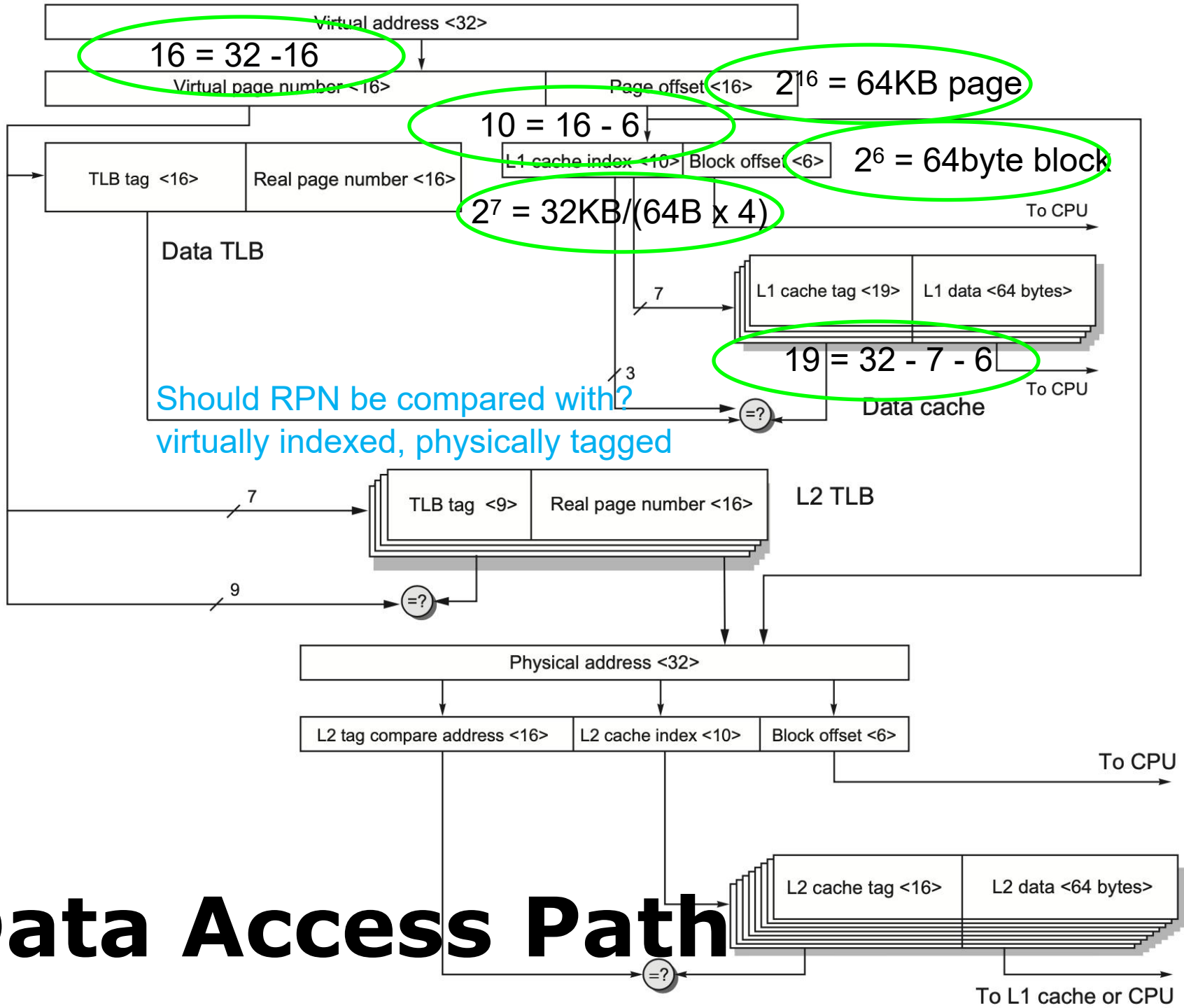
1. 处理器的经典设计：L1 指令 / 数据 Cache 分离（Harvard 架构）

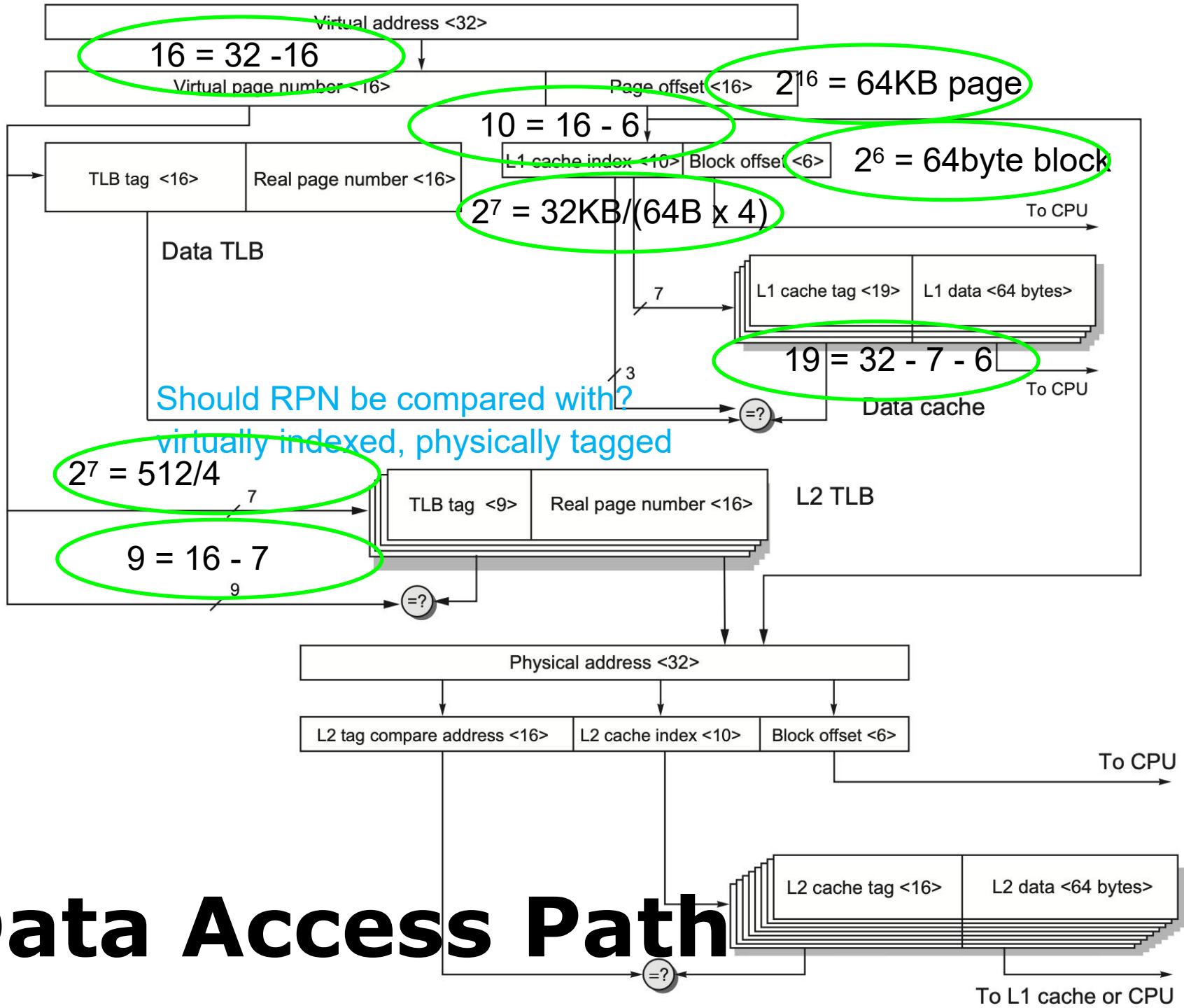
现代 CPU（比如你图里的 ARM Cortex-A53、Intel i7 系列）的 L1 Cache 都是分离式设计：

- **L1 指令 Cache (L1 I-Cache)**：专门存指令，大小 32KB
- **L1 数据 Cache (L1 D-Cache)**：专门存数据，大小 32KB
- 两者是完全独立的硬件，各自有自己的 tag、index、offset 划分，不能混用

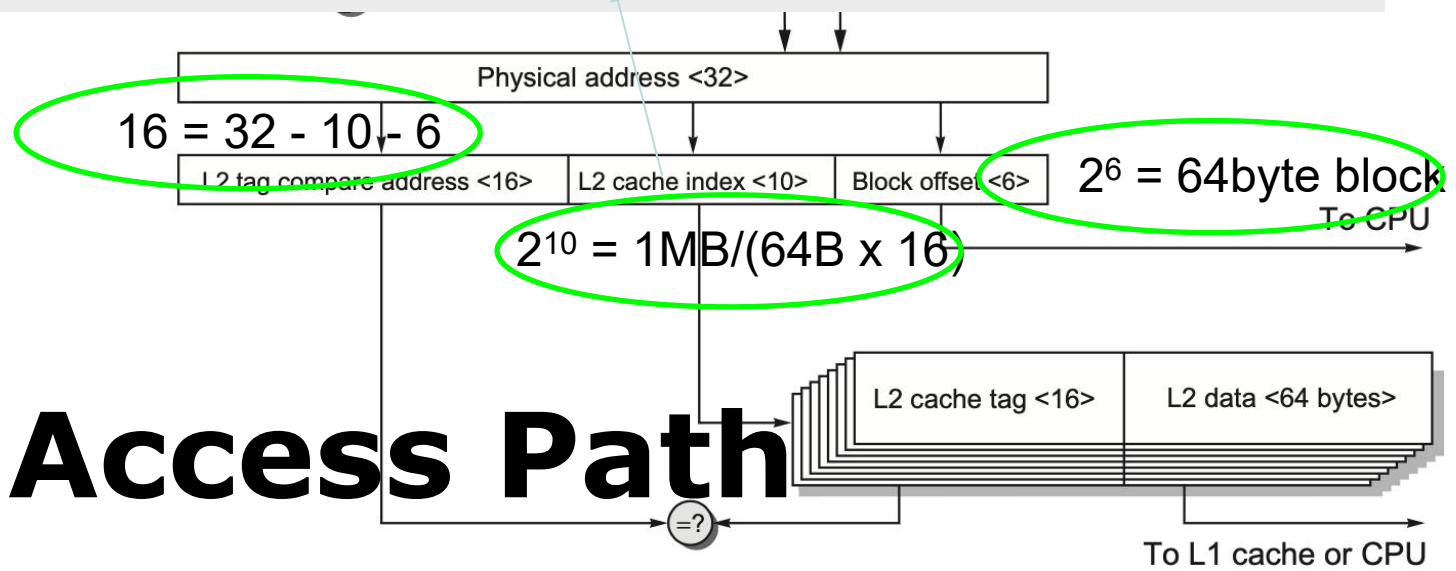
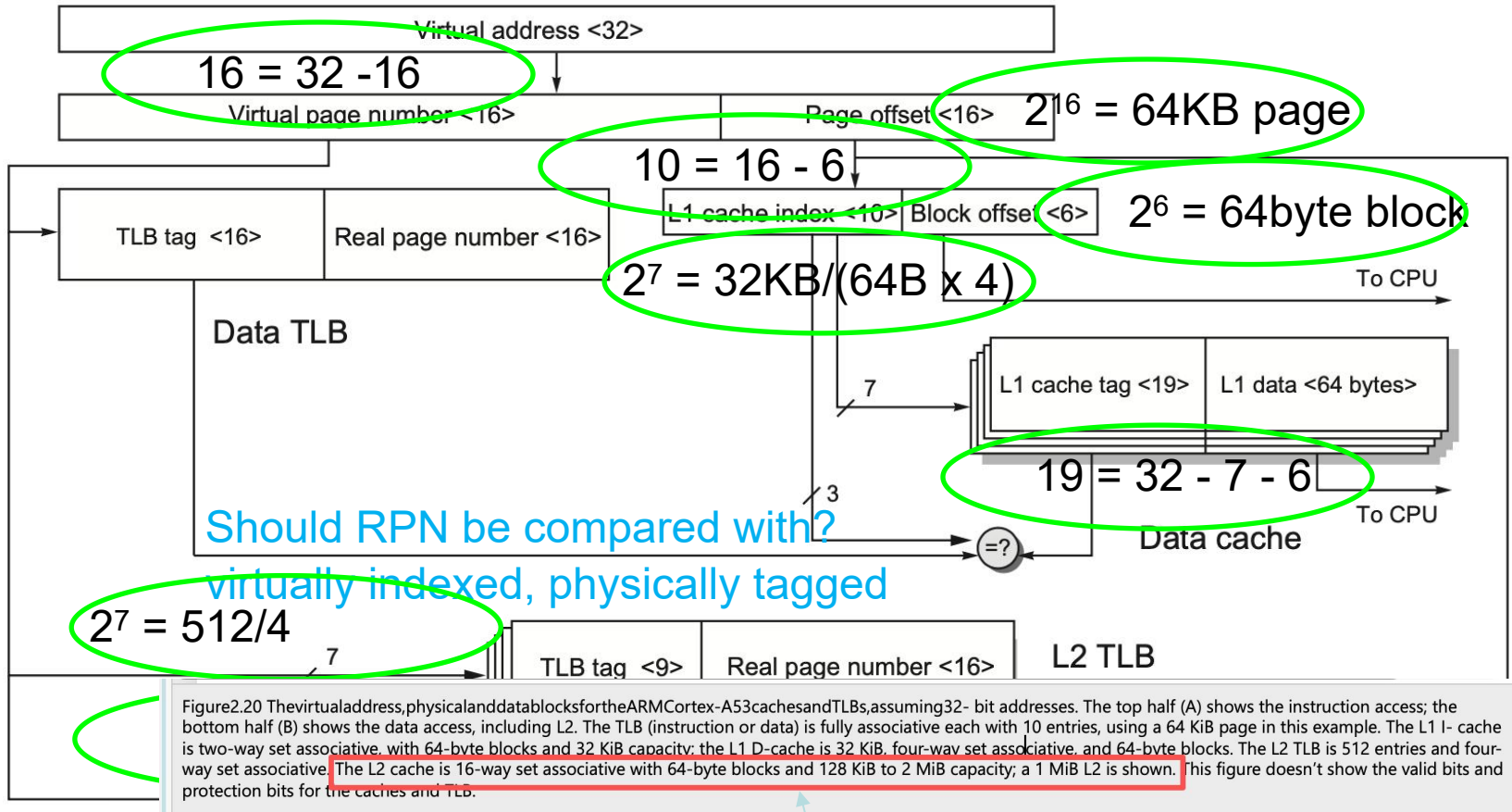
所以你在数据访问路径里算的，是数据 Cache 自己的 **32KB**，不是把指令 + 数据加起来的 64KB。







Data Access Path



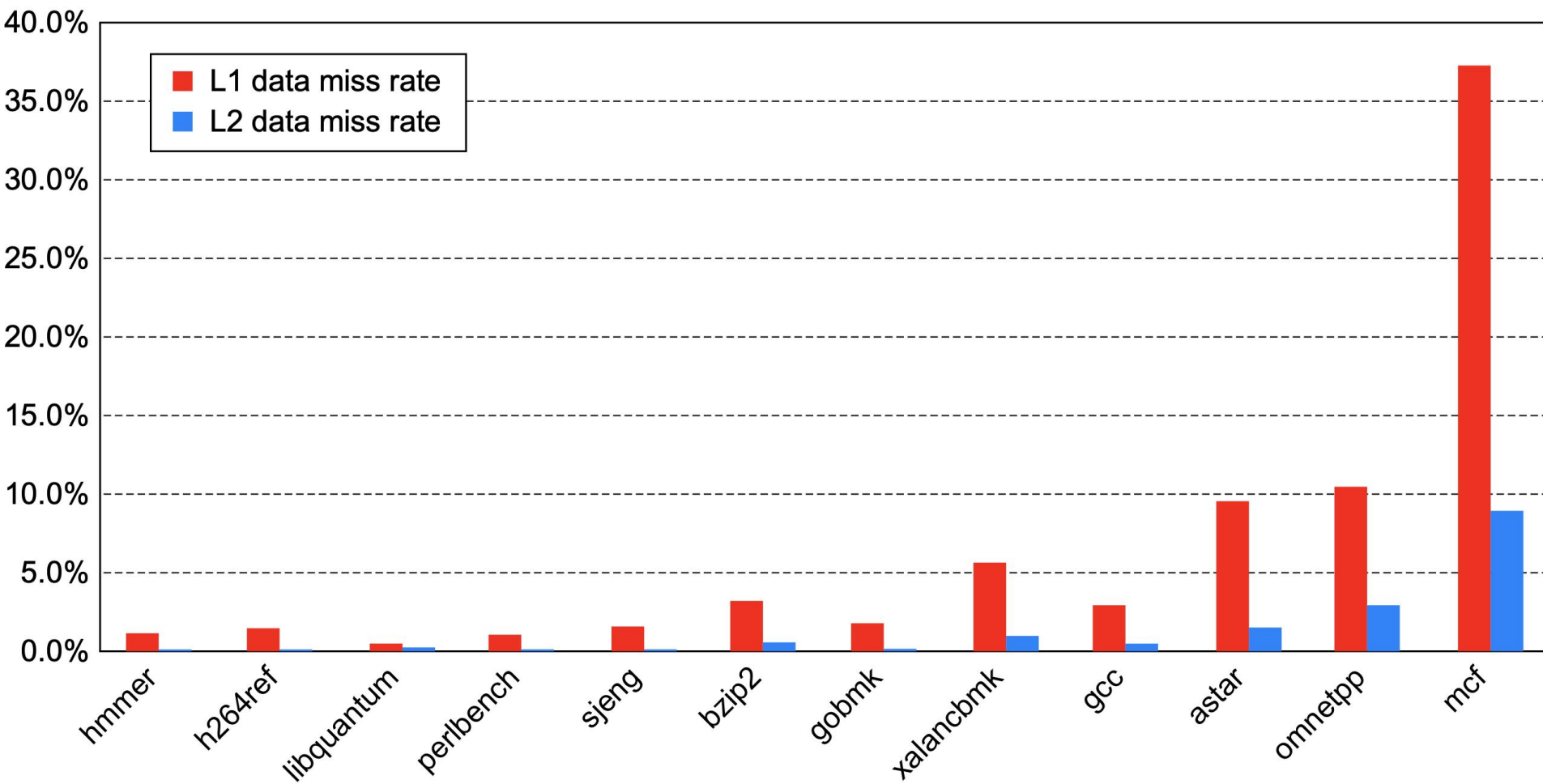
Data Access Path

Performance

- Instruction Cache:
given 32 KiB primary caches and a 1 MiB L2 cache
close to zero miss rate and under 1%
for all of SPECInt2006 benchmarks

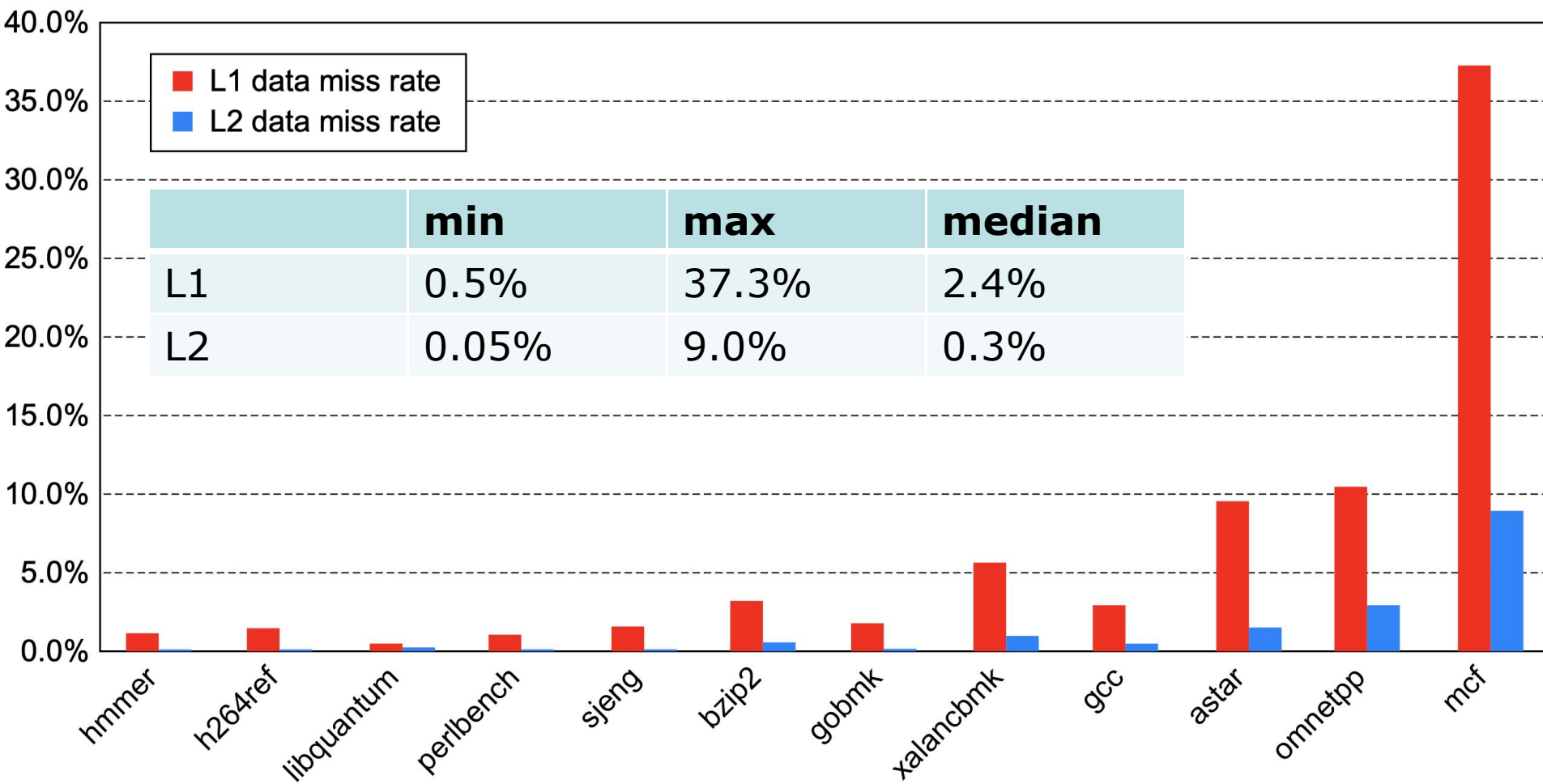
Performance

- Data Cache: global miss rate



Performance

- Data Cache: global miss rate

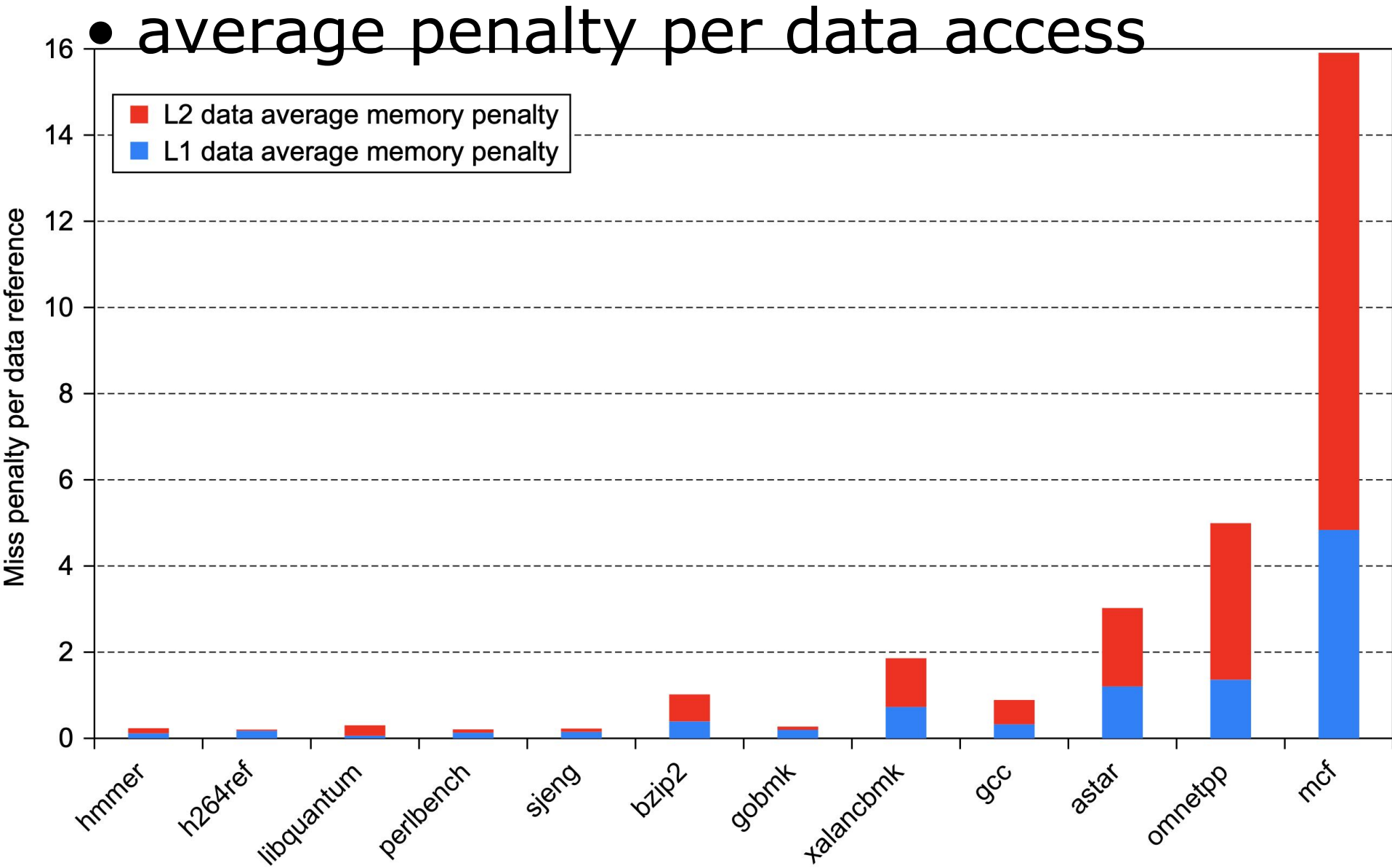


Performance

- Miss Penalty

Structure	Size	Organization	Typical miss penalty (clock cycles)
Instruction MicroTLB	10 entries	Fully associative	2
Data MicroTLB	10 entries	Fully associative	2
L2 Unified TLB	512 entries	4-way set associative	20
L1 Instruction cache	8–64 KiB	2-way set associative; 64-byte block	13
L1 Data cache	8–64 KiB	2-way set associative; 64-byte block	13
L2 Unified cache	128 KiB to 2 MiB	16-way set associative; LRU	124

Performance



1. 坐标轴与图例

- **横轴**：12 个 SPEC CPU2006 整数基准测试程序 (hmmer、h264ref、libquantum 等)，覆盖不同类型的计算负载
- **纵轴**：Cache 缺失率 (Miss Rate)，范围 0%~40%
- **红色柱**：L1 数据 Cache 缺失率
- **蓝色柱**：L2 数据 Cache 缺失率

2. 核心规律

1. **层级差异显著**：所有程序的 L1 缺失率均远高于 L2 缺失率，符合存储层次的设计预期：

- L1 Cache 容量小、速度快，作为第一级缓冲，承担大部分访问，缺失率相对更高
- L2 Cache 容量大、作为 L1 的后备缓冲，仅处理 L1 未命中的请求，因此缺失率极低（多数程序 <1%）

2. **负载特性分化明显**：

- **低缺失率负载**：hmmer、h264ref、libquantum 等，L1 缺失率 <2%，L2 几乎为 0，属于**数据局部性好的**程序（如多媒体、科学计算）
- **高缺失率负载**：mcf 是极端案例，L1 缺失率高达~37%，L2 缺失率也达到~9%，属于**数据局部性极差**的程序（典型的稀疏图算法，随机访存多）
- **中间负载**：astar、omnetpp 等，L1 缺失率 10% 左右，L2 缺失率 1%~3%，属于局部性中等的程序

二、关键程序解读（典型负载）

表格 📄 ⬇️ 🔗			
程序	L1 缺失率	L2 缺失率	负载特性
mcf	~37%	~9%	稀疏图最短路径算法，随机访存密集，数据局部性极差，是 Cache 性能的“杀手”负载
omnetpp	~10.5%	~3%	离散事件网络仿真，数据结构复杂，访存不规则
astar	~9.5%	~1.5%	路径规划算法，图遍历访存，局部性一般
hmmer	~1%	~0.1%	基因序列比对，数据局部性好，顺序访存多
h264ref	~1.5%	~0.1%	视频编码，数据访问规律，局部性优秀

三、体系结构视角的深层解读

1. 全局缺失率的意义

- **全局缺失率** = 该层级 Cache 未命中的次数 / 总访存次数，是评估存储层次对程序性能影响的核心指标
- 缺失率直接决定平均内存访问时间 (AMAT)：

$$AMAT = L1\ Hit\ Time + L1\ Miss\ Rate \times (L2\ Hit\ Time + L2\ Miss\ Rate \times Memory\ Access\ Time)$$

以 `mcf` 为例，极高的 L1/L2 缺失率会导致 AMAT 大幅上升，程序性能严重受限于内存带宽

2. 存储层次设计的验证

- 该图验证了**两级 Cache 层次的有效性**：L2 成功吸收了绝大多数 L1 未命中的请求，仅极少数请求最终穿透到主存
- 同时也暴露了**局部性对性能的决定性影响**：即使是同一处理器，不同程序的 Cache 缺失率差距可达数十倍，程序优化（提升数据局部性）的收益远大于硬件参数调整

四、补充说明

- 该测试通常用于**嵌入式 / 低功耗处理器**（如 ARM Cortex-A53）的性能评估，这类处理器的 L1/L2 Cache 容量较小，因此缺失率特征更明显
- 图中数据符合 SPEC CPU2006 的经典测试结果：`mcf` 始终是整数测试中 Cache 缺失率最高的程序，而多媒体类程序（h264ref）的局部性最优

Intel Core i7-6700

- 64-bit x86-64 ISA
- out-of-order 4-core processor
- multiple issue, dynamically scheduled, 16-stage pipeline
- up to 4 instructions per cycle per core

Intel Core i7-6700

- 64-bit x86-64 ISA
- out-of-order 4-core processor
- multiple issue, dynamically scheduled, 16-stage pipeline
- up to 4 instructions per cycle per core
- 4.0 GHz
- peak rate of 16 billion instructions per second per core

Intel Core i7-6700

- up to 3 parallel memory channels
- DDR3-1066
- peak memory bandwidth of 25+ GB/S
- 48-bit virtual addresses
- 36-bit physical addresses → 36 GiB
- two-level TLB

Intel Core i7-6700

- 48-bit virtual addresses
- 36-bit physical addresses → 36 GiB
- two-level TLB

Characteristic	Instruction TLB	Data DLB	Second-level TLB
Entries	128	64	1536
Associativity	8-way	4-way	12-way
Replacement	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU
Access latency	1 cycle	1 cycle	8 cycles
Miss	9 cycles	9 cycles	Hundreds of cycles to access page table

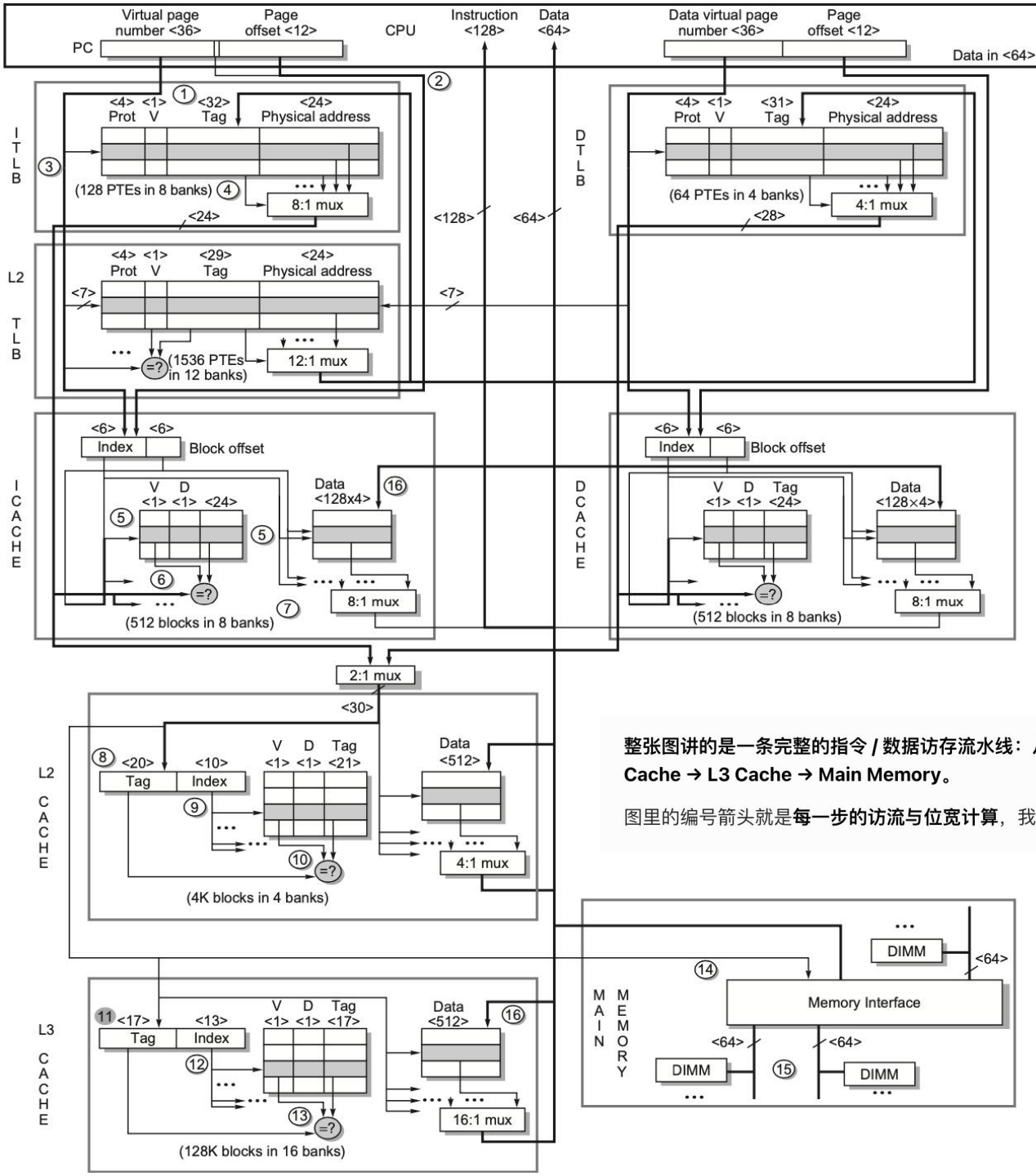
Intel Core i7-6700

- three-level cache hierarchy
- L1: virtually indexed & physically tagged
- L2 & L3: physically indexed

Characteristic	L1	L2	L3
Size	32 KiB I/32 KiB D	256 KiB	2 MiB per core
Associativity	both 8-way	4-way	16-way
Access latency	4 cycles, pipelined	12 cycles	44 cycles
Replacement scheme	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU but with an ordered selection algorithm

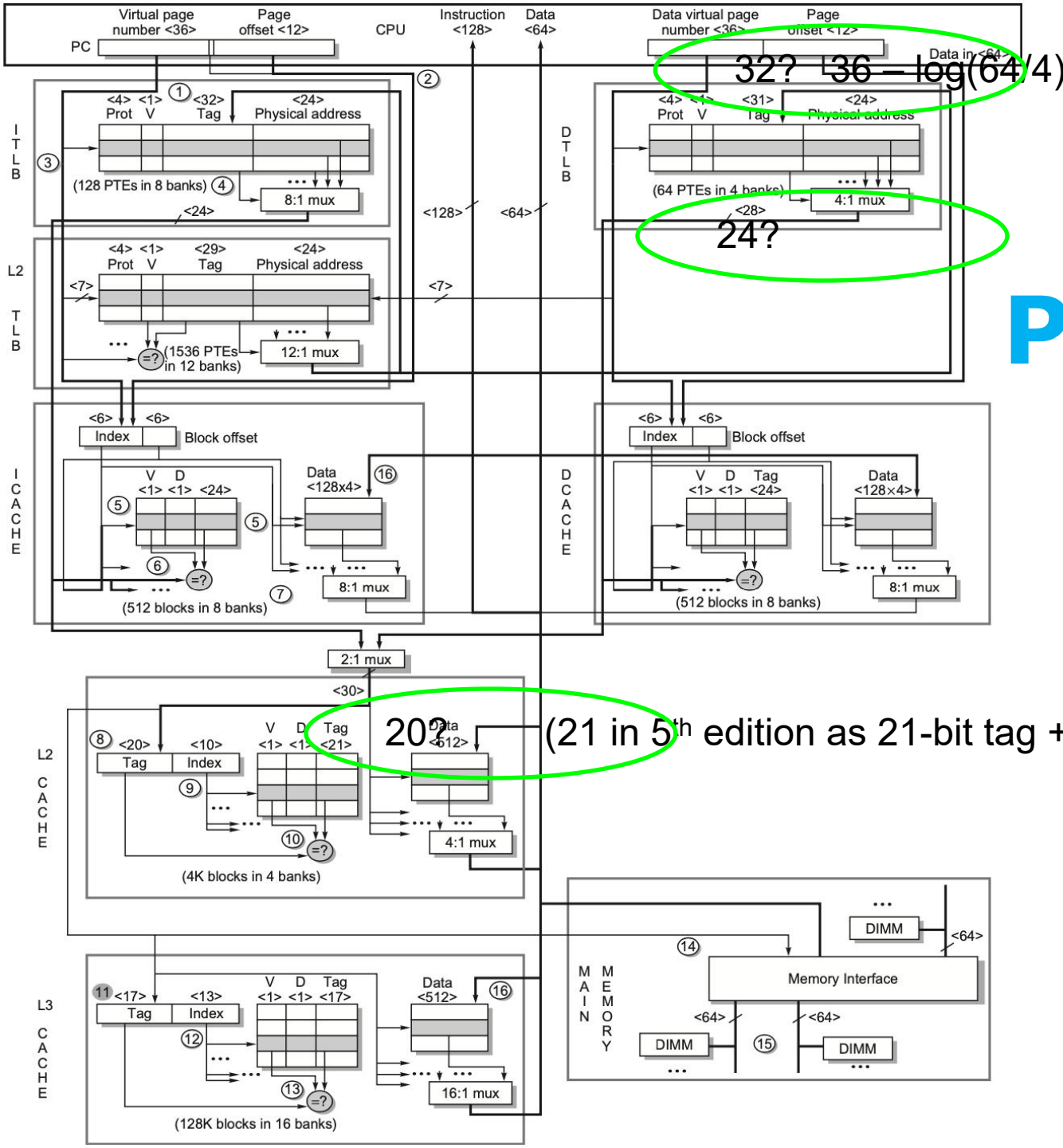
Memory Access

P134-137



整张图讲的是一条完整的指令 / 数据访存流水线：从 ITLB/DTLB → L2 TLB → L1/I Cache → L1/D Cache → L2 Cache → L3 Cache → Main Memory。

图里的编号箭头就是每一步的访流与位宽计算，我按模块给你全讲透。



Memory Access

P134-137

(21 in 5th edition as 21-bit tag + 9-bit index)

一、地址与数据位宽（最上方）

1. 指令访存路径（PC → ITLB → I-Cache）

表格

字段	位宽	说明
PC	<36>	程序计数器（36 位虚拟地址）
Virtual page number	<36>	虚拟页号（36 位）
Page offset	<12>	页内偏移（4KB 页 = 2^{12} ）
Instruction	<128>	指令宽度（128 位）

2. 数据访存路径（CPU → DTLB → DCache）

表格

字段	位宽	说明
Data virtual page number	<36>	数据虚拟页号
Page offset	<12>	页内偏移
Data	<64>	数据宽度（64 位，即 8 字节）

↓

二、TLB 层级（地址转换快表）

1. ITLB（指令 TLB）

- 结构：128 个表项（PTEs），8 组（8 banks）
- 字段：
 - TLB tag <4>：保护 / 标记位（4 位）
 - Tag <32>：物理页号高位（32 位）
 - Physical address <24>：拼接后的物理地址（24 位）
- 多路选择：8:1 mux（8 组选 1）

2. DTLB（数据 TLB）

- 结构：64 个表项，4 组（4 banks）
- 字段：
 - TLB tag <4>：保护位（4 位）
 - Tag <31>：物理页号高位（31 位）
 - Physical address <24>：物理地址（24 位）
- 多路选择：4:1 mux（4 组选 1）

3. L2 TLB（二级快表）

- 结构：1536 个表项，12 组（12 banks）
- 字段：
 - Tag <7>：物理地址高位（7 位）
 - Physical address <7>：物理地址（7 位）
- 多路选择：12:1 mux（12 组选 1）

1. L1 指令缓存 (I-Cache)

- 容量: 32KB (512 块 × 8 组)
- 块大小: 64B (2^6)
- 结构: 512 块, 8 组 (8 banks)
- 字段:
 - Index <10>: 索引 (10 位)
 - Block offset <6>: 块内偏移 (6 位)
 - Tag <19>: 标记位 (19 位)
 - Data <128x4>: 数据宽度
- 多路选择: 8:1 mux

2. L1 数据缓存 (D-Cache)

- 容量: 32KB (512 块 × 8 组)
- 块大小: 64B
- 结构: 512 块, 8 组 (8 banks)
- 字段:
 - Index <10>: 索引 (10 位)
 - Block offset <6>: 块内偏移
 - Tag <19>: 标记位 (19 位)
 - Data <128x4>: 数据宽度
- 多路选择: 8:1 mux

3. L2 缓存 (统一缓存)

- 容量: 32KB (4 组 × 4 banks)
- 块大小: 512B
- 字段:
 - Index <10>: 索引 (10 位)
 - Tag <17>: 标记位 (17 位)
 - Data <512>: 数据宽度 (512 位)
- 多路选择: 4:1 mux

4. L3 缓存 (最后一级缓存)

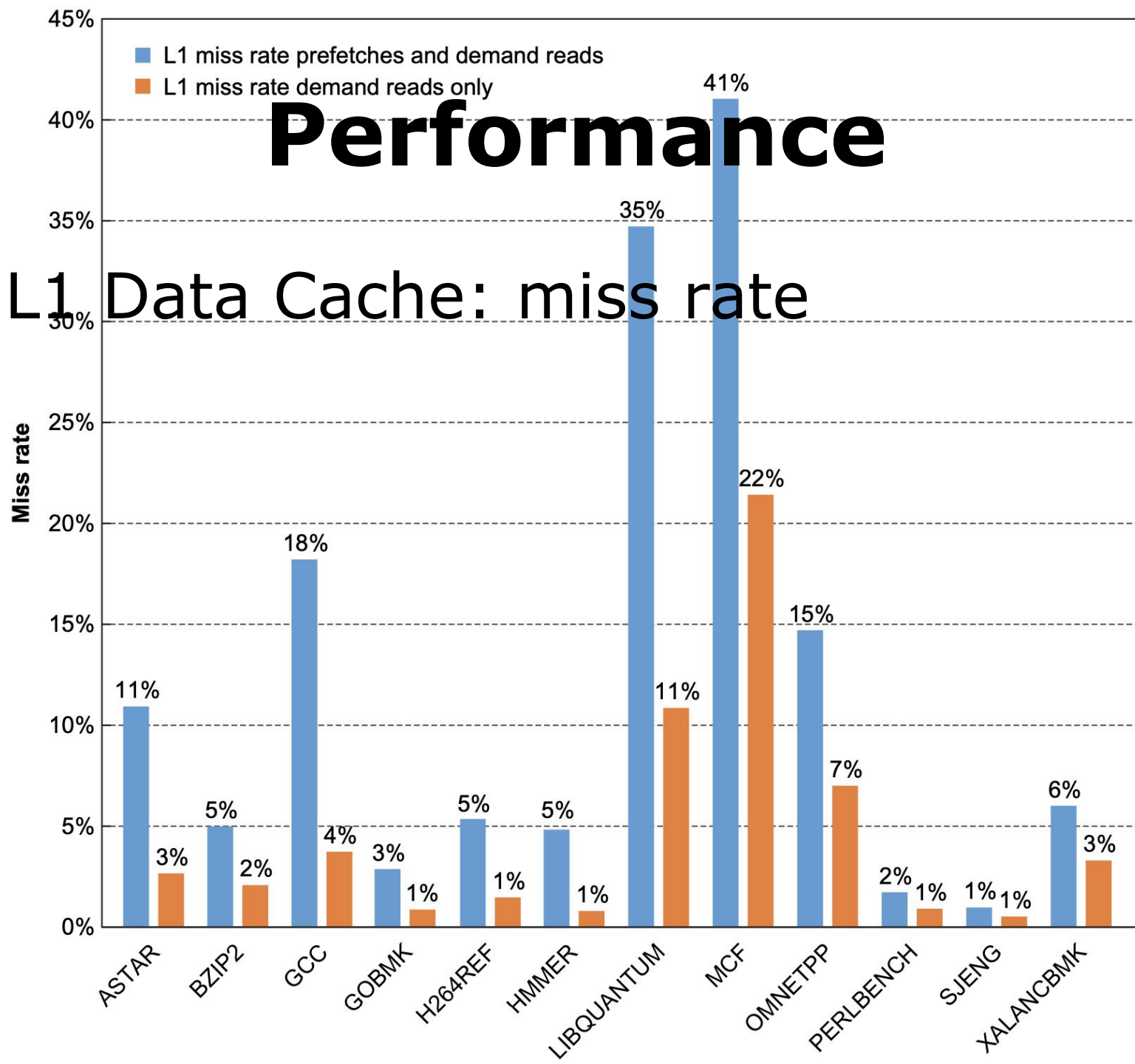
- 容量: 128KB (128K 块, 16 组)
- 块大小: 512B
- 字段:

Performance

- L1 Instruction Cache:
given 32 KiB, 8-way set associative
instruction cache
fetch 16 bytes every cycle (roughly 4.5
instructions on average)
under 1% miss rates for most of
SPECInt2006 benchmarks
2.9% miss rate for XALANCBMK

Performance

- L1 Data Cache: miss rate

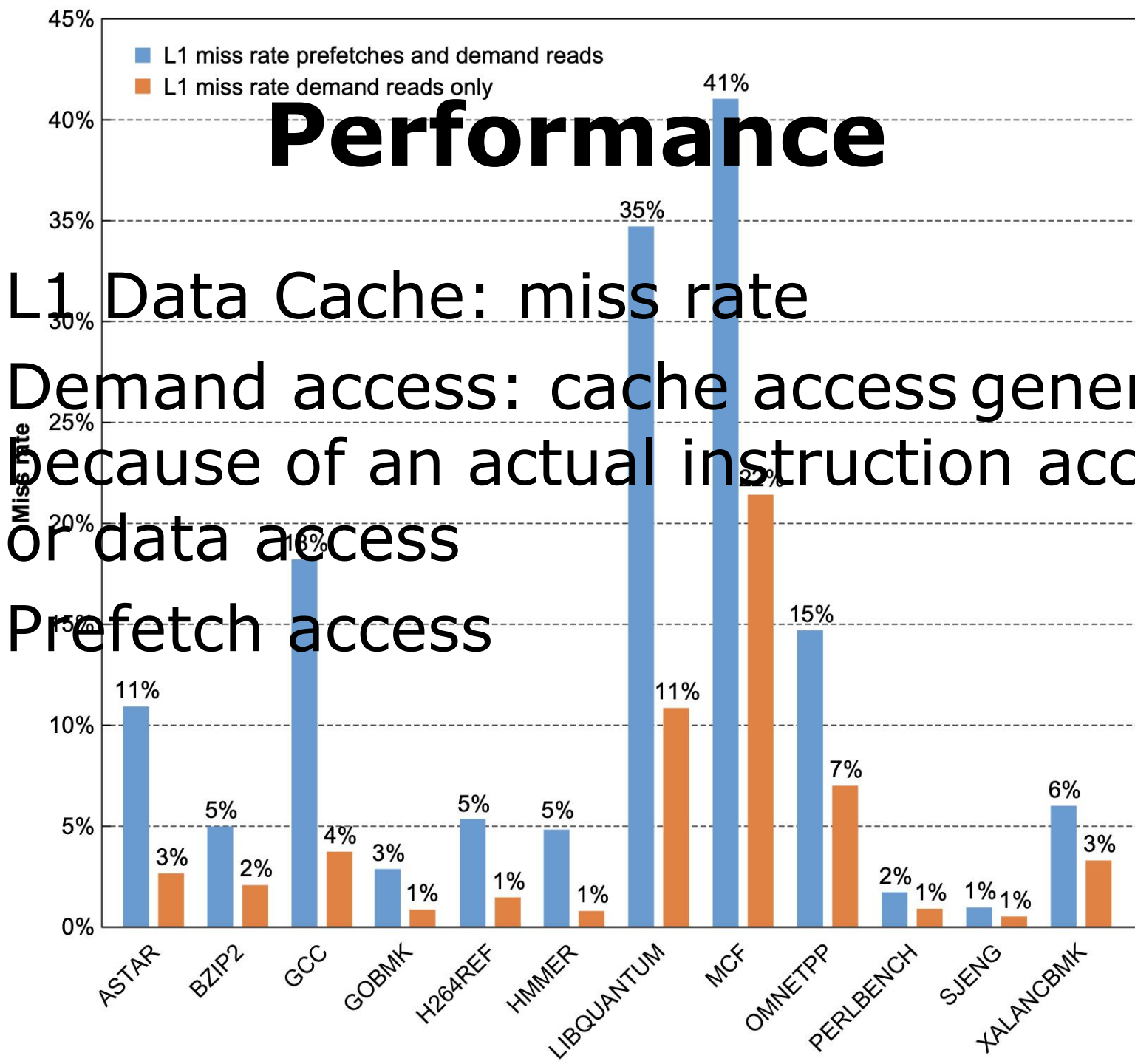


2. 核心规律

1. **预取会显著“拉高”总缺失率**：所有程序的蓝色柱（含预取）都远高于橙色柱（仅按需），因为预取会引入大量**假缺失**（块在传输中，被记为缺失）
 2. **负载特性决定预取效果**：
 - **顺序访存 / 局部性好的程序**（如 H264REF、GOBMK）：真实缺失率极低（<2%），预取的假缺失占比高，但实际性能收益大（预取提前加载数据，减少按需缺失）
 - **随机访存 / 局部性差的程序**（如 MCF、LIBQUANTUM）：真实缺失率本身极高，预取不仅无法有效降低真实缺失，还引入大量假缺失，总缺失率飙升
 3. **MCF 是极端案例**：总缺失率 41%，真实缺失率 22%，是 SPEC CPU2006 中 Cache 性能最差的程序，预取对其几乎无收益
-

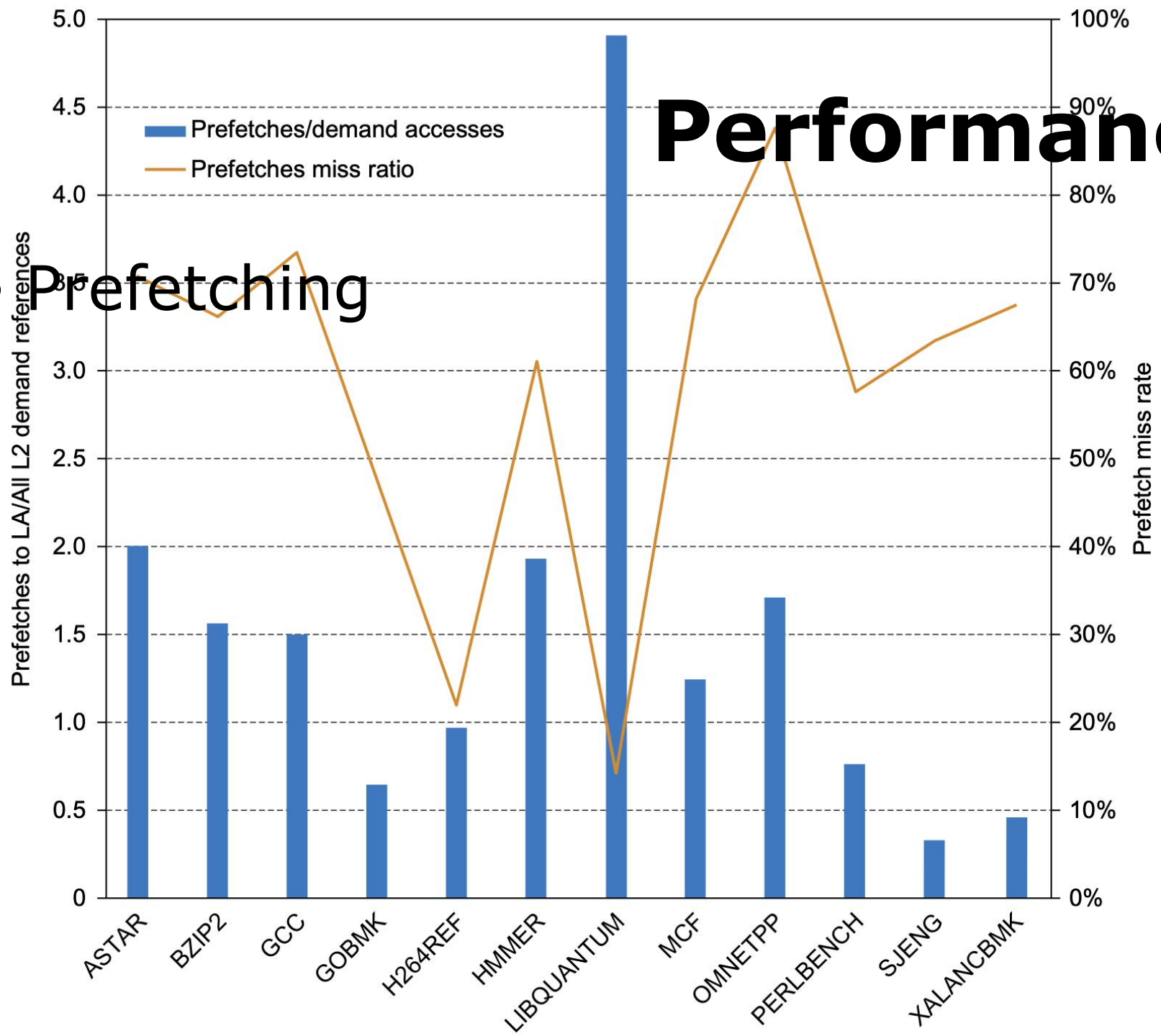
Performance

- L1 Data Cache: miss rate
- Demand access: cache access generated because of an actual instruction access or data access
- Prefetch access



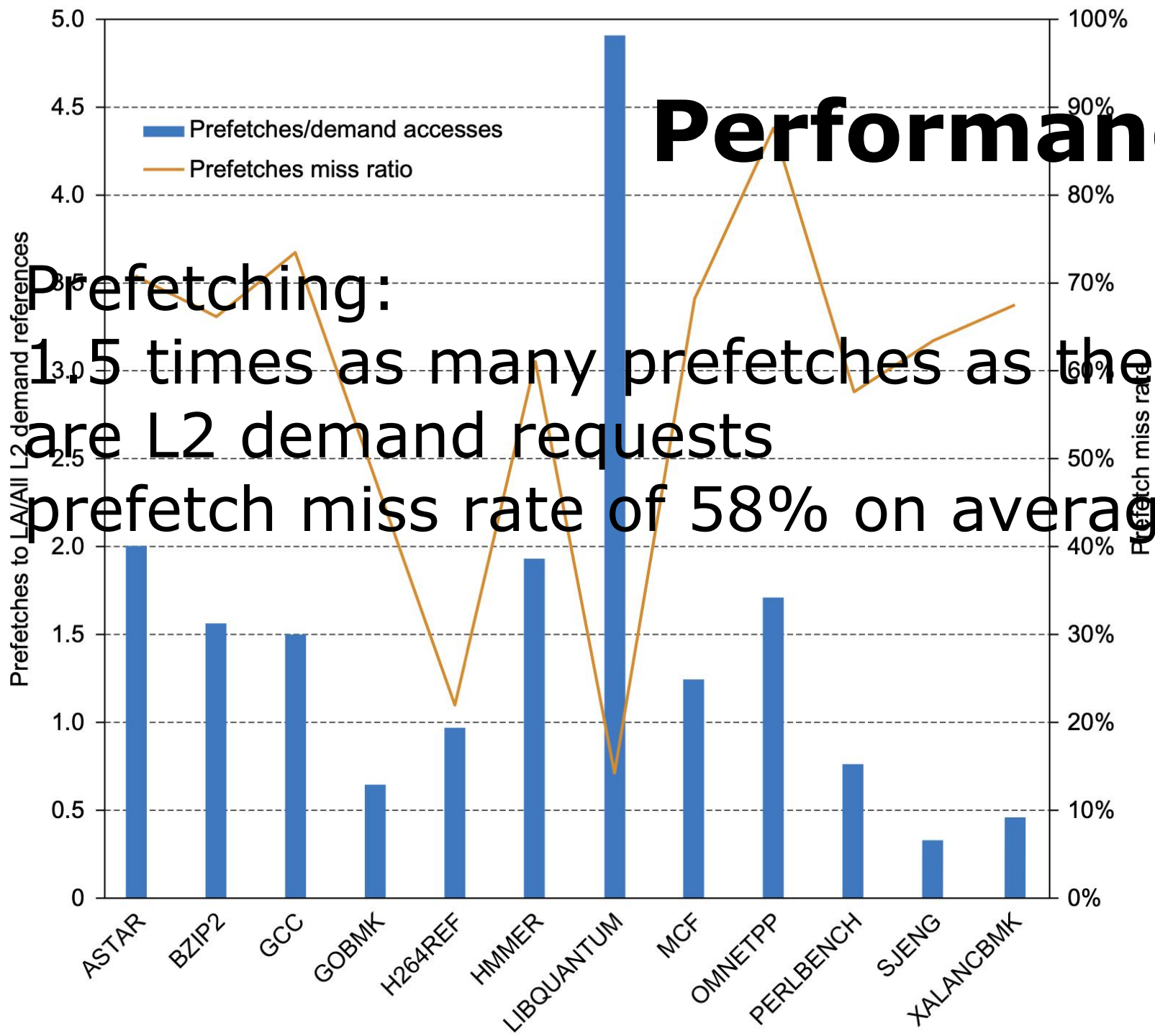
Performance

● Prefetching



Performance

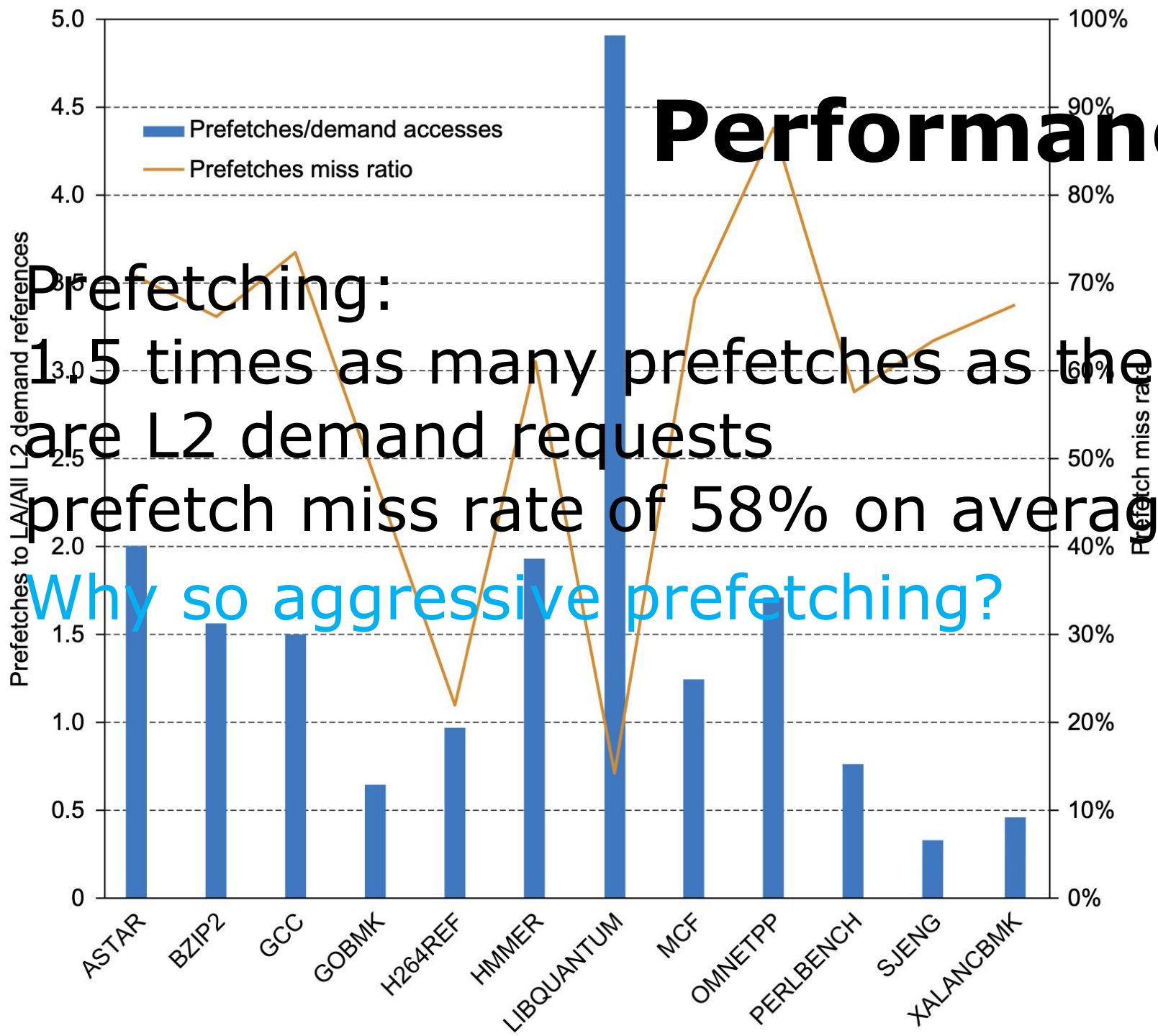
- Prefetching:
 - 1.5 times as many prefetches as there are L2 demand requests
 - prefetch miss rate of 58% on average



Performance

- Prefetching:
 - 1.5 times as many prefetches as there are L2 demand requests
 - prefetch miss rate of 58% on average

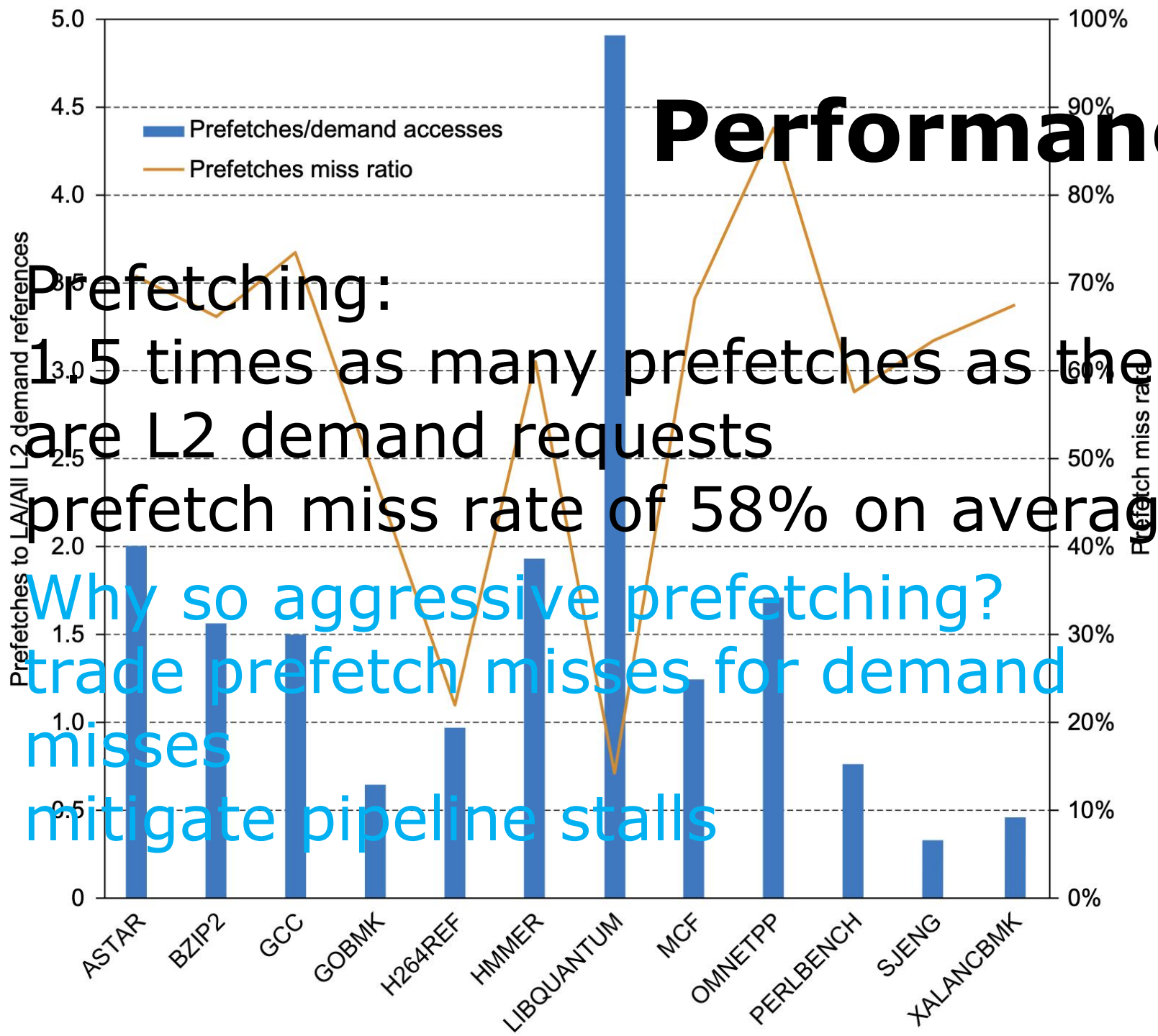
- Why so aggressive prefetching?



Performance

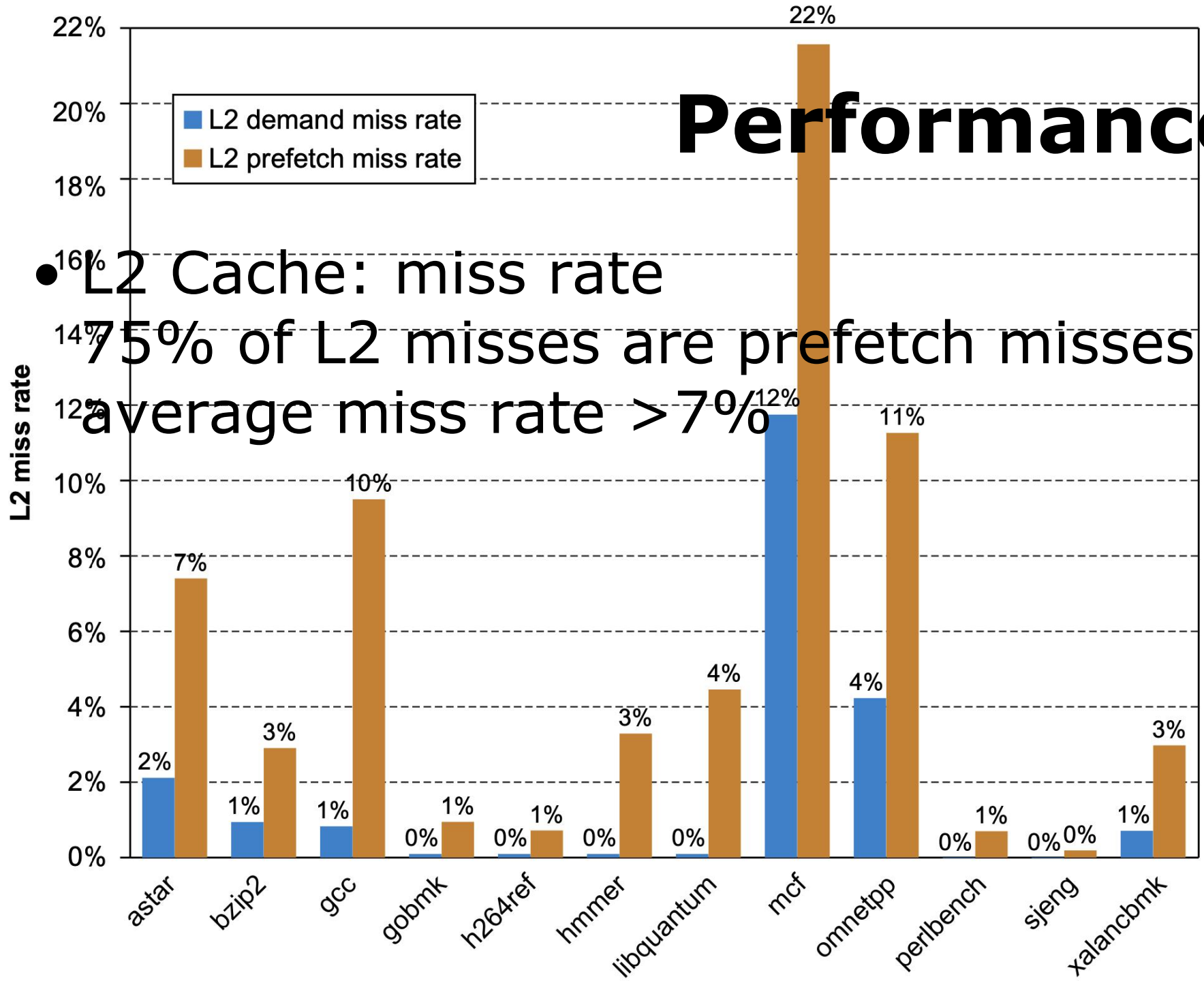
- Prefetching:
 - 1.5 times as many prefetches as there are L2 demand requests
 - prefetch miss rate of 58% on average

- Why so aggressive prefetching?
 - trade prefetch misses for demand misses
 - mitigate pipeline stalls

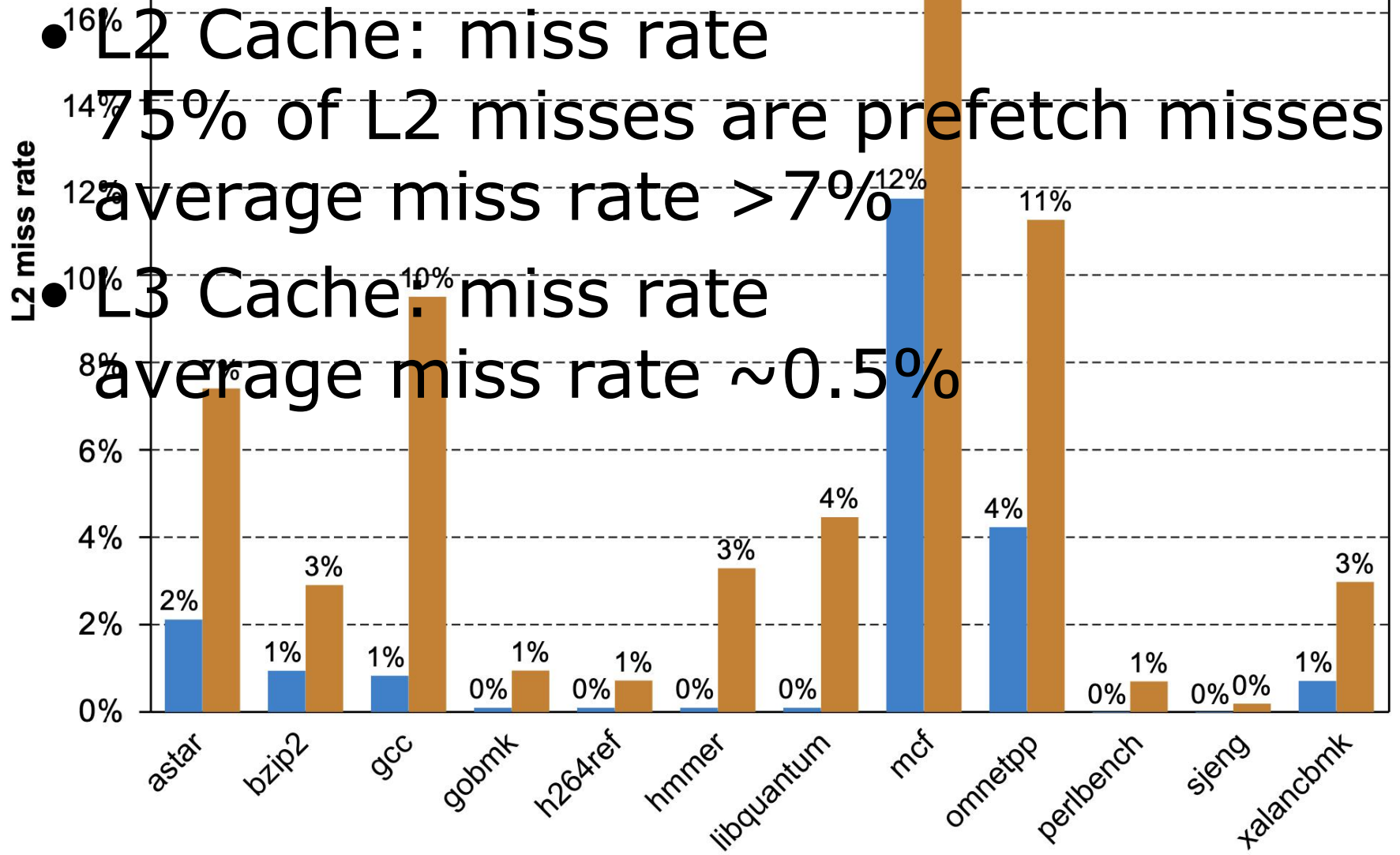


Performance

- L2 Cache: miss rate
- 75% of L2 misses are prefetch misses
- average miss rate > 7%



Performance

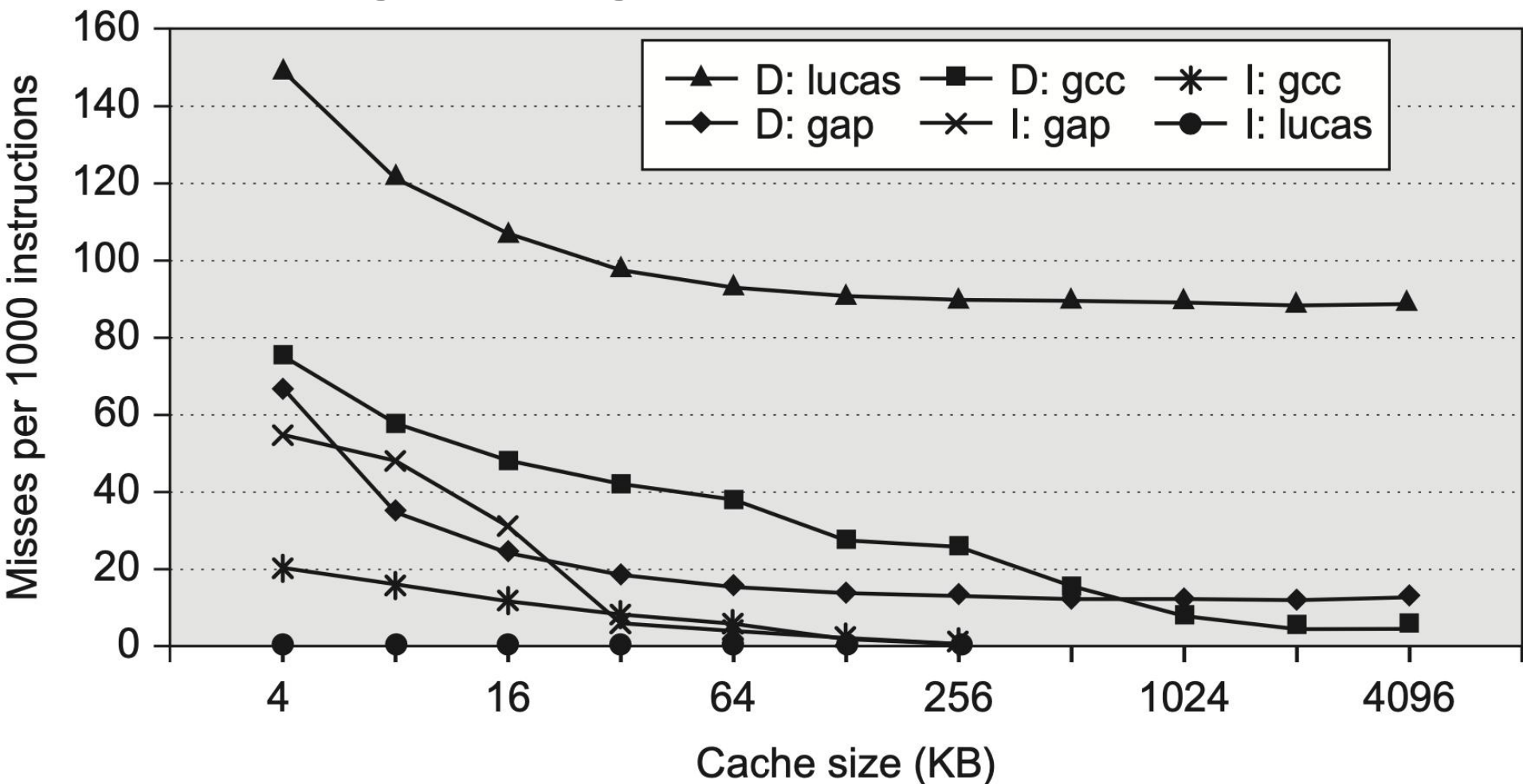


**more on
performance measurement**

Why Many Benchmarks?

No single program represents all: 没有任何一个程序能代表所有负载的 Cache 性能，不同程序的缺失率随 Cache 大小的变化规律天差地别。

- No single program represents all



2. 指令 vs 数据 Cache 的差异

- **指令 Cache (I: xxx)**：所有程序的指令缺失率都远低于数据 Cache，且随 Cache 扩容下降更快，最终趋近于 0（指令局部性天然优于数据）
- **数据 Cache (D: xxx)**：不同程序的缺失率差距极大（lucas 是 gcc 的 2 倍 +），是 Cache 性能的核心瓶颈

三、深层解读：为什么必须用多个基准测试？

1. 单一程序会导致完全错误的结论

- 若仅用 `lucas` 评估：会认为「Cache 扩容到 64KB 就足够，再大没有收益」，但对 `gcc/gap` 来说，64KB Cache 仍有极高缺失率
- 若仅用 `gcc` 评估：会认为「Cache 必须扩容到 1MB 以上才有明显收益」，但对 `lucas` 来说，64KB 就已达到性能瓶颈
- 若仅用指令 Cache 评估：会认为「小 Cache 就足够」，完全忽略数据 Cache 对性能的决定性影响

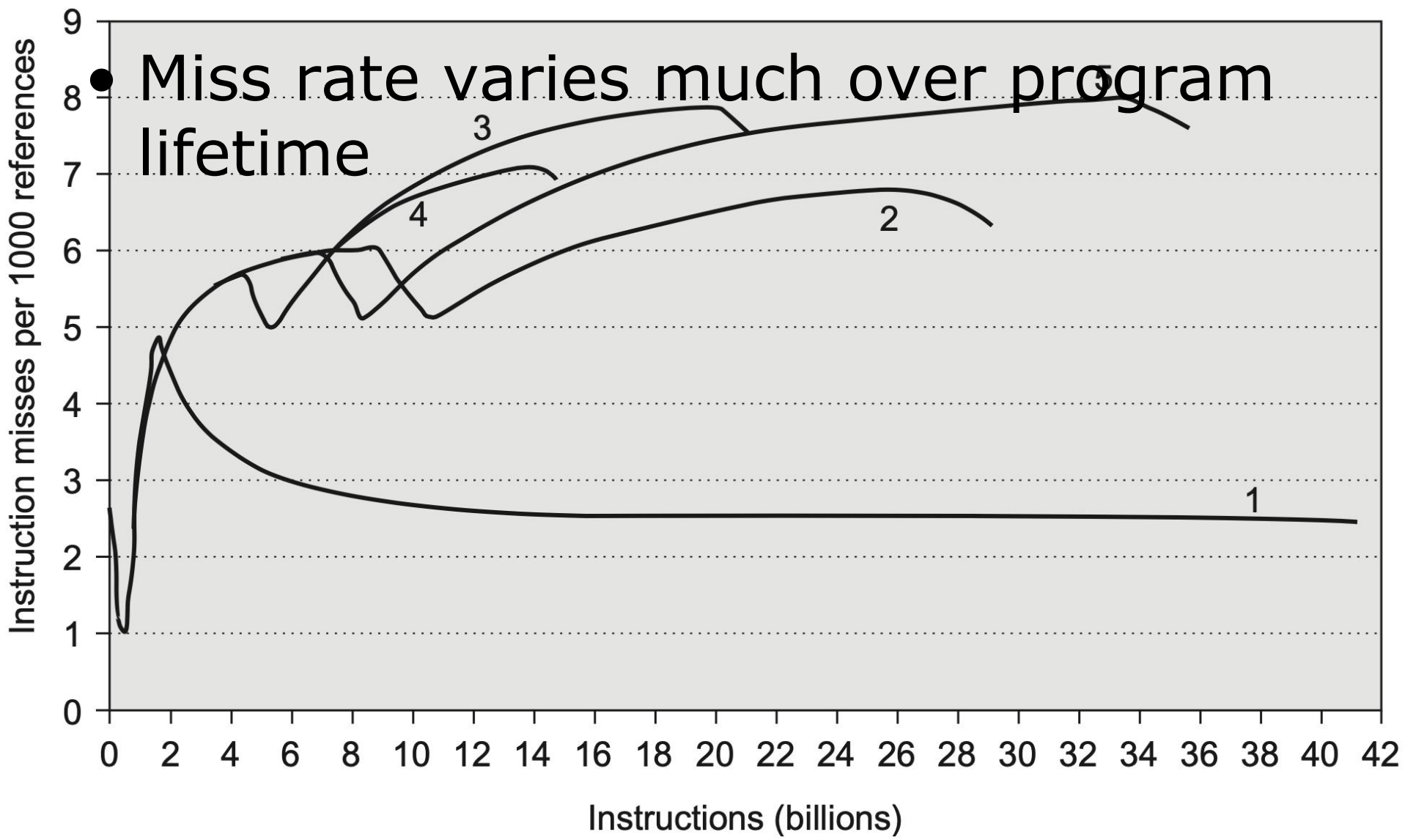
2. 负载特性决定 Cache 设计需求

- **科学计算负载 (lucas)**：需要大的数据 Cache，指令 Cache 要求极低
- **编译器 / 代码负载 (gcc)**：需要均衡的指令 + 数据 Cache，且容量需求大
- **文本处理负载 (gap)**：需要大的指令 Cache，数据 Cache 要求中等
- 处理器的 Cache 设计必须覆盖所有典型负载，不能仅针对某一类程序优化

3. 对体系结构设计的启示

- **Cache 大小的收益不是线性的**：不同程序的「性能拐点」完全不同（lucas 在 64KB，gcc 在 1MB）
- **必须用多基准测试集评估**：SPEC CPU 等标准测试集包含数十个程序，就是为了覆盖不同负载特性，避免单一程序的偏见
- **指令 / 数据 Cache 分离设计的必要性**：两类 Cache 的缺失率规律完全不同，分离式设计可针对性优化容量、相联度等参数

Why Even More Instructions?



2. 曲线说明

图中 4 条曲线（标注 1/2/3/4）代表 perlbnk 的**5 组不同输入**的运行平均缺失率（图注补充）：

- 曲线 1：初始缺失率快速下降，最终稳定在~2.5 次 / 千次引用
- 曲线 2/3/4：初始缺失率快速上升，最终稳定在 6~8 次 / 千次引用，且不同输入的最终缺失率差异极大

二、关键规律与深层解读

1. 初始阶段的误导性

- **前 19 亿指令（图注明确）**：所有输入的缺失率几乎一致，范围仅 2.5%~4.7%，看似「不同输入对性能影响极小」
- **完整运行（160~410 亿指令）**：缺失率出现剧烈分化，最高（曲线 3）与最低（曲线 1）的缺失率差距超过 3 倍，完全暴露了不同输入的真实性能差异

2. 为什么会出现这种分化？

- **程序的阶段特性**：perlbnk 是 Perl 语言解释器，程序运行分为**初始化阶段**和**核心计算阶段**：
 - 初始化阶段：所有输入的代码加载、初始化逻辑高度相似，因此缺失率一致
 - 核心计算阶段：不同输入的正则匹配、字符串处理逻辑完全不同，指令访存局部性天差地别，导致缺失率剧烈分化
- **冷启动效应**：程序启动时 Cache 为空，初始缺失率高，随着数据 / 指令加载，缺失率逐渐稳定，但稳定后的真实性能由核心计算阶段决定

3. 对基准测试的核心启示

1. **短指令流会得出完全错误的结论**：仅用前 19 亿指令评估，会认为「perlbnk 的指令 Cache 缺失率稳定在 3% 左右，不同输入无差异」，完全忽略了核心计算阶段的真实性能
2. **必须用完整程序生命周期评估**：只有运行到程序结束，才能捕捉到不同输入、不同阶段的真实缺失率，避免「初始化阶段的相似性掩盖核心阶段的差异」
3. **基准测试的输入选择至关重要**：即使是同一个程序，不同输入的访存特性也可能天差地别，必须覆盖典型输入，不能仅用单一输入评估

一、图注补充的实验细节